

A.A. 2026-2027

# Reti e Programmazione Distribuita

# Indice


0 Note sul corso 3

I Fondamenti 4

1 Introduzione alle reti di calcolatori 5
- Rete di calcolatori 5
- Internet 5
- Componenti di una rete 5
- Comunicazione dei dati 6
- Topologie di rete 6
- Categorie di reti 7
- ISP e accesso a Internet 8

2 Internet protocolli e livelli 9
- Vista "nuts and bolts" di Internet 9
- Protocolli e standard 9
- Il modello a livelli 9
- Servizi di connessione e affidabilità 10
- Il modello ISO/OSI 10
  1. Livello fisico 11
  2. Livello data link 11
  3. Livello di rete 11
  4. Livello di trasporto 11
  5. Livello di sessione 12
  6. Livello di presentazione 12
  7. Livello applicazione 12
- Incapsulamento 13
- Critiche e confronto TCP/IP vs OSI 13

II Application Layer 14

3 Application Layer 15
- Applicazioni di rete 15
- Architetture delle applicazioni 15
- Comunicazione tra processi 16
- Servizi di trasporto 16
- Protocolli applicativi 16
- FTP e FTPS 17

4 Application Layer HTTP 18
- Web e HTTP 18
- URL 18
- Il protocollo HTTP 19
- Stateless 19
- Connessioni persistenti e non persistenti 19
  - Connessione non persistente 19
  - RTT e tempi di risposta 20
  - Connessione persistente 21
  - Confronto persistente vs non persistente 21

5 HTTP messaggi cookie e web caching 23
- HTTP riepilogo 23
- Formato dei messaggi 23
  - Messaggio di richiesta 23
  - Messaggio di risposta 24
- Cookie 25
- Web caching 26
- Oltre il caching 27

6 Application Layer posta e-mail e P2P 28
- Posta elettronica e SMTP 28
- Accesso alla posta 28
  - POP3 29
  - IMAP 29
  - Accesso via HTTP 29
- Peer-to-Peer 29
- BitTorrent 29
  - Il tracker 30
  - Strategie del client 30

7 Application Layer DNS 31
- Identificazione degli host 31
  - Resolver 31
- Il name space DNS 31
- Resource record 32
  - Tipi di record principali: 32
- Name server e zone 33
  - Name resolution 33
- Il resolver locale e i servizi aggiuntivi 34
- Strumenti di laboratorio 35
  - `nslookup` 35
  - `dig` (Domain Information Groper) 36

8 Lezione 8 Programmazione di rete 38
- Applicazioni di rete 38
- Dove sono implementati i livelli 38
- Il socket 39
  - Procedure client-server 39
- Creazione, chiusura e associazione 39
  - `socket()` 39
  - `close()` 40
  - `bind()` 40
- Indirizzi IP, porte e ordine dei byte 41
  - Strutture degli indirizzi 41

Ordine dei byte 41
`inet_pton()` 42

## 9 Lezione 9 Programmazione con UDP 43

`sendto()` e `recvfrom()` 43
Parametri 43
Comportamento *Blocking* di default 44
Flusso di un server UDP 44
Binding implicito 46
Ordine di avvio di client e server 46
Timeout di ricezione 47
Più client su un solo socket 47
Comunicazione su localhost 47
Buffer dei socket UDP 47

## 10 Lezione 10 Programmazione con TCP (Connessioni e Flusso) 49

Dal socket UDP al modello TCP 49
`connect()` 49
Parametri 49
`listen()` e `accept()` 50
Chiusura a quattro segmenti 50
Lettura e scrittura 51
Opzioni del socket 51
Esempi pratici 52
Client Code 53
Server Code 53
Buffer dei socket TCP 53
Modalità non bloccante 54
Lettura e scrittura complete 54

## 11 Lezione 11 Server Concorrenti e Multiplexing I/O 56

Server a processi con `fork()` 56
Server multithreaded 57
Più connessioni sulla stessa porta 57
Multiplexing I/O con `select()` 57
Insiemi di descrittori 58
Condizioni di prontezza 58
Timeout 59
Server e client multiplexing 59

## 12 Lezione 12 Broadcast, Multicast e Risoluzione dei Nomi (DNS) 60

Broadcast 60
Multicast 60
Risoluzione dei nomi DNS 61
`gethostbyname()` 61
`getaddrinfo()` 61
Richieste HTTP su socket TCP 62

# III Transport Layer 63

## 13 Lezione 13 Il livello di trasporto 64

Comunicazione logica tra processi 64
Responsabilità del livello di trasporto 64
Multiplexing e demultiplexing 64
Demultiplexing senza connessione in UDP 65
Porta di destinazione non raggiungibile 65
Demultiplexing in TCP 65
Porte 66
Ispezione delle porte con nmap e ss 66
UDP 67
Perché UDP invece di TCP 67
Uso di UDP 67
Formato del datagram 68
Checksum 68
Consegna, perdita e ordinamento 69
Buffer delle socket e perdita dei datagram 69
Percorso di ricezione in Linux 69
Affidabilità a livello applicazione 69

## 14 Lezione 14 Trasferimento affidabile dei dati 70

Il problema del trasferimento affidabile 70
Stop-and-wait e ARQ 71
ACK numerati 71
Perdita di pacchetti e timeout 71
Prestazioni dello stop-and-wait 72
Pipelining 73
Go-Back-N 73
Selective Repeat 74
Dimensione della finestra e ambiguità 76

## 15 Lezione 15 Transport Layer TCP 77

TCP rispetto a UDP 77
Buffer e Maximum Segment Size 77
Il segmento TCP 78
Numeri di sequenza e acknowledgment 78
Stima dell'RTT e timeout di ritrasmissione 79
Fast retransmit 80
Come funziona 80
Dopo la ritrasmissione 80
TCP: Go-Back-N o selective repeat? 80

Capitolo 0

# Note sul corso

I seguenti appunti sono stati presi durante le lezioni del corso di **Reti e Programmazione Distribuita**, a.a. 2026/2027.

Gli appunti sono organizzati in cinque parti che seguono i livelli dello stack TCP/IP:

1. **Introduzione alle reti** (L01–L02): fondamenti, storia di Internet, modelli a livelli, OSI vs TCP/IP.
2. **Application Layer** (L03–L09): HTTP, e-mail, DNS, P2P, socket e programmazione di rete.
3. **Transport Layer** (L10–L14): porte e UDP, trasferimento affidabile (GBN, SR), TCP, controllo di flusso e congestione.
4. **Network Layer** (L15–L21): forwarding e router, indirizzamento IPv4/IPv6, DHCP, NAT, routing Distance-Vector e Link-State.
5. **Link Layer e Sicurezza** (L22–L26): framing e gestione degli errori, protocolli MAC, switch e ARP, crittografia, SSL/TLS, firewall e IDS.

**Nota:** gli appunti sono redatti a partire dalle slide ufficiali del corso e dalle loro trascrizioni.

# Parte I

# Fondamenti

Capitolo 1

# Introduzione alle reti di calcolatori

## Rete di calcolatori

**Definition**

**Rete di calcolatori**

**Insieme di dispositivi di calcolo autonomi ma interconnessi**: un numero elevato di computer separati che, grazie ai collegamenti, possono **scambiarsi informazioni** per svolgere un lavoro comune.

Gli usi principali delle reti di calcolatori sono:

- **Accesso all'informazione**, tipicamente secondo il modello client-server o peer-to-peer.
- **Comunicazione persona-persona** ed e-commerce.
- **Intrattenimento** e *Internet of Things* (quasi qualsiasi dispositivo può connettersi).

Nel **modello client-server** un client richiede esplicitamente informazioni a un server che le ospita; il processo client invia una richiesta in rete e attende la risposta. In un sistema **peer-to-peer** non esistono client e server fissi.

## Internet

**Definition**

**Internet**

**Rete di reti**: la più grande rete di calcolatori, tecnicamente un enorme **WAN** che collega sotto-reti e dispositivi su nazioni e continenti.

Cronologia essenziale:

- **1950 ca.**: l'idea di una rete "universale" viene teorizzata (J.C.R. Licklider).
- **1969**: l'ARPA (U.S. Dept. of Defense) finanzia **ARPANET**, inizialmente una rete chiusa di università e centri di ricerca (primi nodi: SRI, UCLA, UCSB, Utah).
- **1974**: R. Kahn e V. Cerf progettano **TCP** e **IP**, i due protocolli della *Internet protocol suite*.
- **1986**: la NSF finanzia **NSFNET**, rete basata su TCP/IP; essa e TCP/IP avviano l'idea di *network of networks*, ma entrambe le reti hanno restrizioni commerciali.
- **1990**: ARPANET viene dismessa; **1995**: NSFNET viene dismessa, rimuovendo le ultime restrizioni al traffico commerciale.

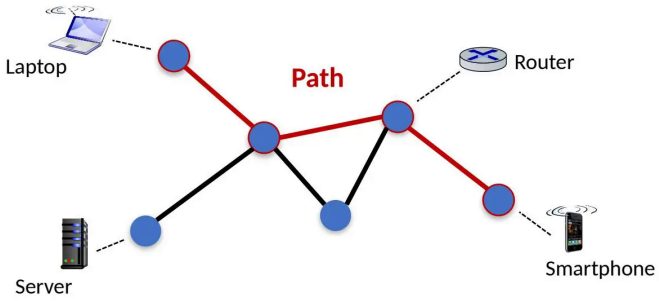
## Componenti di una rete

La rete distingue l'infrastruttura dai sistemi finali:

- **Dispositivi di rete** (*network devices*): router, switch, access point, hub, insieme ai **link** (canali di comunicazione), formano l'infrastruttura che permette ai host di connettersi.
- **Host**: i dispositivi su cui girano applicazioni o programmi (PC, laptop, smartphone, server).

Le reti condividono la terminologia della teoria dei grafi:

- I dispositivi connessi sono i **nodi**, le connessioni sono i **link** (o canali).
- Una **sequenza** di nodi e link è un **cammino** (*path*).
- I **host** (o sistemi finali) sono i nodi che producono o usano i **servizi**; idealmente sono le foglie della rete, mentre i nodi intermedi sono dispositivi di instradamento.

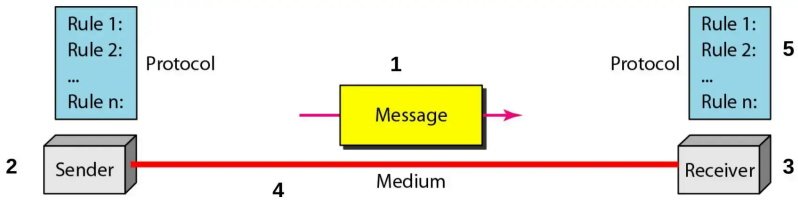



## Comunicazione dei dati

Definition

**Comunicazione dei dati**

Scambio che coinvolge cinque componenti: **messaggio** (i dati da trasmettere), **mittente**, **destinatario**, **mezzo** (canale tra mittente e destinatario) e **protocollo** (regole di comunicazione note a entrambi).




I dati si rappresentano in forme diverse: testo, numeri, immagini, audio, video. Il **flusso** dipende dal tipo e dallo scopo della comunicazione:

- **Simplex**: monodirezionale.
- **Half-duplex**: bidirezionale, ma a turni alterni.
- **Full-duplex**: bidirezionale simultaneo.

Tre grandezze misurano il flusso, in bit/s (o byte/s):

- **Transmission rate**: quantità *massima* di informazione trasmissibile da un *dispositivo*.
- **Banda (bandwidth)**: quantità *massima* trasmissibile da un *cammino* (link e nodi).
- **Throughput**: quantità *effettiva* (istantanea) trasmessa su un cammino.

Le connessioni tra host possono essere di due tipi:

- **Point-to-point**: link dedicato tra due soli dispositivi (cablato o wireless).
- **Multipoint (broadcast)**: più di due dispositivi condividono lo stesso link.

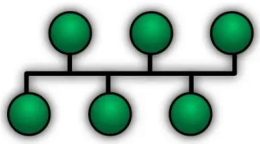
## Topologie di rete

Definition

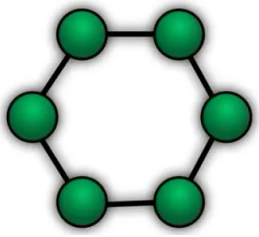
**Topologia di rete**

**Disposizione** degli elementi (nodi, link) di una rete di comunicazione.

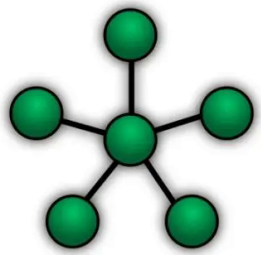
- **Bus**: gli host sono collegati a un cavo dorsale centrale; possibili **collisioni**; semplice ed economica, adatta a reti piccole, ma la dorsale è un punto singolo di guasto. Ha 1 link fisico duplex (o 1 dorsale e `n` link).



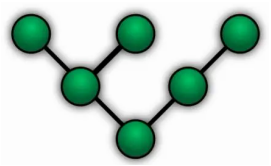
- **Ring**: ogni host è connesso point-to-point a esattamente altri due; il segnale è **inoltrato** lungo l'anello fino alla destinazione; semplice ed economica, con prestazioni migliori del bus, ma aggiungere nodi è più difficile e un guasto di un nodo può compromettere la rete; ha `n` link duplex.



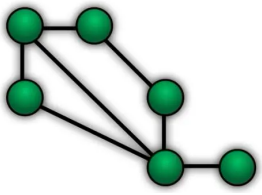
- **Star**: gli host sono collegati a un controllore centrale (hub, switch o router) che reindirizza i messaggi, senza link diretti tra host; economica, semplice, robusta e scalabile, ma il controllore deve essere raggiungibile da tutti ed è un punto singolo di guasto; ha `n` link duplex.



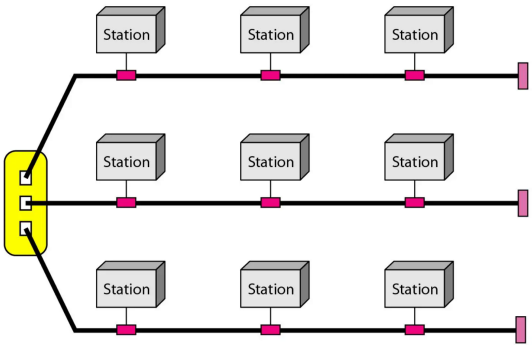
- **Tree**: più topologie a stella integrate, tipicamente tramite un cavo bus; versatile, scalabile, robusta e ben supportata, ma difficile da configurare ed eredita la debolezza del bus.



- **Mesh**: host collegati point-to-point in modo non gerarchico; nella **full mesh** tutti i nodi sono tra loro connessi ($n(n-1)/2$ link duplex), nella **partial mesh** solo alcuni; poco traffico, robusta, sicura, con link dedicati, ma difficilmente scalabile e costosa.



- **Hybrid**: topologie diverse possono essere **mixate** in una rete ibrida.




## Categorie di reti

Le reti si classificano per dimensione, numero di host e banda:

- **PAN** (*Personal Area Network*): dispositivi nel raggio di una persona (es. Bluetooth).
- **LAN/WLAN** (*Local/Wireless Area Network*): rete locale cablata (es. Ethernet switchata) o wireless (es. 802.11).
- **MAN** (*Metropolitan Area Network*): copertura metropolitana (es. distribuzione di TV e Internet tramite *cable head-end*).
- **WAN** (*Wide Area Network*): copertura di nazioni/continenti, tipicamente molto complessa ed eterogenea; la complessità della topologia cresce con la dimensione della rete.

La **rete casalinga** (home LAN) ospita una gamma ampia e diversificata di dispositivi connessi a Internet (IoT): deve essere facile da installare, sicura e affidabile, con interfacce compatibili tra tutti i prodotti.

# ISP e accesso a Internet

**Definition**

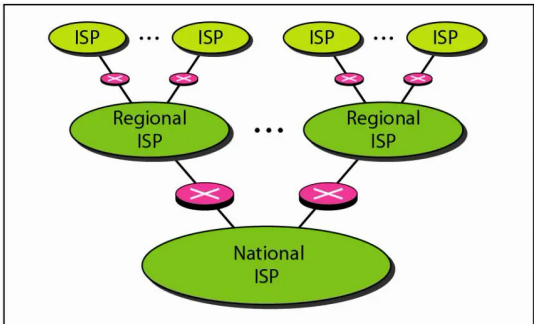
**ISP**

**Internet Service Provider**: organizzazione (commerciale, comunitaria, non-profit, privata) che fornisce servizi di accesso, uso e partecipazione a Internet.

- Le connessioni private avvengono tramite modem/ONT collegato alla rete telefonica; le aziende (medie/grandi) possono avere un collegamento diretto/dedicato all'ISP.
- Differenti ISP si scambiano dati tramite **NAP** (*Network Access Point*) o **IXP** (*Internet Exchange Point*), punti di incontro tra più ISP spesso non gestiti dagli ISP coinvolti.

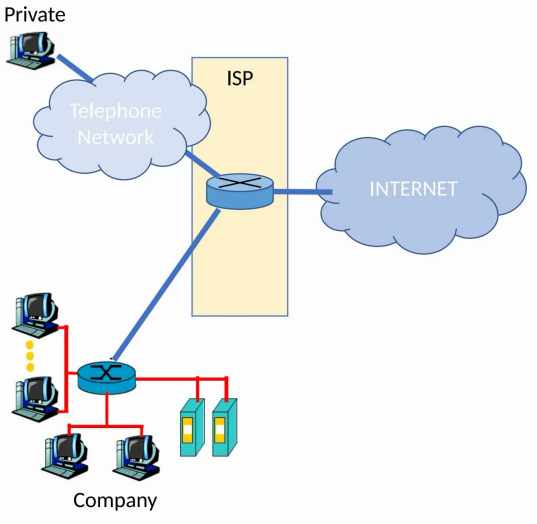
La rete di un ISP è gerarchica:

- **PoP** (*Point of Presence*): gruppo di uno o più router con cui l'ISP raggiunge i clienti.
- **Access ISP** per aree locali, **regional ISP** per aree più ampie, **national ISP** a livello di nazione.




Tecnologie di connessione privata (in ordine di banda):

- Modem analogico (56 kbps), **ISDN** (128 kbps), **ADSL** (1–20 Mbps), doppino in rame (10–100 Mbps), fibra ottica (50 Mbps–40 Gbps, tramite **ONT** senza (de)modulazione).



Capitolo 2

# Internet protocolli e livelli

## Vista "nuts and bolts" di Internet

Dalla prospettiva dell'implementazione, Internet è composta da:

- **Host** (o *end systems*): miliardi di dispositivi connessi che eseguono le applicazioni di rete al "bordo" di Internet.
- **Packet switch**: dispositivi che inoltrano pacchetti (blocchi di dati), principalmente router e switch.
- **Link di comunicazione**: fibra, rame, radio, satellite; il tasso di trasmissione è la **banda**.
- **Reti**: insiemi di dispositivi, router e link gestiti da un'organizzazione.

Info

**Internet**

Internet è una **rete di reti** di ISP interconnessi, in cui i protocolli sono ovunque: governano l'invio e la ricezione dei messaggi (es. HTTP, streaming video, TCP, IP, WiFi, 4G, Ethernet).

Dalla prospettiva dei servizi, Internet è un'**infrastruttura** che offre servizi alle applicazioni (Web, e-mail, e-commerce, social, …) e fornisce una **interfaccia di programmazione** alle applicazioni distribuite: "agganci" per connettersi al servizio di trasporto, con opzioni di servizio analoghe al servizio postale.

## Protocolli e standard

Definition

**Protocollo di rete**

**Insieme di regole** che definisce il *formato*, l'*ordine* dei messaggi scambiati tra le entità di rete e le *azioni* intraprese alla trasmissione e alla ricezione dei messaggi.

I protocolli umani (domande, presentazioni) e i protocolli di rete condividono la stessa struttura: messaggi specifici inviati e azioni specifiche compiute alla ricezione di un messaggio o di un altro evento; nei protocolli di rete, però, le entità sono computer.

Gli standard di Internet sono definiti da una comunità di esperti:

- **IETF** (*Internet Engineering Task Force*): organizzata in **working group** aperti, ciascuno focalizzato su un aspetto specifico di Internet.
- **RFC** (*Request For Comments*): documenti numerati prodotti dai working group, con descrizione e definizione di protocolli, concetti e metodi alla base di uno standard (es. IPv4 è definito nell'RFC 791, 1981).

## Il modello a livelli

Definition

**Layering**

**Approccio architetturale** alla comunicazione: il sistema è diviso in livelli (*divide et impera*), ciascuno un livello di astrazione che **usa i servizi del livello inferiore e offre servizi al livello superiore** attraverso interfacce.

Il modello a livelli serve a domare la complessità della rete (host, router, link eterogenei, applicazioni, protocolli, hardware e software):

- La struttura esplicita permette di **identificare** e mettere in relazione i pezzi del sistema complesso.
- La **modularizzazione** facilita manutenzione e aggiornamento: cambiare l'implementazione del servizio di un livello è trasparente al resto del sistema.
- **Contro**: il layering ha un costo in efficienza ("*layering considered harmful*").

Quando due dispositivi comunicano, **ogni livello del mittente comunica con lo stesso livello del destinatario** mediante un protocollo specifico.

## Servizi di connessione e affidabilità

**Definition**

**Servizio connection-oriented**

**Servizio orientato alla connessione** modellato sul sistema telefonico: l'utente prima **stabilisce** la connessione, la usa, poi la rilascia; può negoziare i parametri da usare.

**Definition**

**Servizio connectionless**

**Servizio senza connessione** modellato sul sistema postale: i pacchetti vengono inviati indipendentemente, senza setup.

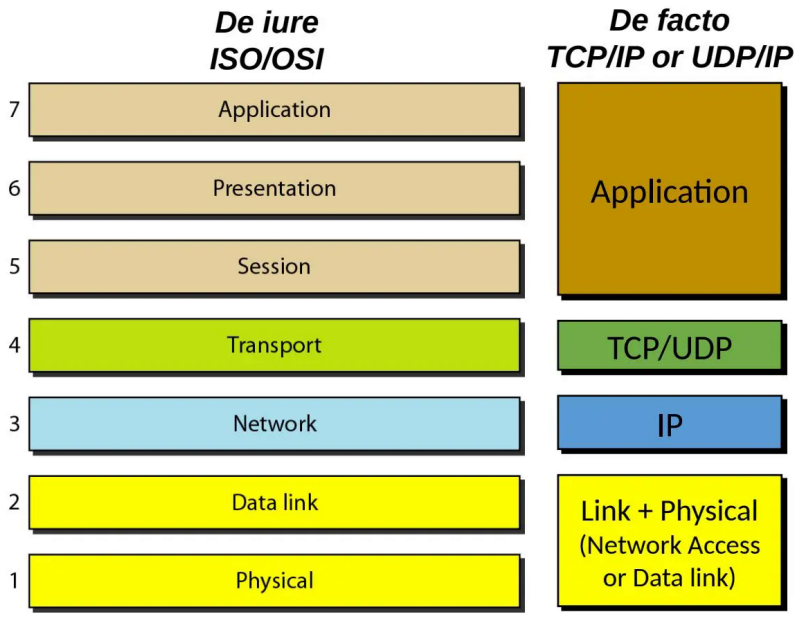
Alcune nozioni collegate:

- **Pacchetto** (*packet*): un messaggio al livello di rete.
- **Store-and-forward switching**: i nodi intermedi ricevono il messaggio per intero prima di inoltrarlo al nodo successivo.
- **Cut-through switching**: la trasmissione presso un nodo inizia prima che il messaggio sia ricevuto completamente.
- **Servizio datagram**: servizio connectionless non affidabile (senza conferme).
- L'**affidabilità** caratterizza sia i servizi orientati sia quelli non orientati alla connessione.

I sei servizi tipici orientati e non orientati alla connessione sono i seguenti.

| Tipo | Servizio | Esempio |
|---|---|---|
| Connection-oriented | Reliable message stream | Sequenza di pagine |
| Connection-oriented | Reliable byte stream | Download di un film |
| Connection-oriented | Unreliable connection | Voice over IP |
| Connectionless | Unreliable datagram | Posta elettronica di massa |
| Connectionless | Acknowledged datagram | Messaggistica testuale |
| Connectionless | Request-reply | Interrogazione di database |

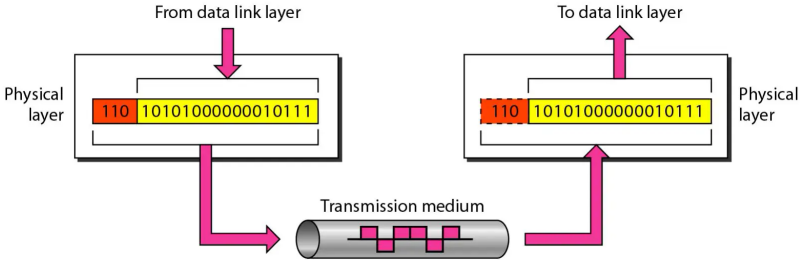
## Il modello ISO/OSI




Negli anni '80 l'**ISO** (*International Standards Organization*) ha definito il modello a livelli **OSI** (*Open System Interconnection*), con 7 livelli:

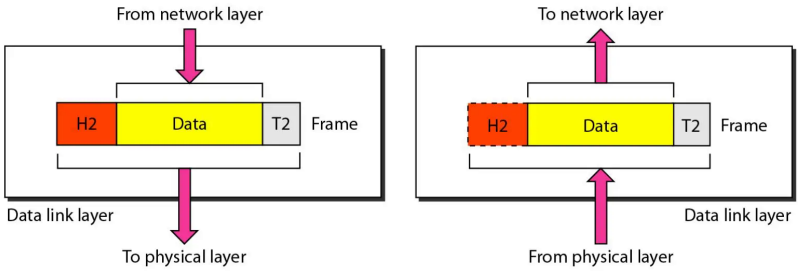
### 1. Livello fisico

Il **livello fisico** è responsabile del movimento dei singoli bit da un hop (nodo) al successivo. Regola le caratteristiche fisiche di interfacce e mezzi (connettori, cavi, segnali elettrici), la configurazione della linea (punto-punto o multipunto), la topologia fisica (mesh, star, ring o bus), la modalità di trasmissione (simplex, half-duplex o duplex), la rappresentazione dei bit, la velocità di trasmissione (*data rate*) e la sincronizzazione dei bit.



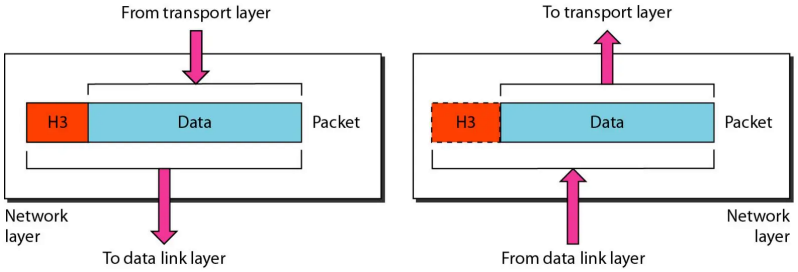

### 2. Livello data link

Il **livello data link** è responsabile del movimento dei frame da un hop al successivo (segmento). Lavora su frame, porzioni di dati tipicamente di poche centinaia di byte, e si occupa di controllo di flusso ed errori (tramite sequenze di controllo del frame) e di controllo di accesso.



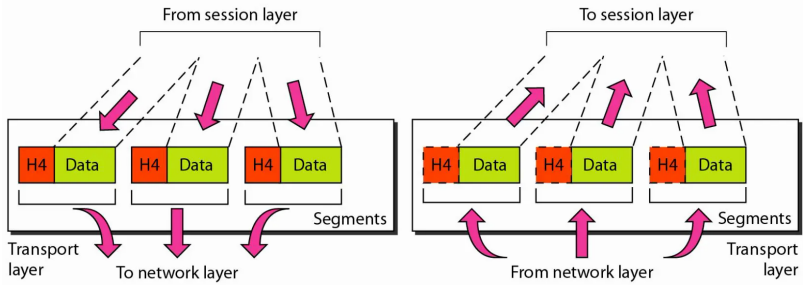

### 3. Livello di rete

Il **livello di rete** è responsabile della consegna dei singoli pacchetti dall'host sorgente all'host destinazione (cammino). Lavora su pacchetti, tipicamente più grandi e complessi dei frame, e gestisce la consegna da sorgente a destinazione, l'indirizzamento logico e il routing tramite tabelle di routing.



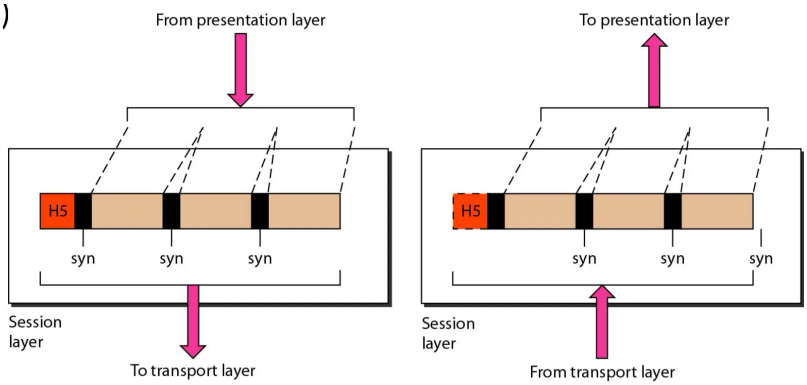

### 4. Livello di trasporto

Il **livello di trasporto** è responsabile della consegna di un messaggio da un estremo all'altro, così che il messaggio arrivi al programma giusto. Si occupa di consegna end-to-end, controllo della connessione (orientato o senza connessione), segmentazione/riassemblaggio da e verso i pacchetti del livello 3, indirizzamento delle porte e controllo di flusso ed errori.



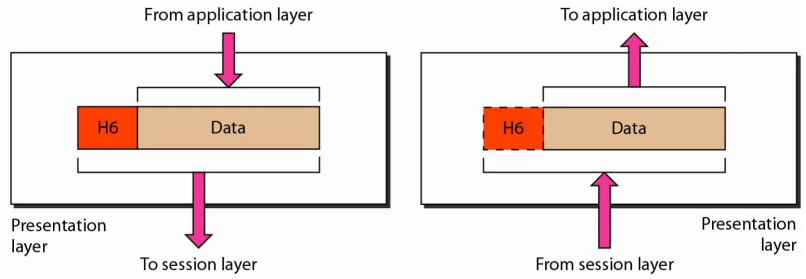

### 5. Livello di sessione

Il **livello di sessione** è responsabile del controllo del dialogo e della sincronizzazione richiesta/risposta: stabilisce, mantiene e sincronizza l'interazione tra i sistemi che comunicano (sessione di comunicazione), gestisce il dialogo e prevede punti di controllo (*checkpoint*) per la sincronizzazione.



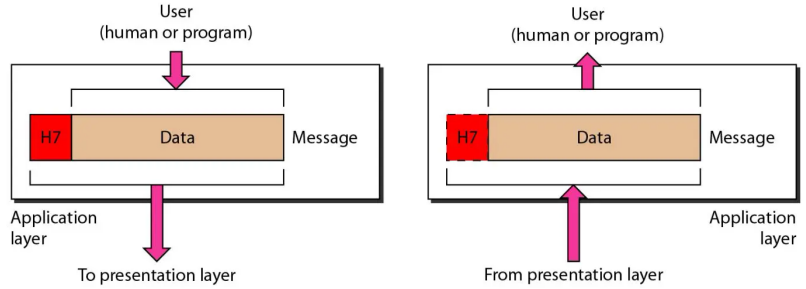

### 6. Livello di presentazione

Il **livello di presentazione** è responsabile della rappresentazione dei dati: si occupa di traduzione (ad esempio da codifica EBCDIC o ASCII a testo), cifratura e decifratura, e compressione.




### 7. Livello applicazione

Il **livello applicazione** è responsabile di fornire servizi all'utente: terminale virtuale di rete (accesso remoto), trasferimento e accesso a file, servizi di posta e accesso al World Wide Web.



Non sempre tutti i livelli sono implementati: nei nodi intermedi (switch, router) sono implementati solo 2–3 livelli (livelli "media"), mentre solo i sistemi finali implementano l'intero stack fino all'applicazione.

## Incapsulamento

Definition

**Incapsulamento**

**Processo top-down** in cui ogni livello del mittente aggiunge al messaggio (payload) un campo proprio del livello, come **header** (`H`) o **trailer** (`T`); la **decapsulazione** (bottom-up) rimuole e interpreta quei campi negli stessi livelli del ricevente.

## Critiche e confronto TCP/IP vs OSI

Info

**Critiche al modello OSI**

**Bad timing** (i protocolli TCP/IP concorrenti erano già in uso), **bad design** (modello e protocolli difettosi), **bad implementations** (implementazioni iniziali enormi e lente), **bad politics** (visto come creatura dei ministeri telecom europei).

Critiche al modello TCP/IP:

- Non distingue chiaramente i concetti di **servizi, interfacce e protocolli**.
- Non è generale: poco adatto a descrivere altri stack di protocolli.
- Il **link layer** non è un vero livello; non distingue il livello fisico dal data link.
- Le altre implementazioni di protocolli erano distribuite gratuitamente.

Confronto:

- **ISO/OSI** è *de iure* lo stack standard per le reti di calcolatori, ma è complesso e dettagliato.
- **TCP/IP e UDP/IP** (*Internet Protocol Suite*) sono *de facto* lo stack realmente usato su Internet e nelle reti locali: **TCP** (*Transmission Control Protocol*), **UDP** (*User Datagram Protocol*), **IP** (*Internet Protocol*).

Esistono altri modelli di rete per la comunicazione tra dispositivi, dipendenti dalla situazione, che però restano basati sul modello a livelli.

# Parte II

# Application Layer

Capitolo 3

# Application Layer

## Applicazioni di rete

Definition

**Applicazione di rete**
**Insieme di più programmi** che girano su sistemi finali diversi (host) e comunicano tra loro tramite la rete.

Le **applicazioni** sono i programmi in esecuzione sui dispositivi che permettono all'utente di accedere ai **servizi**: social network, Web, messaggistica, e-mail, giochi multiutente, streaming video, file sharing P2P, voice over IP, videoconferenza, motori di ricerca, remote login.

Creare un'applicazione di rete significa scrivere programmi che:

- **Girano su sistemi finali diversi**, anche con linguaggi o sistemi operativi diversi.
- **Comunicano sulla rete** (tipicamente via socket) mediante un protocollo specifico.

Non serve scrivere software per i dispositivi del nucleo della rete: i network device **non eseguono applicazioni utente**, e esistono **librerie specifiche** (es. socket) che implementano le funzionalità di rete.

## Architetture delle applicazioni

Definition

**Architettura dell'applicazione**
**Struttura** dell'applicazione sui vari sistemi finali, decisa dallo sviluppatore al momento della progettazione.

Le due architetture fondamentali sono:

- **Client-server**: nodi eterogenei; un host sempre attivo (il **server**) fornisce servizi richiesti da molti altri host (i client).
- **Peer-to-peer (P2P)**: nodi idealmente omogenei; l'applicazione sfrutta la comunicazione **diretta** tra coppie di host connessi intermittentemente (peer).

Nel client-server valgono le seguenti proprietà:

- I client **non comunicano direttamente** tra loro.
- Il server ha un **indirizzo fisso e noto** (hostname o IP), così un client può sempre contattarlo inviando un pacchetto di richiesta.
- Spesso sono coinvolti **più server** per reggere il carico; il client in genere **non se ne accorge** e li percepisce come un unico server.
- I server multipli possono essere raggruppati in **data center** (grande numero di server in una sede, da alimentare, mantenere e connettere), sparsi come **server distribuiti** nel mondo (da interconnettere e coordinare) o organizzati in **data center distribuiti**.

Nel peer-to-peer valgono le seguenti proprietà:

- Usato per applicazioni **traffic-intensive**; non esiste un server singolo con indirizzo fisso: ogni client ha il proprio indirizzo.
- Le applicazioni P2P pure sono rare: la maggior parte è **ibrida**, combinando client-server e P2P (es. server che tracciano gli indirizzi degli utenti, con messaggi scambiati direttamente tra host).
- P2P è **scalabile e distribuito**: ogni peer genera carico ma aggiunge anche capacità di servizio distribuendo file agli altri peer.
- P2P è **economico** (niente infrastrutture o banda significativa) ma ha problemi di **sicurezza, prestazioni e affidabilità**.

## Comunicazione tra processi

In un'applicazione di rete i **processi** comunicano attraverso la rete, anche su sistemi operativi diversi; tra una coppia di processi comunicanti si identifica tipicamente un **processo client** e un **processo server**, e nelle applicazioni P2P i processi possono **cambiare ruolo**.

**Definition**

**Socket**

**Interfaccia software** tra livello applicazione e livello trasporto: è la "cassetta delle lettere" tramite cui un processo invia e riceve messaggi, assumendo che lato trasporto esista un'infrastruttura che recapita il messaggio alla socket del processo destinatario.

- Le socket costituiscono l'**API** (*Application Programming Interface*) tra applicazione e rete.
- Lo sviluppatore controlla tutto lato applicazione, ma ha **poco controllo** del lato trasporto: solo la **scelta del protocollo di trasporto** (TCP/UDP) e pochi parametri (dimensione massima del buffer, maximum segment size, ecc.).
- Scelto il protocollo, l'applicazione assume come dati i servizi di trasporto che quel protocollo fornisce.

Per recapitare un messaggio servono due informazioni, dato che più applicazioni girano su uno stesso host:

1. Un **indirizzo** dell'host (per trovare l'host giusto in rete).
2. Un **identificatore** del processo destinatario (per trovare il processo giusto nell'host).

L'indirizzo lavora a livello **rete** (routing), l'identificatore a livello **trasporto**:

- In Internet l'host è identificato da un **indirizzo IP** (quantità a 32 bit).
- Il processo è identificato da un **numero di porta**; le applicazioni popolari hanno porte convenzionali (es. Web server su porta 80, mail server con SMTP su porta 25); la lista dei numeri di porta noti è mantenuta da IANA.

## Servizi di trasporto

**Definition**

**Quality of Service**

**Insieme di proprietà** che il livello trasporto può implementare su richiesta dell'applicazione.

- **Affidabilità** del trasferimento: i dati inviati da un capo giungono corretti e completi all'altro capo.
- **Throughput**: ritmo a cui il processo mittente può consegnare bit al ricevente (bit/s).
- **Timing**: ogni bit immesso nella socket arriva alla socket del ricevente entro un intervallo di tempo.
- **Sicurezza**: cifratura e decifratura dei messaggi; teoricamente è un problema di trasporto, ma i protocolli sicuri (es. **TLS**) sono costruiti su TCP, quindi in pratica sta a livello applicazione.

I due protocolli di trasporto si collocano così:

- **TCP** (*Transmission Control Protocol*): prevede un **handshake** tra i processi e un servizio di trasferimento affidabile; per questo è detto **connection-oriented**.
- **UDP** (*User Datagram Protocol*): **leggero**, con servizi minimi; nessun handshake, nessuna garanzia di consegna; per questo è detto **connectionless**.

## Protocolli applicativi

**Definition**

**Protocollo applicativo**

**Insieme di regole** che definisce come i processi di un'applicazione, su sistemi finali diversi, si scambiano messaggi.

Un protocollo applicativo definisce quattro cose:

1. I **tipi** di messaggi scambiati (es. richieste e risposte).
2. La **sintassi** dei tipi di messaggio: campi presenti e loro delimitazione.
3. La **semantica** dei campi, cioè il significato dell'informazione contenuta.
4. Le **regole** che stabiliscono quando e come un processo invia e risponde ai messaggi.

I protocolli applicativi più comuni sono i seguenti.

| Applicazione | Descrizione |
|---|---|
| `DHCP` | Dynamic Host Configuration Protocol, assegna indirizzi IP |

| Applicazione | Descrizione |
|---|---|
| `DNS` | Domain Name System, traduce nomi di siti in indirizzi IP |
| `HTTP/HTTPS` | HyperText Transfer Protocol (Secure), trasferisce pagine web |
| `SMTP/SMTPS` | Simple Mail Transfer Protocol (Secure), invia messaggi e-mail |
| `SNMP` | Simple Network Management Protocol, gestisce dispositivi di rete |
| `Telnet/SSH` | Teletype Network (Secure SHell), interfaccia a riga di comando con host remoti |
| `FTP/FTPS` | File Transfer Protocol (Secure), usato per trasferire file |

Alcune applicazioni scambiano **informazioni sensibili** (credenziali, dati personali): sulle reti pubbliche i messaggi vanno protetti, da qui le varianti *secure* dei protocolli.

# FTP e FTPS

**Definition**

**FTP**

**File Transfer Protocol**: uno dei protocolli più antichi di Internet (prima versione 1971), usato per trasferire file tra host tramite rete.

- Nella versione base il trasferimento è in **clear-text** (username, password e file leggibili), quindi è adatto ad applicazioni locali o private.
- La versione sicura **FTPS** protegge username e password e cifra i contenuti.
- FTP e FTPS hanno due componenti: il **protocollo** (comandi per mostrare, scaricare, eliminare file, ecc.) e l'**applicazione software** che lo implementa (lato client e lato server, es. `ftp` e `vsftpd` su Linux).

Comandi FTP più comuni (client connesso al server):

- `help`: elenca i comandi disponibili.
- `cd` / `lcd`: cambia directory sulla macchina remota / locale.
- `ls` / `pwd`: elenca file e directory / stampa la directory di lavoro remota.
- `mkdir` / `rmdir` / `delete`: crea directory, rimuove directory, elimina file sulla macchina remota.
- `get` / `put`: copia un file dal server al client / dal client al server.
- `exit`: chiude la connessione.

Capitolo 4

# Application Layer HTTP

## Web e HTTP

Fino ai primi anni '90 Internet era usato principalmente da ricercatori, accademici e studenti (Telnet per il login remoto, FTP per i file, e-mail); il **World Wide Web**, nei primi anni '90, fu la prima applicazione che catturò il grande pubblico.

**Definition**

**World Wide Web**

**Insieme di informazione** (documenti, immagini, video, audio, ecc.) accessibile su Internet secondo il protocollo **HTTP** (*HyperText Transfer Protocol*).

La comunicazione HTTP è tipicamente **client-server**:

- Un programma client traduce le richieste dell'utente in messaggi HTTP: è il **browser** (Chrome, Firefox, Edge, Safari, ecc.).
- Un programma server esegue le richieste HTTP e restituisce le **risposte** HTTP.

HTTP definisce principalmente due cose:

- La **struttura dei messaggi**.
- **Come client e server si scambiano** tali messaggi.

Il Web e i suoi protocolli sono la piattaforma per applicazioni web-based (YouTube, web-mail, app mobili, social network). HTTP evolve continuamente per bilanciare affidabilità e prestazioni.

## URL

**Definition**

**URL**

**Uniform Resource Locator**: stringa che identifica una risorsa (o oggetto) sul Web, con la struttura `[protocol]://[usrinfo@][host][:port][/path][?query][#fragment]`.

I campi del URL:

- **`[protocol]`**: protocollo da usare per accedere alla risorsa (`HTTP`, `HTTPS`, `FTP`, ecc.).
- **`[usrinfo]`** (opzionale): informazioni utente come username e password seguite da `@` (es. `username:password@`); per lo più deprecato per ragioni di sicurezza.
- **`[host]`**: nome o indirizzo IP del server.
- **`[port]`** (opzionale): porta da usare, spesso dedotta dal protocollo.
- **`[path]`**: percorso della risorsa nel server (es. `/path/to/resource`).
- **`[query]`**: preceduta da `?`, specifica possibili richieste.
- **`[fragment]`**: preceduta da `#`, identifica un elemento nella risorsa (es. un punto di un formulario).

Pagine e oggetti:

- Le pagine Web consistono di un **file HTML** (*HyperText Markup Language*) di base e riferimenti a oggetti aggiuntivi (immagini, video, Java applet, ecc.).
- Una pagina con testo HTML e cinque immagini JPEG contiene **6 oggetti**: il file base più le 5 immagini.
- La URL `http://www.someSchool.edu/someDepartment/picture1.gif` si legge: `http` protocollo, `www.someSchool.edu` hostname, `/someDepartment/picture1.gif` percorso dell'oggetto.
- Una query reale come `https://en.wikipedia.org/w/index.php?title=SSC_Napoli` si legge: `https` protocollo, `en.wikipedia.org` hostname, `/w/index.php` percorso, `?title=SSC_Napoli` query; equivale alla pagina `https://en.wikipedia.org/wiki/SSC_Napoli`.

# Il protocollo HTTP

HTTP definisce come i client Web richiedono oggetti (es. pagine Web) ai server Web e come i server li trasferiscono ai client.

- Quando l'utente chiede un oggetto (es. cliccando un collegamento), il browser invia al server **messaggi di richiesta HTTP** per gli oggetti della pagina.
- Il server riceve le richieste e risponde con **messaggi di risposta** che contengono gli oggetti.

Info

**HTTP e livello trasporto**

HTTP usa **TCP** come protocollo di trasporto sottostante; dalla versione **HTTP/3** (2022) può usare anche **UDP**, e circa il 30% del traffico HTTP complessivo viaggia su UDP.

- Il client prima **inizializza la connessione** con il server; stabilita la connessione, browser e server comunicano attraverso le rispettive interfacce **socket**: il client invia richieste nella propria socket e riceve risposte dalla propria socket; analogamente il server riceve richieste e invia risposte nella propria socket.
- Storicamente si usava TCP per i suoi servizi, prima di tutto il **trasferimento affidabile**: TCP garantisce che i messaggi arrivino integri a destinazione (se possibile).
- I servizi di TCP possono però rallentare la comunicazione, quindi HTTP/3 usa anche UDP: si stima che sia circa il **30% più veloce**; si affida a **QUIC** (*Quick UDP Internet Connection*), che implementa l'affidabilità sopra UDP.
- Qui si vede il vantaggio dell'architettura a livelli: le applicazioni HTTP **non si preoccupano** di raggiungibilità o perdita di dati, delegando il lavoro ai livelli inferiori, e risultano più semplici (logicamente e computazionalmente).

# Stateless

Definition

**Stateless**

**Proprietà** delle applicazioni HTTP: il server **non mantiene informazioni** sull'interazione (sequenza di richieste/risposte) con un client specifico.

- Se un client chiede due volte lo stesso oggetto, il server non considera la richiesta ridondante: reinvia l'oggetto, avendo "dimenticato" il passato.
- La semplicità è importante per server che gestiscono molti riferimenti al secondo (in media ~1000 richieste/s; i grandi motori di ricerca ricevono centinaia di migliaia di query al secondo).
- Non tutte le applicazioni sono stateless: diverse sono invece **stateful**.

# Connessioni persistenti e non persistenti

In molte applicazioni client e server comunicano per un periodo esteso scambiando più coppie richiesta-risposta (back-to-back, periodiche o intermittenti). Con un protocollo connection-oriented (TCP o QUIC) si hanno due modalità:

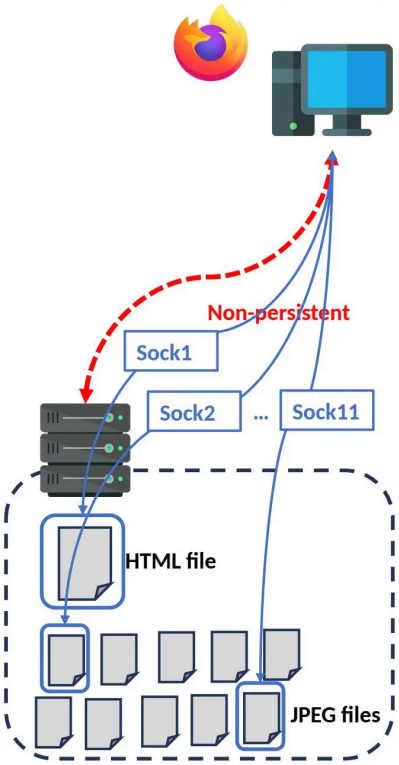
- **Connessione persistente**: tutte le richieste e le relative risposte sono inviate sulla **stessa connessione**.
- **Connessione non persistente**: per ogni coppia richiesta-risposta si stabilisce una **nuova connessione**.

Per impostazione predefinita le applicazioni HTTP odierne usano connessioni persistenti, ma client e server possono essere configurati per la modalità non persistente; le prime versioni di HTTP (1.0) usavano di default la non persistenza.

### Connessione non persistente

In una connessione non persistente la connessione si chiude a ogni richiesta servita (es. a ogni oggetto inviato). La sequenza su TCP per una pagina con file HTML base e 10 immagini JPEG (11 oggetti sullo stesso server):

1. Il processo client HTTP inizializza la connessione TCP verso il server sulla porta 80 (porta di default di HTTP): una socket al client e una al server.
2. Il client invia il messaggio di richiesta HTTP con il percorso `/someDepartment/home.index`.
3. Il server riceve la richiesta, recupera l'oggetto (da RAM o disco), lo incapsula in un messaggio di risposta e lo invia tramite la propria socket.
4. Il server chiede a TCP di chiudere la connessione; TCP la termina solo dopo aver certificato che il client ha ricevuto la risposta integra (affidabilità).
5. Il client riceve la risposta, estrae il file HTML e trova i riferimenti alle 10 immagini JPEG.
6. I primi quattro passi si ripetono per ciascuno degli oggetti referenziati.




**Example**

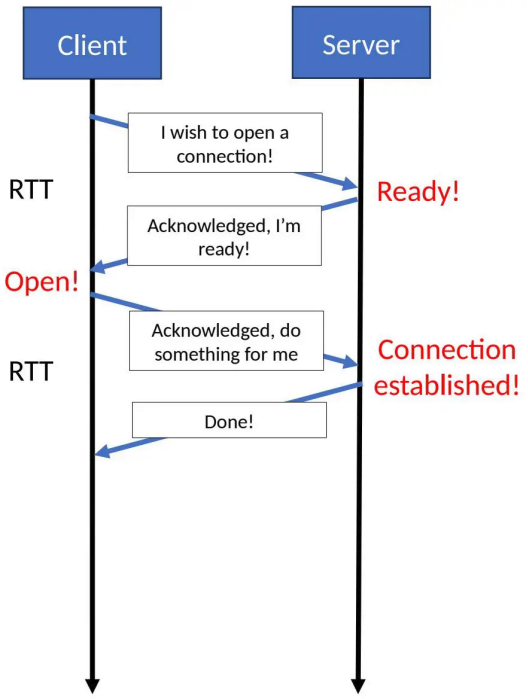
- Servono **11 connessioni TCP** (e 11 paia di socket) per trasferire l'intera pagina, quindi gli elementi possono arrivare in momenti diversi.
- Le connessioni possono essere stabilite in **sequenza o in parallelo**: ricevuta la pagina base, si possono scaricare in parallelo tutte le immagini; i browser aprono tipicamente 5–10 connessioni TCP parallele, ciascuna per una transazione richiesta-risposta.
- Il parallelismo **accorcia il tempo di risposta** ma **aumenta il costo computazionale**, e stabilire più connessioni TCP può introdurre comunque un overhead significativo.
- HTTP definisce solo il protocollo di comunicazione tra programma client e programma server, **non come i contenuti sono visualizzati**: la resa dipende dal browser (due browser possono interpretare la stessa pagina in modo diverso).

### RTT e tempi di risposta

**Definition**

**RTT**
**Round-Trip Time**: tempo che un pacchetto piccolo impiega per andare dal client al server e tornare al client.

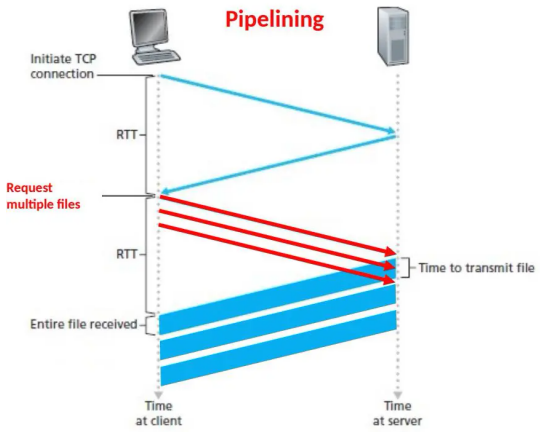
- Per stabilire una connessione TCP si esegue il **three-way handshake**: il client invia un segmento TCP per richiedere la connessione; il server risponde con un segmento di acknowledgement; il client conferma al server.
- Dalla prospettiva dell'RTT servono **2 RTT** per stabilire la connessione (senza perdite); un handshake simile avviene anche alla chiusura.
- Le prime due parti del handshake consumano 1 RTT; il client invia la richiesta HTTP **insieme alla terza parte** del handshake (l'acknowledgment); arrivata la richiesta, il server invia il file HTML, consumando un altro RTT.
- Il **tempo totale di risposta** è circa **2 RTT più il tempo di trasmissione** del file HTML da parte del server.




**Connessione persistente**

- Il server **lascia aperta** la connessione TCP dopo aver inviato una risposta.
- Un'intera pagina Web (file HTML e oggetti referenziati) e anche più pagine dello stesso server verso lo stesso client viaggiano su **una sola** connessione persistente.
- Il risparmio è di circa **2–4 RTT per oggetto**.

**Pipelining**: le richieste degli oggetti possono essere fatte back-to-back, senza attendere le risposte; da HTTP/2 richieste e risposte multiple possono essere **interleave** sulla stessa connessione, con meccanismi per **prioritizzare** richieste e risposte. Connessione persistente con pipelining è la modalità di default di HTTP.




**Confronto persistente vs non persistente**

**Persistente**:

- Più **veloce**, soprattutto con pipelining.
- Richiede **meno risorse** (soprattutto memoria).
- Più **complessa** da implementare, soprattutto con pipelining.
- La connessione può restare aperta anche inutilizzata: il server la chiude dopo un **timeout** di inattività.

**Non persistente**:

- **Facile** da implementare e coerente con la *statelessness* di HTTP.
- Richiede più risorse: per ogni connessione si allocano buffer TCP e variabili TCP sia al client sia al server.
- Più **lenta**: 1–2 RTT aggiuntivi per richiesta.

- Le connessioni non possono restare aperte.

Capitolo 5

# HTTP messaggi cookie e web caching

## HTTP riepilogo

**HyperText Transfer Protocol (HTTP)**: uno dei protocolli più usati su Internet (applicazioni web-based), con le seguenti proprietà:

- È **client-server** e definisce la struttura dei messaggi e come client e server li scambiano.
- **Non** decide come implementare le applicazioni client e server.
- Usa messaggi di tipo **richiesta/risposta**, scritti in **ASCII** leggibile.
- È **stateless**: client e server non ricordano le interazioni passate.
- Assume un trasporto affidabile e connection-oriented (TCP, recentemente anche UDP).
- Usa connessioni persistenti e non persistenti.

## Formato dei messaggi

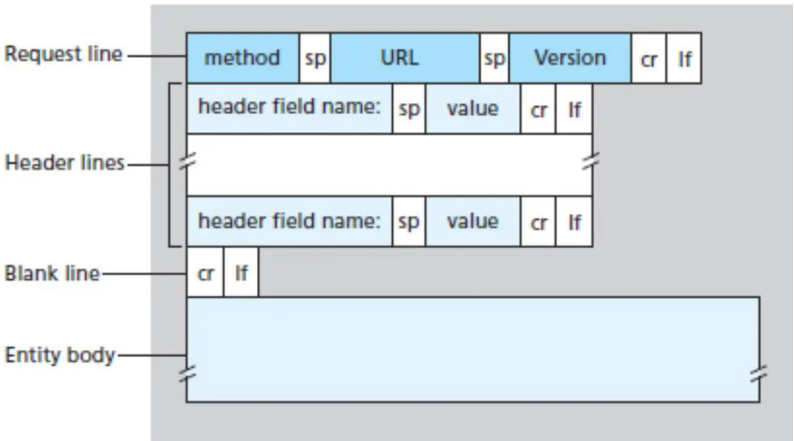
HTTP ha due formati distinti:

- **Richiesta**: specifica il comando (**metodo**) che il server deve eseguire (es. dammi una pagina, elabora un form).
- **Risposta**: riporta l'esito del comando (successo, fallimento, ecc.) e i possibili dati (es. il file richiesto).

### Messaggio di richiesta

La richiesta include i seguenti campi:

- **Method**: comando richiesto al server.
- **URL**: identifica l'oggetto su cui operare.
- **Version**: versione HTTP (es. `HTTP/1.1`).
- **Header lines**: parametri della richiesta, in numero e tipo non fissi; ogni riga contiene nome e valore del parametro (es. persistenza della connessione; sono possibili header personalizzati).
- **Body**: dipende dal metodo, contiene dati potenzialmente associati al comando (es. testo di un form).




I campi sono separati da caratteri speciali: **sp** (spazio), **cr** (carriage return, `\r`), **lf** (line feed, `\n`).

Metodi principali:

- `GET`: recupera oggetti (risorse) dal server; è il comando più usato, poiché ogni pagina richiesta va recuperata; il body è vuoto.
- `POST`: imposta informazioni dentro oggetti (risorse) del server; il body contiene le informazioni da inviare. Le richieste generate da un form non usano necessariamente `POST`: i form HTML usano spesso `GET`, includendo i dati nell'URL richiesto.

- **HEAD**: come GET, ma il server risponde senza restituire l'oggetto; usato spesso per il debugging.
- **PUT**: carica un oggetto su un percorso (directory) specifico di un server Web; usato con strumenti di Web publishing e da applicazioni che caricano oggetti.
- **DELETE**: elimina un oggetto su un server Web.

Esempio di richiesta (browser Mozilla/5.0 che chiede `/somedir/page.html` a `www.someschool.edu`):

```HTTP
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
Connection: close
User-agent: Mozilla/5.0
Accept-language: fr
```

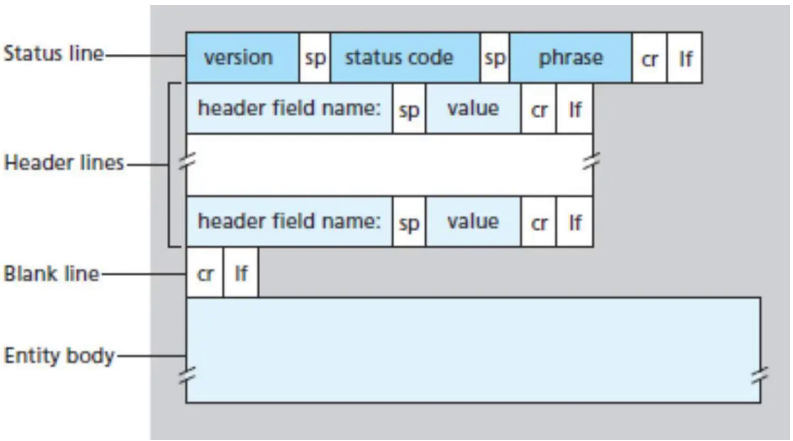
Significato delle righe di header:

- **Host**: host su cui risiede l'oggetto; la destinazione non è sempre l'host successivo (il messaggio può passare da un proxy), quindi l'indirizzo nel TCP può differire dall'host reale di destinazione.
- **Connection: close**: il browser chiede una connessione **non persistente** (chiusura al termine).
- **User-agent**: tipo di browser; utile perché il server può inviare versioni diverse dello stesso oggetto (stesso URL) in base al tipo.
- **Accept-language**: preferenza linguistica dell'utente (es. versione francese dell'oggetto, se esiste).

### Messaggio di risposta

Il formato è simile alla richiesta, ma al posto della request line c'è una **status line** con:

- **Version**: versione HTTP della risposta del server.
- **Status code**: codice (numero) che specifica l'esito del comando.
- **Phrase**: frase con il risultato della richiesta.




Classi di status code:

- **100–199** informational: informazioni sulla richiesta.
- **200–299** successful: richiesta eseguita con successo.
- **300–399** redirection: servono azioni aggiuntive del client.
- **400–499** client-error: la richiesta non può essere eseguita per un problema del client.
- **500–599** server-error: la richiesta non può essere eseguita per un problema del server.

**Example**

Codici tipici:

- **200 OK**: richiesta riuscita, l'informazione è nella risposta.
- **301 Moved Permanently**: l'oggetto è stato spostato permanentemente; la nuova URL è indicata nell'header `Location` della risposta.
- **400 Bad Request**: errore generico, il server non ha capito la richiesta.
- **404 Not Found**: il documento richiesto non esiste sul server.
- **505 HTTP Version Not Supported**: versione del protocollo non supportata dal server.

Example

**Esempio di risposta:**

```HTTP
HTTP/1.1 200 OK
Connection: close
Date: Tue, 18 Aug 2015 15:44:04 GMT
Server: Apache/2.2.3 (CentOS)
Last-Modified: Tue, 18 Aug 2015 15:11:03 GMT
Content-Length: 6821
Content-Type: text/html
```

Significato degli header:

- `Connection: close`: la connessione verrà chiusa dopo questo messaggio (non persistente).
- `Date`: data e ora di creazione/invio della risposta da parte del server.
- `Server`: software che ha generato il messaggio (es. Apache; analogo a `User-agent`).
- `Last-Modified`: data di creazione o ultima modifica dell'oggetto; utile per il caching, poiché le copie in cache possono essere datate.
- `Content-Length`: numero di byte dell'oggetto.
- `Content-Type`: tipo dell'oggetto (es. `text/html`): il tipo è dichiarato ufficialmente da questo header e **non** dall'estensione del file.

# Cookie

Un server HTTP è tipicamente **stateless**: questo semplifica la progettazione, riduce l'uso di risorse e permette migliaia di connessioni TCP simultanee; ma la pura statelessness è un limite, perché diverse funzioni Web sono **client-specific** (es. carrello, suggerimenti basati sulle preferenze).

Definition

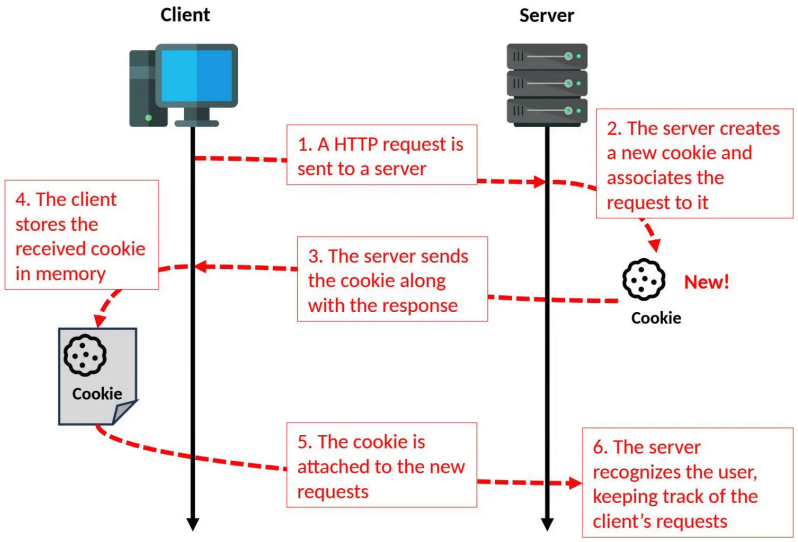
**Cookie**

**Token digitale** (ID alfanumerico) creato dal server e consegnato al client, usato dal server per **identificare un client specifico**; i cookie possono avere attributi (es. data di scadenza) e permettono ai siti di tenere traccia degli utenti.

La tecnologia dei cookie ha quattro componenti:

1. Una riga di header `Set-cookie` nel messaggio di risposta HTTP.
2. Una riga di header `Cookie` nel messaggio di richiesta HTTP.
3. Un **file sul sistema client**, gestito dal browser, che associa hostname del server e numero ID del cookie.
4. Un **database sul lato server**, con una voce per ogni cookie.

Funzionamento: alla prima richiesta di un client verso un sito, il server crea un cookie (numero ID) e la relativa voce nel proprio database; la risposta contiene l'header `Set-cookie: 1678`. Il browser aggiunge al file dei cookie hostname e ID; da quel momento ogni richiesta verso quel sito include l'header `Cookie: 1678`, così il server può tracciare l'attività dell'utente nel database (pagine visitate, ordine, tempi).




Usi principali e criticità:

- **Carrello**: il server mantiene la lista degli acquisti durante la navigazione.
- **Registrazione utente**: associando le informazioni dell'utente al cookie nel database (carta di credito, nome,

e-mail, indirizzo) per non reinserirle ogni volta.
- **Consigli sui prodotti** basati sulle pagine visitate.

I cookie sono **controversi**: sono considerati un'invasione della privacy, poiché combinando cookie e informazioni fornite dall'utente un sito può sapere molto di lui e potenzialmente vendere tali informazioni a terzi.

# Web caching

**Definition**

**Web cache**

**Entità di rete** che soddisfa le richieste HTTP **per conto** di un server Web di origine (*origin server*); le entità intermedie che lavorano per conto di altri sono dette **proxy server**.

- La cache ha un proprio disco dove conserva copie degli oggetti richiesti di recente.
- Il browser può essere configurato per indirizzare **tutte** le richieste HTTP alla cache, che verifica la disponibilità di una copia dell'oggetto.
- Una cache è **sia server** (quando fornisce oggetti) **sia client** (quando li richiede) allo stesso tempo.

Procedura di accesso a un oggetto tramite cache (es. `http://www.someschool.edu/campus.gif`):

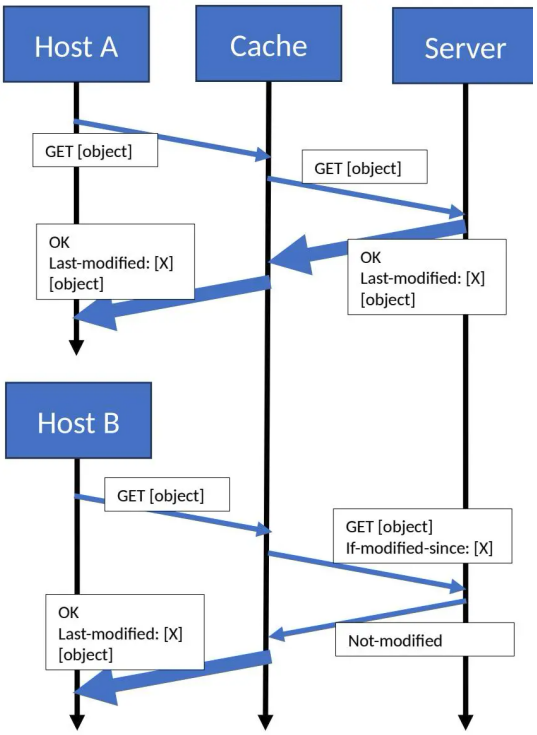
1. Il browser apre una connessione TCP con la cache e le invia la richiesta HTTP dell'oggetto.
2. La cache verifica di avere una copia locale: in caso affermativo la restituisce al browser in un messaggio di risposta.
3. Se non la possiede, la cache apre una connessione TCP con il server di origine e gli invia la richiesta.
4. Ricevuto l'oggetto, la cache ne memorizza una copia e ne invia una al browser in un messaggio di risposta.

Motivazioni del deployment:

1. **Ridurre il tempo di risposta** per il client, specialmente se tra client e cache c'è una connessione ad alta velocità.
2. **Ridurre il traffico** dell'azienda/istituzione verso Internet, riducendo i costi di banda.

Misure e problemi:

- **Hit**: la cache fornisce l'oggetto senza contattare il server di origine; l'**hit rate** (frazione di richieste soddisfatte dalla cache) è tipicamente tra **0.2 e 0.7**, e cresce all'aumentare dei clienti che usano la cache: fino al 70% delle richieste può essere servito localmente.
- **Problema di obsolescence**: la copia in cache può essere datata; per verificarla esiste il **conditional GET**: richiesta HTTP con metodo `GET` e header `If-Modified-Since:`, con cui la cache controlla se l'oggetto è aggiornato e, in tal caso, restituisce la versione memorizzata senza ulteriore comunicazione.
- Anche i **browser** fanno caching locale: memorizzano gli oggetti per non recuperarli dal server; la tecnica migliora molto le prestazioni, ma la versione locale può non essere aggiornata, generando errori piuttosto frequenti.




## Oltre il caching

- I proxy servono anche ad **accedere a servizi istituzionali**: le reti aziendali/universitarie forniscono un web proxy istituzionale (es. `proxy.unina.it`).
- Poiché le richieste sono inoltrate dal proxy istituzionale, dal punto di vista del server di origine **tutte le richieste arrivano dall'istituzione**.
- Su Internet esistono diversi proxy commerciali e gratuiti.

Capitolo 6

# Application Layer posta e-mail e P2P

## Posta elettronica e SMTP

La posta elettronica è una delle applicazioni Internet più antiche e importanti; è tipicamente implementata in stile client-server con due tipi di host:

- **User agent**: applicazione per la gestione della posta da parte dell'utente (es. Pine, K-9 Mail).
- **Mail server**: server che **immagazzina le e-mail** e mantiene le **mailbox** specifiche degli utenti.

Diversamente da HTTP, qui la comunicazione avviene **anche tra server**: servono due protocolli, uno tra user agent e mail server e uno tra mail server e mail server.

**Definition**

**SMTP**

**Simple Mail Transfer Protocol**: principale protocollo di livello applicativo che lavora **tra mail server**; permette ai server di scambiarsi le e-mail, mentre gli user agent **non** usano SMTP.

- Gli utenti non si scambiano posta direttamente (come nella messaggistica istantanea): si preferiscono mail server, più affidabili e specializzati.
- Una volta arrivata sul server, la mail è memorizzata nella mailbox dell'utente, in attesa che l'user agent la scarichi localmente.
- In SMTP esistono un lato client (mittente) e un lato server (ricevente), entrambi sui mail server, che usano il protocollo affidabile **TCP**.

**Example**

Invio di una mail (Alice → Bob): il processo coinvolge l'agente di Alice, il mail server di Alice, l'agente di Bob e il mail server di Bob.

1. Alice invoca il suo user agent, fornisce l'indirizzo di Bob, compone il messaggio e ordina l'invio.
2. L'agente invia il messaggio al mail server di Alice, dove viene posto in una **coda dei messaggi**.
3. Il lato client SMTP del server di Alice vede il messaggio in coda e apre una connessione TCP verso il server SMTP sul mail server di Bob.
4. Dopo un handshaking iniziale, il client SMTP invia il messaggio nella connessione TCP.
5. Il lato server SMTP del server di Bob riceve il messaggio e lo colloca nella mailbox di Bob.
6. Bob invoca il suo user agent e legge il messaggio quando vuole.

Dettagli del protocollo:

- Il client SMTP stabilisce una connessione TCP alla **porta 25** del server SMTP; se il server è giù, il client **riprova più tardi**.
- Stabilita la connessione, client e server eseguono un handshaking di livello applicativo: **si presentano** prima di trasferire informazioni.
- Durante l'handshaking il client SMTP indica l'indirizzo e-mail del mittente e quello del ricevente.
- SMTP conta sul servizio di trasferimento affidabile di TCP per consegnare il messaggio al server senza errori.

## Accesso alla posta

Il deployment di server SMTP è più affidabile dello scambio utente-utente diretto: i server sono sempre accesi e visibili in rete, certificati, e possono ritentare l'invio in continuazione (processo computazionalmente costoso).

Per accedere alle mail sul server serve però un'applicazione client-server aggiuntiva con un protocollo diverso. I più diffusi:

- **POP3** (*Post Office Protocol* version 3).
- **IMAP** (*Internet Mail Access Protocol*).
- **HTTP**.

### POP3

**Definition**

**POP3**

**Protocollo di accesso alla posta estremamente semplice**: l'user agent (client) apre una connessione TCP al mail server sulla **porta 110**.

Con la connessione stabilita, POP3 procede in tre fasi:

1. **Authorization**: l'user agent invia username e password per autenticare l'utente.
2. **Transaction**: l'user agent può recuperare i messaggi, marcarli per la cancellazione, rimuovere i segni di cancellazione e ottenere statistiche sulla posta.
3. **Update**: dopo il comando `quit` che chiude la sessione, il server **elimina i messaggi marcati** per la cancellazione.

### IMAP

Con POP3 i messaggi possono solo essere scaricati o cancellati: azioni come cercare o organizzare le mail in cartelle non sono previste e vanno fatte sulla macchina locale.

**Definition**

**IMAP**

**Internet Mail Access Protocol**: protocollo di accesso che permette al server di fornire funzionalità aggiuntive rispetto a POP3.

- **Gestione e creazione di cartelle**.
- **Ricerca** nelle cartelle remote di messaggi che rispettano criteri specifici.
- Ottenimento di **solo parti** dei messaggi (es. solo header, solo allegati).
- **Client multipli** connessi contemporaneamente allo stesso server.

### Accesso via HTTP

- Sempre più utenti inviano e leggono posta tramite **browser Web**: l'accesso web-based è stato introdotto da Hotmail a metà anni '90 ed è oggi l'approccio dominante (Google, Yahoo!, Virgilio, ecc.), usato da quasi tutte le università e aziende.
- L'user agent è un comune browser che comunica con la mailbox remota via **HTTP** anziché POP3 o IMAP.
- Vale per gli user agent: **i mail server continuano a usare SMTP** per scambiarsi i messaggi.

## Peer-to-Peer

**Definition**

**Architettura P2P**

**Architettura** che fa uso minimo (o nullo) di server: coppie di host connessi intermittentemente (**peer**) comunicano **direttamente** tra loro; i peer non sono di proprietà di un fornitore di servizi, ma desktop e laptop controllati dagli utenti.

Il file sharing è l'applicazione naturale:

- Nel client-server il server deve distribuire i file a tutti i client: grave onere per il server, che richiede grande banda.
- Nel P2P i peer che ricevono il file possono **condividerlo a loro volta** con altri peer: in qualche modo i peer sono client e server allo stesso tempo.
- Il P2P scala tipicamente **meglio** del client-server nel file sharing, ma è più complesso da implementare e ha problemi di **sicurezza**, con tutti questi client in comunicazione diretta.

## BitTorrent

**Definition**

**BitTorrent**

**Protocollo P2P** popolare per la condivisione/distribuzione di file (utenti stimati 150–170 milioni nel 2023); definisce il comportamento di un **torrent**.

- Un **torrent** è l'insieme dei peer che partecipano alla distribuzione di un file; i peer si scaricano l'un l'altro **chunk** di uguali dimensioni del file (tipicamente 256 kB).
- Un peer che entra nel torrent senza chunk inizia ad accumularne; mentre scarica chunk, **carica** chunk verso gli altri peer.

- Ottenuto il file completo, il peer può lasciare il torrent (egoisticamente) o restare e continuare a caricare (altruisticamente); i peer possono uscire in qualsiasi momento con un sottoinsieme di chunk e rientrare più tardi.

**Il tracker**

- Ogni torrent ha un nodo di infrastruttura chiamato **tracker**, che tiene traccia dei peer partecipanti (i tracker sono di fatto server).
- Un nuovo peer si **registra** col tracker e lo informa periodicamente del proprio stato; il tracker gli fornisce gli **indirizzi IP** di un sottoinsieme casuale di peer del torrent.
- Il nuovo host tenta connessioni TCP **concorrenti** con tutti i peer della lista (**vicini**); durante l'esecuzione alcuni vicini possono uscire, mentre altri peer (fuori dalla lista iniziale) possono tentare nuove connessioni.
- In ogni istante ogni peer possiede un sottoinsieme di chunk, diverso da peer a peer; **periodicamente** ogni host chiede a ciascun vicino connesso (sulle connessioni TCP) la **lista dei chunk** che possiede.

**Strategie del client**

Il client in download decide quali chunk richiedere (e a chi) secondo il principio **rarest-first**: hanno priorità i chunk con **meno copie ripetute** tra i vicini; così i chunk più rari vengono ridistribuiti più rapidamente e il numero di copie nel torrent si equalizza.

Il client in upload decide quali richieste servire secondo due principi intrecciati:

- **Trading**: il host da priorità ai **4 migliori vicini** che stanno fornendo dati alla massima velocità; il controllo avviene periodicamente (ogni 10 secondi) e la lista dei 4 migliori si aggiorna di conseguenza.
- **Random selection (optimistic unchoking)**: ogni 30 secondi ogni host sceglie anche **un vicino a caso** e gli invia chunk; se lo scambio casuale risulta buono (i due host sono buoni partner), entrano nelle rispettive liste dei migliori; il processo permette ai peer con velocità di upload compatibili di **trovarsi**.

Capitolo 7

# Application Layer DNS

## Identificazione degli host

Ci sono due modi per identificare un host:

- **Hostname**: identificatore mnemonico, preferito dalle persone.
- **Indirizzo IP**: identificatore a lunghezza fissa e struttura gerarchica, preferito dai dispositivi di rete.

Definition

**DNS**

**Domain Name System**: protocollo di livello applicativo che gestisce la **traduzione da hostname a indirizzo IP**; è un protocollo **client-server** in cui un client DNS chiede a un server DNS una traduzione specifica nome-indirizzo.

- Il DNS è un **database distribuito**, implementato in una gerarchia di server, più il protocollo applicativo che permette agli host di interrogarlo; query e risposte viaggiano in datagrammi **UDP sulla porta 53** (può usare anche TCP porta 53).
- I server DNS sono spesso macchine UNIX con il software **BIND** (*Berkeley Internet Name Domain*), il più diffuso su Linux e sistemi Unix-like.
- È l'unica applicazione che nessuno usa di proposito: lavora per le altre applicazioni e si vede solo quando fallisce.

Un repository centralizzato (tabella unica) è impossibile da gestire per il grande numero di host e per la distanza geografica (e i relativi ritardi): il DNS è quindi un sistema **distribuito e decentralizzato**, ovvero **gerarchico**, **basato su domini** e **implementato tramite un database distribuito**.

- **Singolo punto di guasto**: se crasha, ogni applicazione che risolve un nome si ferma con lui.
- **Traffico**: risponderebbe per ogni richiesta Web e ogni mail inviata sul pianeta.
- **Distanza**: non può essere vicino a tutti: dall'altro capo del mondo ogni lookup paga il viaggio.
- **Manutenzione**: un record per host, con host aggiunti e spostati di continuo.

Il DNS è quindi **distribuito per progetto**: gerarchico, basato su domini, e da nessuna parte completo ; un sistema **distribuito e decentralizzato**, ovvero **gerarchico**, **basato su domini** e **implementato tramite un database distribuito**.

### Resolver

Definition

**Resolver**

**Componente** usato dalle applicazioni per richiedere la risoluzione dei nomi: costruisce la query DNS e ottiene la risposta, appoggiandosi a un server DNS configurato.

## Il name space DNS

- La gerarchia dei nomi è organizzata da **ICANN** (*Internet Corp. for Assigned Names and Numbers*).
- Esistono circa 250 **top-level domain (TLD)**, di due tipi, tutti gestiti da registrar nominati da ICANN:
  - **Generici**: `aero`, `com`, `edu`, ecc.
  - **Geografici** (country): `uk`, `it`, `tv`, ecc.
- Ogni dominio è partizionato in sotto-domini, e così via; lo name space è gerarchico **dalla radice verso il basso**, con parti diverse delegate a organizzazioni diverse.

Alcuni domini generici di riferimento:

| Dominio | Uso previsto | Anno | Ristretto? |
|---|---|---|---|
| `com` | Commerciale | 1985 | No |
| `edu` | Istituzioni educative | 1985 | Sì |
| `gov` | Governo | 1985 | Sì |

| Dominio | Uso previsto | Anno | Ristretto? |
|---|---|---|---|
| `int` | Organizzazioni internazionali | 1988 | Sì |
| `mil` | Militare | 1985 | Sì |
| `net` | Fornitori di rete | 1985 | No |
| `org` | Organizzazioni non-profit | 1985 | No |
| `aero` | Trasporto aereo | 2001 | Sì |
| `biz` | Imprese | 2001 | No |
| `info` | Informativo | 2002 | No |
| `name` | Persone | 2002 | No |
| `pro` | Professionisti | 2002 | Sì |
| `tel` | Dettagli di contatto | 2005 | Sì |
| `xxx` | Industria del sesso | 2010 | No |

Regole dei nomi di dominio:

- I nomi sono **percorsi verso l'alto** dal dominio alla radice (es. `eng.mit.edu`); i percorsi **assoluti terminano con un punto**, quelli relativi vanno interpretati.
- È possibile registrarsi sotto **più di un** top-level domain (es. `ibm.com`, `ibm.us`).
- Ogni dominio controlla l'allocazione dei domini sottostanti: creare un nuovo dominio richiede il **permesso del dominio padre**.
- Il naming **non segue** i confini fisici/geografici.
- **Cyber-squatting**: acquisto di domini solo per rivenderli successivamente ad aziende interessate.

# Resource record

I record di risorsa (**resource record**) sono le voci del database distribuito: sono ciò che un server restituisce e ciò di cui è fatta una zona. Una risposta DNS trasporta uno o più record, e questi record, sparsi su molti server, **sono** il database, non è mai conservato interamente da nessuna parte.

**Definition**

**Resource record**

**Voce** del database distribuito DNS con cinque campi: **name**, **time to live**, **class**, **type**, **value**. Il tipo decide come leggere gli altri due: cosa significa il nome e cosa significa il valore.

La distribuzione DNS mappa i domini su valori tramite **resource record**, con i seguenti campi:

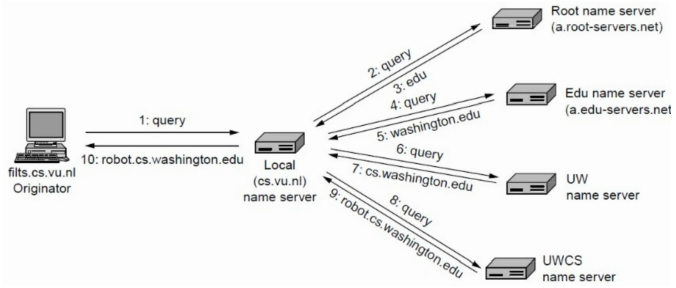
- **Domain name (NAME)**: dominio a cui il record si applica; è la **chiave di ricerca** principale delle query DNS; per ogni dominio esistono normalmente più record su server diversi.
- **Time to live (TTL)**: vita del record; lungo per host stabili (es. 86400 = 1 giorno), breve per host volatili (es. 60 = 1 minuto); massimo $2^{31} - 1$, circa 68 anni.
- **Class (CLASS)**: sempre `IN` (Internet) per i record Internet; altri codici sono raramente usati.
- **Type (TYPE)**: tipo del record (vedi sotto).
- **Value (RDATA)**: dipende dal tipo; può contenere un dominio, un valore o una stringa ASCII.

**Tipi di record principali:**

| Tipo | Significato | Valore |
|---|---|---|
| `SOA` | Start of authority | Parametri della zona, contatti dell'amministratore |
| `A` | Indirizzo IPv4 dell'host | Intero a 32 bit (più interfacce → più record) |
| `AAAA` | Indirizzo IPv6 dell'host | Intero a 128 bit |
| `MX` | Mail exchange | Priorità, dominio che accetta la posta per il dominio |
| `NS` | Name server | Nome di un server per il dominio (usato nella risoluzione) |

| Tipo | Significato | Valore |
|---|---|---|
| `CNAME` | Canonical name | Alias: la lookup prosegue col nuovo nome |
| `PTR` | Pointer | Alias per il reverse lookup (nome restituito senza procedere) |
| `SPF` | Sender policy framework | Codifica testuale della policy di invio mail |
| `SRV` | Service | Host che fornisce il servizio |
| `TXT` | Text | Testo ASCII descrittivo |

## Name server e zone




Un unico server DNS centralizzato non avrebbe senso: lo space DNS è partizionato in **zone non sovrapposte**; il partizionamento spetta all'amministratore della zona.

- Ogni zona contiene **uno o più name server**: uno primario e uno (o più) secondario.
- I name server contengono i dati delle porzioni di name space dette zone.

**Definition**

**Zona**

**Dominio**: un nodo dell'albero DNS e tutti i suoi discendenti (gerarchia di naming). **Zona**: la porzione di quel dominio gestita dalla **stessa autorità DNS**; delegando un sottodominio si crea una nuova zona, che però resta parte del dominio padre (es. se `cs.unina.it` è delegata, diventa una zona separata restando parte del dominio `unina.it`). Dominio = gerarchia di naming; zona = autorità amministrativa.

### Name resolution

La **risoluzione dei nomi** (name resolution) procede così:

1. L'host interroga un **server DNS locale**.
2. Se il dominio cade sotto la giurisdizione (zona) del server locale, viene restituito un record **authoritative**.
3. Altrimenti il server locale gestisce la query **ricorsivamente** e inoltra query remote: se il server DNS interrogato ha l'indirizzo lo restituisce, altrimenti restituisce il name server di una **zona più bassa**; il processo continua **iterativamente** fino a trovare l'indirizzo.

### Query ricorsive e iterative

- **Query ricorsiva**: chi viene interrogato deve produrre la risposta finale, o un errore; chi chiede aspetta e non fa nulla.
- **Query iterativa**: la risposta è o la risposta, o un *referral* al server successivo; chi chiede continua a lavorare.

Suddivisione dei ruoli:

- L'**host** chiede una volta, ricorsivamente: una query fuori, un indirizzo dentro.
- Il **server locale** fa il percorso, iterativamente, un server alla volta (**recursive DNS resolver**).
- I server **root, TLD e authoritative** rispondono normalmente in modo iterativo: restituiscono una risposta autoritativa o un referral, invece di fare ricorsione per il client.

### Root server

Sui **root server**:

- Sono il punto di partenza quando non si ha alcuna informazione.

- Sono **13**, da `a.root-servers.net` a `m.root-servers.net`, altamente replicati.
- Sono molto occupati, quindi in genere restituiscono informazioni su zone più basse, non sui singoli record.

Sui tipi di query e record:

- I server DNS locali gestiscono **query ricorsive** (servizio verso i propri host): restituiscono la risposta completa o un errore.
- Molti server DNS (i più occupati, es. quelli vicini alla cima) **non gestiscono** query ricorsive.
- I record sono **authoritative** oppure **cached**: tutte le risposte vengono messe in cache, i record cached **scadono** (ricorda il TTL) e non provengono dal server che gestisce la zona (risposte *non authoritative*); il caching migliora le prestazioni, poiché l'indirizzo può essere restituito subito dal server locale.
- I root server sono interrogati solo in **assenza totale** di informazioni su un dominio; se per `cs.mit.edu` arriva una richiesta e il server ha l'indirizzo del DNS server di `mit.edu`, può interrogare direttamente quello.

**DNS caching**

Ogni risposta ricevuta da un resolver può essere **messa in cache** e riusata per le query successive:

- La risposta di un resolver ricorsivo è **non-authoritative**: il resolver non è autoritativo per la zona interrogata, che la risposta arrivi dalla cache o sia appena risolta.
- Vengono messi in cache **non solo gli indirizzi** (A/AAAA), ma anche i record **NS** e altre informazioni DNS: questo evita di ripetere l'intero percorso di risoluzione; gli indirizzi dei server TLD in cache fanno sì che una risoluzione di solito **parta sotto la radice**.
- I record in cache valgono solo per il loro **TTL**, dopo il quale vanno rinfrescati.

> *Il caching è ciò che fa funzionare il tutto a questa scala ; e il TTL è ciò che impedisce a una copia in cache di mentire per sempre.*

Un resolver non parte necessariamente dalla radice: **parte dal punto noto più vicino nella gerarchia DNS**.

**Example**

Se deve risolvere `www.cs.mit.edu` e il record A/AAAA finale non è in cache, usa l'informazione già in cache **più specifica**:

| Informazione già in cache | La risoluzione parte da |
|---|---|
| A/AAAA per `www.cs.mit.edu` | Risposta immediata |
| NS per `cs.mit.edu` | Server autoritativo di `cs.mit.edu` |
| NS per `mit.edu` | Server autoritativo di `mit.edu` |
| NS per `.edu` | Server TLD `.edu` |
| Nessuna delegazione utile | Root server |

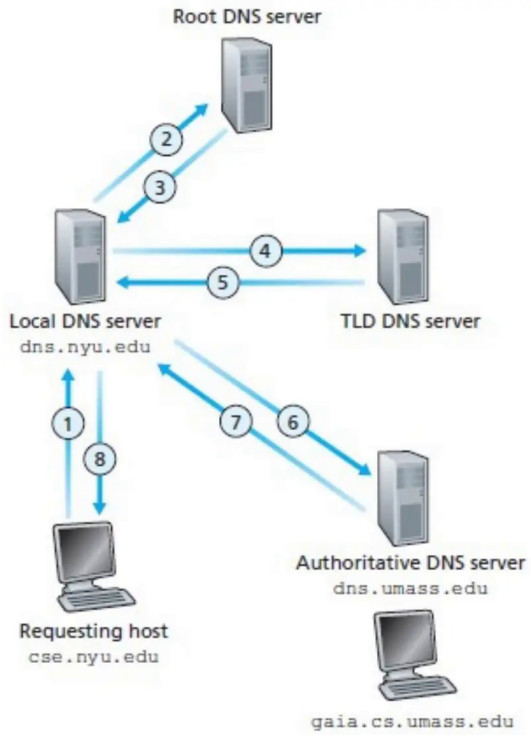
# Il resolver locale e i servizi aggiuntivi

Definition

**Local DNS server**

**Server DNS locale** (o resolver): tipo di server DNS che **non appartiene strettamente** alla gerarchia, tipicamente gestito dagli ISP; fa da **proxy**, inoltrando le query dell'host alla gerarchia DNS.

- Anche Google fornisce resolver pubblici (Google Public DNS) agli indirizzi `8.8.8.8` e `8.8.4.4`.
- Quando un host si connette, l'ISP gli fornisce gli IP di uno o più server DNS locali, tipicamente "vicini" all'host.




**Stub resolver**

Un host in genere **non esegue da solo** la risoluzione DNS: sull'host agisce uno **stub resolver**, che riceve le richieste delle applicazioni e invia **una query ricorsiva** al resolver ricorsivo configurato (ISP, istituzione o servizio pubblico); il resolver ricorsivo esegue la risoluzione vera e propria e restituisce la risposta finale. Su molti sistemi Linux è un servizio DNS locale a fare da stub/forwarder: è il caso di `127.0.0.53`.

Servizi aggiuntivi del DNS:

- **Aliasing**: servizio che associa nomi lunghi/complicati (nomi **canonicali**) a nomi più semplici (**alias**); un host con nome complicato può avere alias mnemonici (es. `relay1.west-coast.enterprise.com` → `www.enterprise.com`); l'applicazione può comunque chiedere al DNS il nome canonico di un alias insieme all'indirizzo IP.
- **Load distribution**: distribuzione del carico tra **server replicati** (*round-robin DNS*); i siti trafficati sono replicati su più server con indirizzi IP diversi (più IP associati a un solo nome canonico); il server può **ruotare l'ordine** degli indirizzi nelle risposte, così i client vengono distribuiti sulle repliche.

# Strumenti di laboratorio

### `nslookup`

Permette di eseguire manualmente la query iterativa normalmente effettuata dal server locale, ed effettuare anche il **reverse lookup** (dal nome all'indirizzo IP, e viceversa). Può anche interrogare uno specifico server DNS, riproducendo a mano il lavoro del resolver ricorsivo.

Lab

```bash
#Richiesta
❱ nslookup www.vatican.va

#Risposta
Server:         127.0.0.53
Address:        127.0.0.53#53

Non-authoritative answer:
www.vatican.va  canonical name = vatican-g5fcczeabnb9atb6.a03.azurefd.net.
vatican-g5fcczeabnb9atb6.a03.azurefd.net        canonical name = mr-a03.tm-azurefd.net.
Name:   mr-a03.tm-azurefd.net
Address: 150.171.110.52
Name:   mr-a03.tm-azurefd.net
Address: 2603:1061:14:133::1
```

> *Da notare nell'output: la catena di* canonical name *(aliasing in azione) e il fatto che la risposta è* non-authoritative*: la query va al servizio DNS locale* `127.0.0.53:53`*, che la inoltra al resolver ricorsivo configurato; l'host fa una query ricorsiva, è il resolver a percorrere iterativamente la gerarchia.*

Lab

### Verificare la configurazione DNS

```bash
resolvectl status
# oppure
cat /etc/resolv.conf
```

### `dig` (Domain Information Groper)

Oltre a `nslookup`, **`dig`** è uno strumento da riga di comando avanzato per interrogare i server DNS e ispezionare le risposte in un formato molto vicino alla struttura reale dei messaggi DNS.

**Funzionalità principali:**

- Interrogare **tipi di Resource Record specifici**: A, AAAA, NS, MX, SOA…
- Inviare query a uno **specifico server DNS**.
- Mostrare l'intero **percorso di delega** lungo la gerarchia con il parametro `+trace`.
- Analizzare nel dettaglio **intestazione e flag del messaggio**:
  - `rd` (*recursion desired*), `ra` (*recursion available*);
  - assenza di `aa` → risposta *non-authoritative*;
  - sezione *Question*, record della sezione *Answer* con i relativi TTL;
  - tempo di esecuzione della query, server interrogato, protocollo e dimensione del messaggio.

**Filtrare l'output:** con i flag `+noall +answer` si isola la sola sezione delle risposte.

Info

### Anatomia dell'output di dig vatican.va

L'output rispecchia la struttura del messaggio DNS (header di 12 byte + quattro sezioni):

- **Header**: flag `rd` richiesto e `ra` disponibile; **assenza di `aa`** → risposta *non-authoritative*;
- **Question**: il record richiesto;
- **Answer**: i resource record restituiti, con i relativi **TTL**;
- **Final**: tempo di esecuzione della query, server DNS contattato, protocollo e dimensione del messaggio.

Ricorda la struttura del messaggio DNS: **header** (identificatore copiato nella risposta, così il client può associare risposte a domande, + flag: authoritative answer, recursion desired, recursion available) · **Question** (nome richiesto e tipo) · **Answer** (i record che rispondono) · **Authority** (server che "saprebbero") · **Additional** (record utili, tipicamente gli indirizzi dei server appena nominati).

Lab

### Record A con catena di CNAME

```bash
❱ dig +noall +answer www.mit.edu A
```

Nella risposta compaiono **due CNAME prima dell'indirizzo**: il nome pubblicato è un alias di un altro nome, e la macchina che infine risponde appartiene a un soggetto completamente diverso. Osserva i **TTL**: l'alias è tenuto per minuti, l'indirizzo finale per pochi secondi.

Lab

### Record MX: priorità e mail server

```bash
# Record MX: TTL, classe, preferenza (valore più basso = priorità più alta), mail server
❱ dig +noall +answer mit.edu MX
```

La RDATA di un record MX contiene **due campi**: valore di preferenza e nome del mail server; **valori più bassi = priorità più alta**.

```
mit.edu.   65   IN   MX   100   mit-edu.mail.protection.outlook.com.
↑          ↑    ↑    ↑    ↑     ↑
TTL        classe TYPE pref.  mail server
```

Lab

### Record NS: l'NS RRset della zona

```bash
# Record NS: server di nome autoritativi per la zona (NS RRset)
❱ dig +noall +answer mit.edu NS
```

Una query NS può restituire **più record** che insieme formano un **NS RRset**: ciascuno identifica un name server autoritativo per la zona.

```
mit.edu.   1800   IN     NS     asia2.akam.net
└NAME———   └TTL   └CLASS └TYPE └authoritative name server
```

Capitolo 8

# Lezione 8 Programmazione di rete

## Applicazioni di rete

La maggior parte delle applicazioni di rete è composta da un programma lato client e un programma lato server che comunicano attraverso la rete: la loro esecuzione crea un processo client e un processo server, che comunicano leggendo e scrivendo su **socket**.

Definition

**Protocollo aperto**

**Protocollo** le cui regole sono pubbliche: chiunque può scrivere uno dei due lati e i due programmi interoperano.

Definition

**Protocollo proprietario**

**Protocollo** "custom", non pubblicato in un RFC o altrove; un solo team scrive entrambi i lati e controlla il formato, e nessun altro può scrivere un programma che vi dialoga.

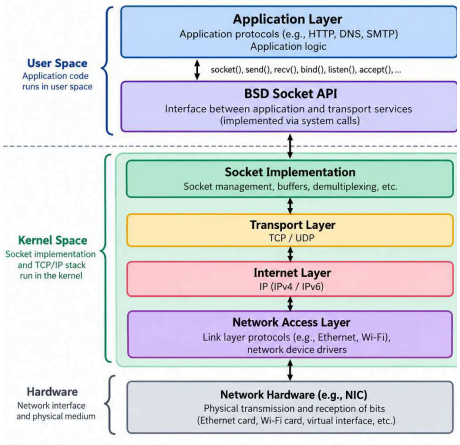
- Un protocollo aperto usa la sua **well-known port**; un protocollo proprio deve stare lontano da quelle porte.
- **TCP** (*Transmission Control Protocol*): orientato alla connessione, fornisce un canale affidabile di byte-stream attraverso cui i dati fluiscono tra i due sistemi terminali.
- **UDP** (*User Datagram Protocol*): connectionless, invia pacchetti indipendenti da un sistema all'altro senza garanzie di consegna.

## Dove sono implementati i livelli

*[!aside] La separazione tra user space e kernel space decide dove vivono protocolli e servizi.*

- *I protocolli di **livello applicativo** (web, DNS) sono implementati da **applicazioni in user space**, anche se i loro pacchetti attraversano tutto lo stack.*
- *Dal livello di **trasporto** in giù tutto è gestito nel **kernel space**, almeno su Unix/Linux e in generale sui kernel monolitici; sui microkernel la separazione è più articolata.*
- *Lo schema di interfacciamento è: applicazione (user space) → **API socket** → chiamate di sistema e servizi del kernel → trasporto → rete → accesso.*
- *Nel kernel il livello di accesso si appoggia ai **driver** delle schede di rete, che realizzano la parte fisica.*
- *Ogni host implementa gli strati secondo il proprio ruolo: i terminali implementano tutto lo stack, i nodi interni solo i livelli necessari al loro compito (per esempio fino al livello rete per instradare i pacchetti).*

---

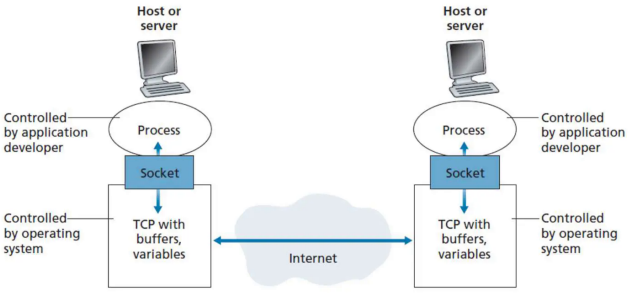


# Il socket

**Definition**

**Socket**

**Estremità** di un canale di comunicazione bidirezionale tra due processi, identificata da un descrittore.




- Introdotto in **BSD 4.2** nel 1983, è ancora lo standard de facto: Unix, macOS e Windows (con uno spelling diverso).
- I due processi possono essere sulla stessa macchina o su due macchine ai lati opposti del mondo: le chiamate non cambiano.
- Non è legato a TCP/IP: la stessa API ha servito altre famiglie di protocolli, ed è per questo che le chiamate hanno forma generica.
- Essendo un descrittore, accetta `read()`, `write()` e `close()`, le chiamate già note per i file.
- La **Socket API** dà alle applicazioni accesso ai servizi di trasporto implementati dallo stack di rete del sistema operativo.
- Diverse librerie, framework e middleware incapsulano la Socket API semplificandone l'uso, ma l'API base resta diffusamente usata.
- In dominio Unix si usano i **Berkeley socket** (detti anche BSD o POSIX socket), API native in C, sia per TCP sia per UDP; API simili sono disponibili in quasi tutti i linguaggi e sistemi operativi.

### Procedure client-server

Le procedure di comunicazione distinguono il modello connectionless e quello connection-oriented in quattro fasi.

| Fase | Connectionless (UDP) | Connection-oriented (TCP) |
|---|---|---|
| Creazione | Create Socket (client e server) | Create Socket (client e server) |
| Connessione | assente | client: `connect()`; server: attende le connessioni |
| Messaggi | Send/Read Message, Read/Send Reply | Send/Read Message, Read/Send Reply |
| Chiusura | Close Socket | Close Socket; il server chiude il socket client-specifico |

- Client e server devono creare socket **dello stesso tipo**: stessa famiglia di indirizzi (non può parlare una in IPv4 e l'altra in IPv6) e stesso tipo di canale.
- Lo schema è pensato per connessioni client/server ma non lo impone: entrambe le parti possono mandare messaggi.

# Creazione, chiusura e associazione

### `socket()`

I socket si creano con la funzione `socket()`, che restituisce il descrittore `sockfd` del nuovo socket.

```c
#include <sys/socket.h>
int socket(int domain, int type, int protocol);
```

- **`domain`**: specifica il dominio di rete e la famiglia di indirizzamento.
- **`type`**: specifica la semantica di comunicazione (stream o datagram).
- **`protocol`**: indica un protocollo particolare, spesso `0` per quello di default.
- I socket aderiscono alla filosofia Unix *"everything is a file"*: le sorgenti di I/O sono trattate come file e associate a un file descriptor.
- Il descrittore entra nella **tabella dei file aperti** del processo, accanto a standard input, output ed error (`0`, `1`, `2`): il canale è trattato in modo uniforme rispetto a file, pipe e FIFO.

I valori principali di `domain` sono i seguenti.

| Domain | Famiglia |
|---|---|
| `AF_INET` | IPv4 |
| `AF_INET6` | IPv6 |
| `AF_LOCAL` | due processi su questa macchina, attraverso il file system |

I valori principali di `type` sono i seguenti.

| Type | Canale |
|---|---|
| `SOCK_STREAM` | un byte stream: connesso, ordinato, affidabile |
| `SOCK_DGRAM` | messaggi indipendenti, senza connessione né garanzie |
| `SOCK_RAW` | il pacchetto è costruito a mano, sotto il livello di trasporto |

### `close()`

I socket si chiudono con la funzione `close()`, che restituisce `0` in caso di successo e `-1` altrimenti.

```c
#include <unistd.h>
int close(int socket_id);
```

- **`socket_id`**: è il file descriptor del socket da chiudere.

### `bind()`

La funzione `bind()` associa un socket a un indirizzo e a una porta locali, e restituisce `0` in caso di successo e `-1` altrimenti.

```c
int bind(int socket, const struct sockaddr *address,
         socklen_t address_len);
```

- **`socket`**: file descriptor del socket da associare.
- **`address`**: puntatore a una struttura `sockaddr` che specifica porta e indirizzo; `address.sin_addr` è spesso impostato a `INADDR_ANY` per ascoltare su tutti gli indirizzi (più IP).
- **`address_len`**: lunghezza della struttura `sockaddr`, ottenibile con `sizeof()`.
- La funzione è usata sui server, che devono associare il socket a una porta specifica, mentre sui client è opzionale: il sistema operativo assegna automaticamente una porta libera.
- Il bind **non crea una connessione**: crea solo l'associazione a un indirizzo; la connessione TCP richiede `listen()` e `accept()` sul server e `connect()` sul client.
- Nessuno garantisce l'assenza di **conflitti di porta**: una porta già occupata rende il bind fallimentare e, quando un canale viene chiuso, la porta resta per un periodo non riutilizzabile.

# Indirizzi IP, porte e ordine dei byte

### Strutture degli indirizzi

I socket di dominio internet in C si appoggiano ad alcune strutture che definiscono indirizzi e porte degli host mittente e destinatario. La struttura `sockaddr_in` (IPv4) e la struttura `sockaddr_in6` (IPv6) hanno forma analoga ma dimensioni diverse.

```c
struct sockaddr_in {
    sa_family_t sin_family;
    in_port_t sin_port;
    struct in_addr sin_addr;   // 4 B
    unsigned char sin_zero[8]; // per uniformare la dimensione
};                             // 16 B totali

struct sockaddr_in6 {
    sa_family_t sin6_family;
    in_port_t sin6_port;
    uint32_t sin6_flowinfo;
    struct in6_addr sin6_addr; // 16 B
    uint32_t sin6_scope_id;
};                             // ~28-32 B totali
```

Un socket internet è identificato da due elementi.

- Un **indirizzo IP**: intero a 32 bit (es. `143.225.5.3`).
- Una **porta**: intero a 16 bit, da 0 a 65535.

Le porte esistono per offrire diversi servizi dallo stesso indirizzo IP e si classificano per intervallo.

- **0–1023**: porte riservate (ai processi di root).
- **5000–32768**: porte utente.
- **Altre**: porte effimere, per i client, ai quali non interessa scegliere una porta specifica.
- `sin_zero` è padding di allineamento alla dimensione della struttura generica `sockaddr`; serve congruenza tra famiglia dichiarata, struttura usata e argomenti delle funzioni.
- L'equivalente IPv6 di `INADDR_ANY` è la costante `in6addr_any`, che fa ascoltare il socket su tutti gli indirizzi locali IPv6.

### Ordine dei byte

Un intero in memoria occupa 4 byte e può essere codificato in due modi; i processori possono essere di entrambi i tipi.

- **Big Endian**: è memorizzato prima il byte più significativo.
- **Little Endian**: è memorizzato prima il byte meno significativo.
- I protocolli TCP/IP lavorano con il big endian, chiamato **network byte order**, quindi sono necessarie funzioni di conversione.
- Le funzioni di conversione tra interi senza segno si chiamano combinando le iniziali: `h` = Host, `n` = Network (big endian), `l` = Long (4 byte), `s` = Short (2 byte).
- Ne derivano `htonl()`, `htons()`, `ntohl()` e `ntohs()`, dichiarate in `netinet/in.h` con argomenti `uint32_t` o `uint16_t`.
- Il programmatore non deve conoscere l'endianess della propria macchina: le funzioni di conversione fanno la traduzione localmente e rendono il codice portabile; prima di mandare un numero si converte con `htonl()`/`htons()`, dopo averlo ricevuto con `ntohl()`/`ntohs()`.
- La conversione serve per i dati numerici, non per i byte.

**Example**

```c
#include <netinet/in.h>

uint32_t htonl(uint32_t x);
uint16_t htons(uint16_t x);
uint32_t ntohl(uint32_t x);
uint16_t ntohs(uint16_t x);
```

| | |
|---|---|
| **h** | = Host |
| **n** | = Network (big endian) |
| **l** | = Long (4 bytes) |
| **s** | = Short (2 bytes) |

### `inet_pton()`

La funzione `inet_pton()` converte la notazione dotted di un indirizzo IPv4 o IPv6 in formato binario, riempiendo la struttura `in_addr` (IPv4) o `in6_addr` (IPv6).

```c
indirizzo.sin_family = AF_INET;
if (inet_pton(AF_INET, "143.225.5.3", &indirizzo.sin_addr) != 1)
    perror("inet_pton");
```

- Restituisce `1` in caso di successo, `0` per una stringa non valida e `-1` in caso di errore (es. famiglia non supportata).
- Per l'indirizzo si può usare `INADDR_ANY` con `htonl()`, mentre per IPv6 la costante globale equivalente è `in6addr_any`.
- `inet_ntop()` fa l'inverso: converte il formato binario in notazione dotted.

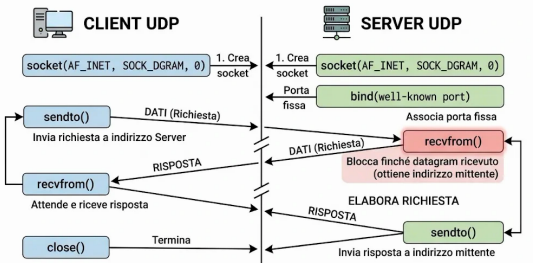
# Lezione 9 Programmazione con UDP

**Definition**

**UDP**

**Protocollo di trasporto** semplice, senza connessione e senza garanzie di affidabilità: mentre TCP trasmette un flusso di byte, UDP invia pacchetti discreti chiamati *datagram*.

- Non richiede handshake di stabilimento o terminazione, quindi introduce meno overhead di TCP e può essere più veloce per piccole quantità di dati.
- Quando una perdita occasionale di dati è accettabile, UDP fornisce una comunicazione molto rapida ed efficiente.




## `sendto()` e `recvfrom()`

L'invio e la ricezione di messaggi in UDP sono eseguiti dalle funzioni `sendto()` e `recvfrom()`, che restituiscono il numero di byte inviati o ricevuti, oppure `-1` in caso di errore.

```c
ssize_t sendto(int sockfd, const void *buf, size_t len, int flags,
               const struct sockaddr *dest_addr, socklen_t addrlen);
ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags,
                 struct sockaddr *src_addr, socklen_t *addrlen);
```

### Parametri

- **`sockfd`**: descrittore del socket su cui inviare o ricevere.
- **`buf`**: puntatore al buffer dei dati da inviare o ricevere.
- **`len`**: numero di byte da inviare, o massimo numero di byte ricevibili.
- **`dest_addr`/`src_addr`**: puntatori alle strutture `sockaddr` con l'indirizzo di destinazione, o in cui viene restituito l'indirizzo del mittente.
- **`addrlen`**: dimensione della struttura di indirizzo, per valore in `sendto()` e per puntatore in `recvfrom()`.

Entrambe le funzioni accettano un parametro `flags` che ne controlla il comportamento.

| Flag | Descrizione |
|---|---|
| `0` | comportamento di default, nessuna opzione speciale |
| `MSG_DONTWAIT` | operazione non bloccante; ritorna subito se bloccerebbe |
| `MSG_PEEK` | legge un datagram senza rimuoverlo dalla coda di ricezione |
| `MSG_TRUNC` | su Linux, recupera la lunghezza completa del datagram anche se il buffer è troppo piccolo |
| `MSG_WAITALL` | attende tutti i byte richiesti; utile soprattutto con TCP, non combina più datagram |

- Entrambe le parti devono essere pronte a ricevere: per ogni `sendto()` deve esserci dall'altra parte un processo su una `recvfrom()`.

- La configurazione più tipica è **simmetrica**: entrambe le parti fanno il bind, così ciascuna può mandare e ricevere; il client può comunque ometterlo grazie al binding implicito.

**Comportamento *Blocking* di default**

**Modalità attiva** :

- `sendto()` può bloccare finché il datagram è accettato dal sistema operativo, senza attendere la consegna al ricevente.
- `recvfrom()` blocca finché un datagram è disponibile.

**Gestione attesa** : Per non attendere indefinitamente si usano `MSG_DONTWAIT` o un timeout di ricezione (`SO_RCVTIMEO`).

**Warning** Se il datagram ricevuto è più grande del buffer, i byte in eccesso sono di norma scartati.

## Flusso di un server UDP

Il ciclo tipico di un server UDP è un loop di ricezione e risposta su un solo socket.

C

```c
s = socket(AF_INET, SOCK_DGRAM, 0);
bind(s, (struct sockaddr *)&addr, sizeof(addr));
while (1) {
    n = recvfrom(s, buf, sizeof(buf), 0,
                 (struct sockaddr *)&from, &flen);
    sendto(s, buf, n, 0, (struct sockaddr *)&from, flen);
}
```

- Il server riceve con `recvfrom()` (che riempie la struttura del mittente con IP e porta del client) e risponde con `sendto()` verso quell'indirizzo: può così rispondere anche a un client che non ha fatto il bind.
- Il buffer di ricezione raccoglie byte: per trattarli come stringa si aggiunge il terminatore alla posizione `n`.

**Lab** **Esempio Server**

C

```c
#include <netinet/in.h>
#include <arpa/inet.h>
#include <string.h>
#include <unistd.h>

int main(void) {
    int s;
    struct sockaddr_in addr = {0};
    char buf[1024];

    s = socket(AF_INET, SOCK_DGRAM, 0);

    addr.sin_family = AF_INET;
    addr.sin_port = htons(5200);
    addr.sin_addr.s_addr = htonl(INADDR_ANY);

    bind(s, (struct sockaddr *)&addr, sizeof(addr));

    while (1) {
        struct sockaddr_in from = {0};
        socklen_t flen = sizeof(from);

        int n = recvfrom(s, buf, sizeof(buf), 0,
                         (struct sockaddr *)&from, &flen);

        sendto(s, buf, n, 0,
               (struct sockaddr *)&from, flen);
    }

    close(s);
}
```

Lab

### Esempio client

```c
#include <netinet/in.h>
#include <arpa/inet.h>
#include <string.h>
#include <unistd.h>
#include <stdio.h>

int main(void) {
    int s;
    struct sockaddr_in addr = {0};
    char buf[1024] = "ciao";

    s = socket(AF_INET, SOCK_DGRAM, 0);

    addr.sin_family = AF_INET;
    addr.sin_port = htons(5200);
    inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr);

    sendto(s, buf, strlen(buf), 0,
           (struct sockaddr *)&addr, sizeof(addr));

    int n = recvfrom(s, buf, sizeof(buf), 0, NULL, NULL);

    /* ... usa buf ... */
    buf[n] = '\0';
    printf("ricevuto dal server: %s\n", buf);

    close(s);
}
```

## Binding implicito

Quando un socket UDP è creato con `socket()` non è ancora associato a una porta locale; il client può lasciare che sia il sistema operativo a farlo.

- Se il client chiama `sendto()` senza prima chiamare `bind()`, il sistema operativo assegna automaticamente una porta locale disponibile (**binding implicito**).
- L'indirizzo IP sorgente è scelto in base all'interfaccia di rete e alla configurazione di routing.
- La porta assegnata resta associata al socket finché non viene chiuso; il sistema cerca di dare **porte alte**, evitando quelle basse per non entrare in conflitto.

Warning

**Binding implicito**

Il binding implicito **non stabilisce una connessione** tra client e server.

`getsockname()` interroga il socket per conoscere indirizzo, porta e interfaccia effettivamente assegnati dal sistema.

Error

**Bind esplicito dopo quello implicito**

**Operazione fallimentare a runtime**: una successiva chiamata a `bind()` sullo stesso socket fallisce, perché il socket è già associato; per associare di nuovo l'indirizzo bisogna chiudere il socket e aprirne un altro.

## Ordine di avvio di client e server

L'esito della comunicazione dipende da quale dei due programmi parte prima.

- **Client prima del server**: se un datagram raggiunge l'host di destinazione ma nessun socket è associato alla porta di destinazione, lo strato UDP lo scarta e può essere generato un messaggio ICMP *Port Unreachable*.
- Il server, quando parte, non può recuperare il datagram perso; il client può restare bloccato su `recvfrom()` o ricevere un errore (es. `ECONNREFUSED`).
- **Server prima del client**: il server chiama `bind()` e attende su `recvfrom()`, poi riceve il datagram e invia la

risposta, che il client riceve.

Info

**Nessun buffering per chi non ascolta**

UDP non mette in coda i datagram per applicazioni che non sono in ascolto, e i datagram persi non sono ritrasmessi automaticamente.

- Dev'esserci sempre un **processo in ascolto nel momento giusto**: le sincronizzazioni si ottengono con le primitive bloccanti.

## Timeout di ricezione

Poiché `recvfrom()` è bloccante, è possibile configurare un timeout di ricezione con `setsockopt()` e l'opzione `SO_RCVTIMEO`.

C

```c
struct timeval timeout = {5, 0};
setsockopt(sockfd, SOL_SOCKET, SO_RCVTIMEO,
           &timeout, sizeof(timeout));
```

- Se un datagram arriva prima dello scadere del timeout, `recvfrom()` restituisce i dati ricevuti.
- Se nessun datagram arriva entro 5 secondi, `recvfrom()` restituisce `-1` con `errno` pari a `EAGAIN` o `EWOULDBLOCK`: non sono equivalenti, e si controllano entrambi per portabilità tra sistemi.
- Il timeout non ritrasmette i datagram persi.

## Più client su un solo socket

UDP non richiede che una connessione sia stabilita tra client e server, il che semplifica la gestione di più client.

- Un server UDP usa un **solo socket** associato a una porta nota.
- Più client possono inviare datagram allo stesso socket del server.
- Ogni `recvfrom()` riceve un datagram e fornisce indirizzo IP e porta del mittente.
- Il server usa `sendto()` per inviare la risposta al client corrispondente.
- Non sono richiesti `listen()`, `accept()` né un socket separato per ogni client.
- È più semplice del TCP quando i messaggi sono brevi in entrambe le direzioni e i client sono molti; non c'è garanzia se due datagram arrivano insieme mentre il server non è ancora tornato in `recvfrom()`.

## Comunicazione su localhost

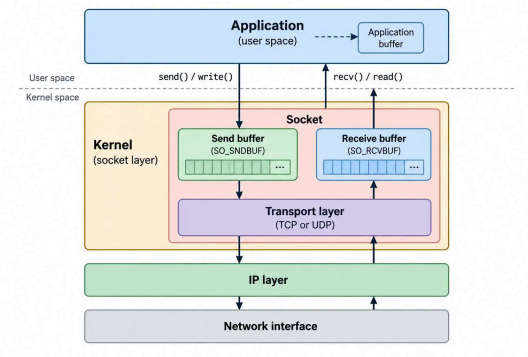
Nella comunicazione su `localhost` lo stack di rete viene attraversato fino al livello di trasporto, che rimanda indietro il pacchetto (**loopback**): nessun pacchetto reale viaggia sulla rete.

- Questi esempi servono a prendere confidenza con le API sulla stessa macchina, senza attraversare la rete fisica.

## Buffer dei socket UDP

Un socket UDP ha due buffer gestiti dal kernel: un buffer di invio per i datagram in uscita e un buffer di ricezione per quelli in arrivo.

- La dimensione dei due buffer è configurabile con le opzioni `SO_SNDBUF` e `SO_RCVBUF`.
- Ogni `sendto()` invia un datagram UDP completo: socket bloccante, attende finché il datagram può essere accettato; socket non bloccante, restituisce `-1` con `EAGAIN/EWOULDBLOCK` se il datagram non può essere accettato.
- Il blocco può essere interrotto (per esempio da un segnale): anche in modalità bloccante i dati vanno controllati.
- UDP non supporta invii parziali: il datagram completo è accettato oppure l'operazione fallisce, tutto intero o niente, anche se il payload può essere un frammento del dato applicativo.
- Ogni `recvfrom()` recupera al più un datagram; se il buffer dell'applicazione è più piccolo del datagram ricevuto, questo è troncato e i byte in eccesso sono scartati, irrecuperabili con ricezioni successive.
- In modalità non bloccante, se la coda è vuota la gestione si riduce a un polling (busy waiting).

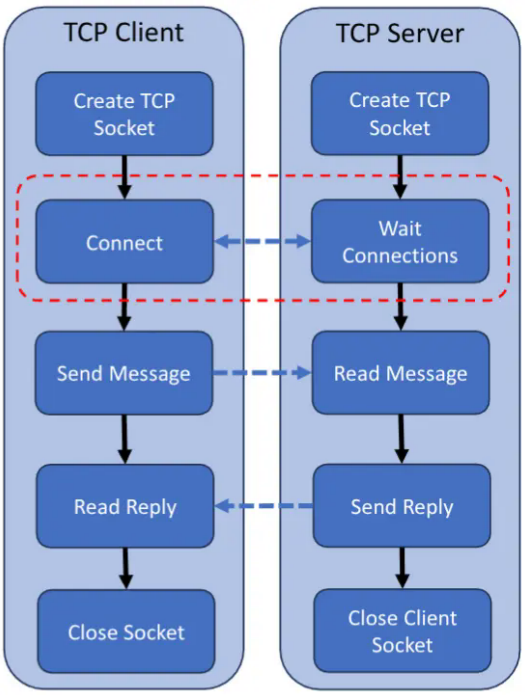


# Lezione 10 Programmazione con TCP (Connessioni e Flusso)

## Dal socket UDP al modello TCP

A differenza di UDP, in TCP client e server devono completare un handshake prima di trasmettere messaggi; sul lato server servono due socket distinti.

- **Welcoming socket**: sempre attivo, permette ai client di eseguire l'handshake con il server.
- **Socket client-specifico**: creato dopo l'handshake, usato da client e server per comunicare.
- Il **three-way handshake** avviene nello strato di trasporto ed è completamente invisibile ai due programmi.
- Una volta che il server accetta la connessione (handshake riuscito), la comunicazione passa al socket appena creato.




## `connect()`

La connessione TCP lato client è eseguita dalla funzione `connect()`, che restituisce `0` in caso di successo e `-1` altrimenti.

```c
int connect(int socket, const struct sockaddr *address,
            socklen_t address_len);
```

### Parametri

- **`socket`**: file descriptor del socket con cui connettersi.
- **`address`**: puntatore a una struttura `sockaddr` con l'indirizzo del server.
- **`address_len`**: lunghezza della struttura, ottenibile con `sizeof()`.
- `connect()` è **bloccante**: è il punto di sincronizzazione con la `accept()` del server, da cui la comunicazione prosegue nelle due direzioni.

Info

**connect() con UDP**

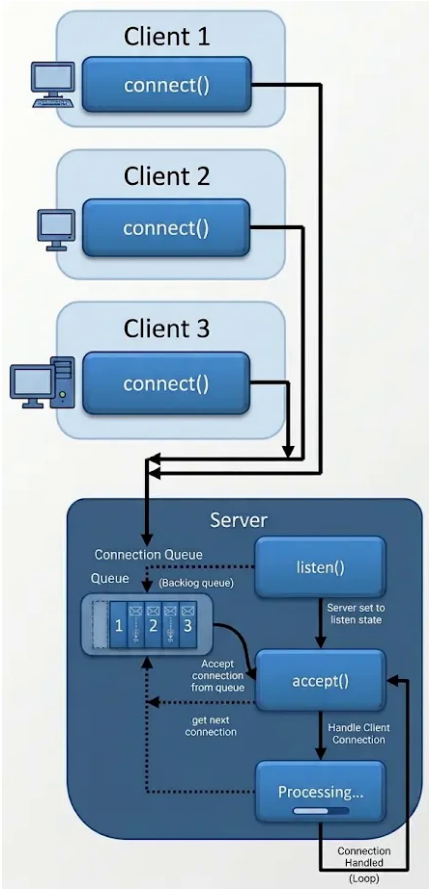
`connect()` può essere usata anche con UDP: a differenza di TCP **non stabilisce una connessione**, imposta solo un indirizzo di destinazione di default e filtra i datagram in entrata; la comunicazione resta UDP, con perdite comprese.

# `listen()` e `accept()`

L'attesa di connessioni lato server (*wait-for-connection*) è eseguita dalle funzioni `listen()` e `accept()`, che operano in combinazione con la `connect()` del client.

```c
int listen(int sockfd, int backlog);
int accept(int sockfd, struct sockaddr *addr,
           socklen_t *addrlen);
```

- `listen()` marca il socket come **listening socket**, pronto ad accettare connessioni; si usa con i tipi orientati alla connessione come `SOCK_STREAM` e `SOCK_SEQPACKET`.
- **`backlog`**: lunghezza massima della coda di connessioni pendenti, unica per tutti i client che si collegano, cioè quante connessioni completate possono attendere prima che il kernel inizi a rifiutarle o ignorarle (di solito le rifiuta).
- `accept()` restituisce un **nuovo descrittore**, oppure `-1`: crea un nuovo socket dedicato a quella connessione, mentre il vecchio socket resta in ascolto.
- Il secondo e terzo argomento di `accept()` identificano il client e possono essere `NULL`.
- `accept()` blocca il processo se non vi sono connessioni in attesa; il socket può essere marcato non bloccante.




**Info**

**Due generi di socket sul server**

Il socket di ascolto è creato una volta, associato alla porta nota e **non trasporta mai dati**; ogni `accept()` restituisce un socket di connessione dedicato a un solo client. Un server che serve tre client detiene quattro descrittori.

- Stabilita la connessione, i due processi comunicano come su un'**inter-process communication**: chi parla e chi ascolta va definito a livello applicativo, perché se entrambi si mettono su `send()` (o entrambi su `recv()`) non comunicano.

## Chiusura a quattro segmenti

La terminazione TCP avviene a quattro segmenti, due per ciascuna direzione di chiusura.

- Il client chiama `close()`: parte un segmento **FIN**, confermato dall'**ACK** del server; dopo la `close()` il client entra nello stato **FIN_WAIT_1** e, ricevuto l'ACK, passa a **FIN_WAIT_2**: la sua socket è chiusa ma il canale resta aperto, in attesa della chiusura del server.
- Quando anche il server chiude il socket client-specifico, manda il suo **FIN** (stato **LAST_ACK**); il client risponde con l'ultimo **ACK** e passa allo stato **TIME_WAIT**.
- Il server, ricevuto l'ultimo ACK, può chiudere senza dubbi: l'incertezza sull'arrivo dell'ultimo messaggio resta solo al client, che per questo mantiene la finestra di attesa.

Definition

**TIME_WAIT**

**Periodo refrattario** dopo la chiusura attiva, con due scopi: consentire la ritrasmissione dell'ACK finale se il peer, non ricevendolo, ritrasmette il suo FIN; lasciar scadere i segmenti ritardati della vecchia connessione prima che gli stessi identificatori siano riusati.

- `shutdown(socket, how)` chiude selettivamente una direzione del canale: `SHUT_RD` per la lettura, `SHUT_WR` per la scrittura, `SHUT_RDWR` per entrambe; su Linux interrompe anche una `accept()` bloccata, quindi è il modo ordinato di sbloccare il server alla ricezione di un segnale.

## Lettura e scrittura

L'invio e la ricezione in TCP sono eseguiti da `read()`, `write()`, `send()` e `recv()`, più semplici delle controparti UDP, e usano la stessa interfaccia della lettura/scrittura su file.

- I socket descriptor si possono passare a processi figli, che non devono distinguere tra socket descriptor e file descriptor.
- Le stesse funzioni si possono usare anche con processi che lavorano su file locali.
- `write(fd, buf, count)`: invia il contenuto del buffer al socket, si usa esclusivamente con `SOCK_STREAM`, e restituisce il numero di byte inviati oppure `-1`; è la stessa funzione della scrittura su file.
- `read(fd, buf, count)`: solo per socket connessi (`SOCK_STREAM`), legge al massimo `count` byte; se non c'è alcun messaggio il programma rimane sospeso (chiamata bloccante); restituisce il numero di byte letti, `-1` in caso di errore, `0` se il peer ha chiuso; è la stessa funzione della lettura da file.

Esistono inoltre funzioni specifiche con modalità aggiuntive: tre varianti di send e tre di receive.

- **`send()`**: come `write()` (richiede connessione stabilita), ma accetta flag per specificare la modalità di scrittura.
- **`sendto()`**: permette la scrittura su socket connectionless, perché richiede l'indirizzo di destinazione; resta utilizzabile anche su socket connessi, col destinatario implicito nella connessione.
- **`sendmsg()`**: per specificare buffer multipli.
- **`recv()`**: come `read()` ma con flag; con flag `0` è identica a `read()`.
- **`recvfrom()`**: permette anche di ottenere l'indirizzo della sorgente.
- **`recvmsg()`**: per ricevere da buffer multipli.
- `send()` può essere usata solo se il socket è connesso; `sendto()` sempre; `recv()` e `recvfrom()` ricevono in `buf` al più `len` byte e se `from` non è `NULL` riempiono la struttura `sockaddr` con l'indirizzo del mittente; i flag si possono lasciare a `0`.

I flag abilitano funzionalità extra rispetto a `write()`/`read()`; senza flag il comportamento è lo stesso.

| Flag | Effetto |
|---|---|
| `MSG_DONTWAIT` | non bloccante |
| `MSG_PEEK` | legge senza modificare la coda di ricezione |
| `MSG_WAITALL` | attende finché non si raggiungono `len` byte |
| `MSG_NOSIGNAL` | su `send()`, evita `SIGPIPE` |

- `MSG_DONTWAIT` su `recv()`: non attende i dati (es. polling); su `send()`: non attende se il buffer è pieno.
- `MSG_PEEK` su `recv()`: esamina i dati prima di rimuoverli dalla coda.
- `MSG_WAITALL` su `recv()`: attende finché sono letti `len` byte; resta comunque interrompibile da segnali ed errori, quindi il valore di ritorno va controllato.

Warning

**SIGPIPE**

Scrivere su un **canale rotto** (l'estremo di lettura ha chiuso) genera il segnale **SIGPIPE**: se non è catturato o ignorato, il comportamento di default è la **terminazione del processo** che lo riceve; la schermatura del segnale andrebbe comunque predisposta oltre a `MSG_NOSIGNAL`.

### Opzioni del socket

Le opzioni si leggono e si configurano con `getsockopt()` e `setsockopt()`, che restituiscono `0` in caso di successo e `-1` in caso di errore (con `errno` impostato).

```c
int getsockopt(int sockfd, int level, int optname,
               void *optval, socklen_t *optlen);
int setsockopt(int sockfd, int level, int optname,
               const void *optval, socklen_t optlen);
```

- **`level`**: livello di protocollo in cui l'opzione è definita: `SOL_SOCKET` (opzioni a livello socket), `IPPROTO_TCP` (opzioni TCP), `IPPROTO_IP` (opzioni IPv4).
- **`optname`**: identifica l'opzione.
- **`optval`**: puntatore al valore dell'opzione; in `getsockopt()` `optlen` è un puntatore a una lunghezza di input/output.
- Le opzioni non riguardano solo il livello del socket: si può scendere fino ai livelli più bassi dello stack di rete, e vanno impostate **prima del bind** (`SO_REUSEADDR`, `SO_REUSEPORT`, `SO_RCVTIMEO`, `SO_SNDBUF`/`SO_RCVBUF`).

### `SO_REUSEADDR`

Definition

**SO_REUSEADDR**

**Opzione a livello socket** (`SOL_SOCKET`) che permette di riutilizzare un indirizzo e una porta locali in certe condizioni, in particolare quando connessioni TCP precedenti sono in `TIME_WAIT`.

- `optval` è un puntatore a un intero: valore non nullo abilita l'opzione, valore zero la disabilita; `optlen` è `sizeof(int)`.
- L'opzione va impostata **prima** di chiamare `bind()`.

```c
int opt = 1;
setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
```

- Alla chiusura di una connessione TCP, l'endpoint che esegue la chiusura attiva entra nello stato `TIME_WAIT` per un periodo limitato.
- Le connessioni in `TIME_WAIT` possono impedire a un nuovo socket di fare `bind()` sullo stesso indirizzo e porta: riavviare un server TCP poco dopo lo stop può fallire con *Address already in use*.
- `SO_REUSEADDR` consente il riuso, secondo le regole del sistema operativo; non consente normalmente a due server TCP di ascoltare sullo stesso IP e porta contemporaneamente.

### `SO_REUSEPORT`

Definition

**SO_REUSEPORT**

**Opzione a livello socket** (non parte dello standard POSIX) che permette a più socket di fare `bind()` sullo stesso indirizzo IP e sulla stessa porta.

- Con TCP, più socket possono chiamare indipendentemente `listen()` sullo stesso endpoint: il kernel distribuisce le connessioni in arrivo tra i socket in ascolto, fornendo un bilanciamento di carico integrato.
- Il kernel organizza i socket compatibili in una **reuseport group** e su Linux distribuisce le connessioni con selezione basata su hash; ogni socket in ascolto ha la propria coda di `accept()`.
- L'opzione va abilitata su ogni socket prima di chiamare `bind()`.
- È utile per il bilanciamento di carico tra processi o thread, per server ad alte prestazioni multicore e per ridurre la contesa su un singolo socket in ascolto.
- Su Linux i processi che condividono lo stesso indirizzo e porta devono avere lo stesso effective user ID.

## Esempi pratici

Tieni presente che, essendo frammenti minimali mancano delle direttive `#include` necessarie (come `<sys/socket.h>`, `<netinet/in.h>`, `<arpa/inet.h>`, `<string.h>`, `<stdio.h>`, `<stdlib.h>`, `<unistd.h>`) per poter essere compilati ed eseguiti direttamente.

Lab

### Client Code

```c
int main(void) {
    char buf[256];
    struct sockaddr_in srv;
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) { perror("socket"); exit(EXIT_FAILURE); }
    memset(&srv, 0, sizeof srv);
    srv.sin_family = AF_INET;
    srv.sin_port   = htons(5000);
    inet_pton(AF_INET, "127.0.0.1", &srv.sin_addr);  /* not INADDR_ANY */
    if (connect(fd, (struct sockaddr *) &srv, sizeof srv) < 0) {
        perror("connect"); exit(EXIT_FAILURE); }
    write(fd, "hello\n", 6);
    ssize_t n = read(fd, buf, sizeof buf - 1); /* room for the '\0' */
    if (n > 0) { buf[n] = '\0'; printf("%s", buf); }
    close(fd);
}
```

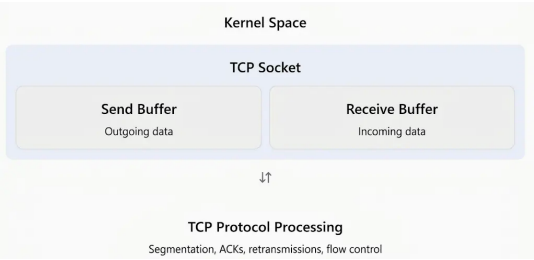
Lab

### Server Code

```c
struct sockaddr_in me;
int yes = 1;
int listen_fd = socket(AF_INET, SOCK_STREAM, 0);
if (listen_fd < 0) { perror("socket"); exit(EXIT_FAILURE); }
setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);
memset(&me, 0, sizeof me);
me.sin_family      = AF_INET;
me.sin_port        = htons(5000);
me.sin_addr.s_addr = htonl(INADDR_ANY);      /* any local interface */
if (bind(listen_fd, (struct sockaddr *) &me, sizeof me) < 0) {
    perror("bind"); exit(EXIT_FAILURE); }
if (listen(listen_fd, 16) < 0) {
    perror("listen"); exit(EXIT_FAILURE); }
```

# Buffer dei socket TCP

Anche i socket TCP hanno due buffer: un buffer di ricezione per i dati in entrata e un buffer di invio per i dati in uscita, entrambi in spazio kernel; non vanno confusi con il buffer applicativo definito dal programmatore.




- Le dimensioni si configurano con `SO_SNDBUF` (invio) e `SO_RCVBUF` (ricezione), anche in modo asimmetrico.
- Un buffer di invio più ampio può accodare più dati in uscita riducendo il blocking in `send()`; un buffer di ricezione più ampio riduce il rischio di stalli del controllo di flusso TCP.
- I buffer possono essere liberi, vuoti o parzialmente occupati: ciò che si vuole leggere o scrivere può non coincidere con l'intero contenuto.

- `write()` può restituire **meno byte di quelli richiesti**: il valore di ritorno indica i byte accettati per la trasmissione, non necessariamente consegnati al ricevente.
- Socket bloccante: `write()` attende che tutti i byte richiesti siano accettati nel buffer, salvo interruzioni o timeout di invio; socket non bloccante: restituisce subito un conteggio parziale oppure `EAGAIN/EWOULDBLOCK`.
- `read()` con `n` byte copia dal buffer di ricezione fino a `n` byte: socket bloccante, restituisce subito i byte disponibili senza attendere il riempimento del buffer, e blocca solo se non c'è alcun dato; socket non bloccante, restituisce i byte disponibili oppure `-1` con `EAGAIN/EWOULDBLOCK`.
- Un valore di ritorno `0` indica che il peer ha chiuso il lato di invio e tutti i dati pendenti sono stati letti.
- C'è una **asimmetria** tra le primitive: la `write()` bloccante è "paziente" (prova a trasferire tutto), la `read()` è "impaziente" (restituisce subito la parte disponibile).

Example

**Esempio con write(sockfd, buffer, 1000) mostra le differenze tra le due modalità.**

| Spazio libero nel buffer di invio | `write()` bloccante | `write()` non bloccante |
|---|---|---|
| 0 byte | attende spazio libero | `-1` (`EAGAIN/EWOULDBLOCK`) |
| 300 byte | normalmente attende di trasferire tutti i 1000 byte | può restituire 300 |
| 1000 byte | normalmente restituisce 1000 | normalmente restituisce 1000 |

Example

**Esempio con read(sockfd, buffer, 1000) è speculare.**

| Dati disponibili | `read()` bloccante | `read()` non bloccante |
|---|---|---|
| 0 byte | attende i dati | `-1` (`EAGAIN/EWOULDBLOCK`) |
| 300 byte | normalmente restituisce 300 | normalmente restituisce 300 |
| 1000 byte | può restituire 1000 | può restituire 1000 |

## Modalità non bloccante

Il metodo POSIX per rendere non bloccante un descrittore è impostare il flag `O_NONBLOCK` con `fcntl()`.

C

```c
int flags = fcntl(fd, F_GETFL, 0);
fcntl(fd, F_SETFL, flags | O_NONBLOCK);
```

- I flag sono **maschere di bit**: si recuperano con `F_GETFL` e si reimpostano con `F_SETFL` aggiungendo `O_NONBLOCK` ai vecchi flag.
- `recv()`/`read()`: restituiscono `-1` con `EAGAIN` o `EWOULDBLOCK` se non ci sono dati disponibili.
- `send()`/`write()`: restituiscono `-1` con `EAGAIN` o `EWOULDBLOCK` se i dati non possono essere accettati immediatamente.
- `connect()`: può restituire `-1` con `EINPROGRESS` mentre la connessione viene stabilita.
- Il meccanismo vale per TCP, UDP, pipe e altri file descriptor supportati.
- `MSG_DONTWAIT` abilita il comportamento non bloccante **per una singola chiamata** a `send()` o `recv()`, senza cambiare la modalità del socket.

## Lettura e scrittura complete

Poiché `read()` e `write()` possono trasferire meno byte di quelli richiesti, per leggere o scrivere `n` byte in sicurezza servono cicli che ripetono l'operazione finché tutti i byte sono trasferiti.

- Su `EINTR` la chiamata è stata interrotta da un segnale: si riprova.
- Su ritorno `0` la connessione è stata chiusa: ci si ferma restituendo i byte trasferiti fino a quel momento.
- Il **puntatore avanza** nel buffer con l'aritmetica dei puntatori: ogni volta che si trasferisce una quantità di byte, ci si sposta per trattare i rimanenti.

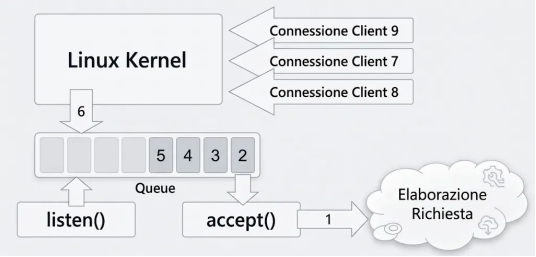
```c
while (sent < n) {
    w = send(fd, p + sent, n - sent, 0);
    if (w < 0) { if (errno == EINTR) continue; return -1; }
    sent += w;
}
```

- Lo schema è identico per la lettura robusta, che ritorna i byte letti finora anche in caso di chiusura del canale.

# Lezione 11 Server Concorrenti e Multiplexing I/O

Un generico server attende le richieste di connessione su una porta determinata, ma le richieste possono arrivare in modo concorrente da molteplici client.

- Soluzione senza programmazione concorrente: i client sono serviti **uno alla volta**; finché una connessione non è terminata non vengono serviti gli altri client interessati al servizio.
- Il tempo di attesa di ogni client dipende dalla durata delle altre connessioni e può risultare eccessivo.
- Se le connessioni durano molto, il server ben presto diverrebbe indisponibile anche solo ad accodare le nuove richieste; nei server reali le richieste dei vari client vanno gestite concorrentemente.
- Anche in un server sequenziale è comunque il sistema operativo a gestire le richieste concorrenti, che di fatto serializza.




## Server a processi con `fork()`

Un server concorrente genera un processo ad hoc per ogni client, così che più client siano serviti in modalità concorrente; l'attesa in coda si limita alla gestione della richiesta e all'avvio del nuovo processo.

```c
while (1) {
    fd2 = accept(fd1, NULL, NULL);
    if ((pid = fork()) == 0) {  // processo figlio
        close(fd1);             // al figlio non serve fd1
        /* gestisce la connessione con fd2 */
        exit(0);
    }
    close(fd2);                 // il padre ripete il ciclo
}
```

- Il `fork()` restituisce al padre il **PID del figlio** e al figlio `0`: dentro lo stesso codice si distingue il ramo del padre da quello del figlio.
- Padre e figlio nascono **indistinguibili** (a parte il PID) e condividono i descrittori.
- Il figlio chiude il socket di ascolto `fd1` e gestisce la connessione con `fd2`; il padre chiude `fd2` e ripete il ciclo: le chiusure sono simmetriche, ciascun processo chiude solo ciò che non usa.

Warning

**Processi zombie**

Se il padre non raccoglie l'**exit status** dei figli, questi restano **zombie**: il descrittore resta occupato in attesa della consegna e, su un server pensato per restare in piedi a lungo, si accumula un **memory leak**.

Per evitare processi **zombie** si installa un gestore di `SIGCHLD` che raccoglie tutti i figli terminati.

- Il gestore chiama `waitpid(-1, NULL, WNOHANG)` in un ciclo, senza bloccarsi se non ci sono figli.
- La registrazione avviene con `sigaction()`, impostando `SA_RESTART` perché le system call interrotte (`accept()`, `recv()`) vengano riavviate automaticamente; all'arrivo del segnale il flusso di controllo è interrotto, si esegue l'handler, poi si riprende.
- In alternativa si imposta `SIG_IGN` su `SIGCHLD`, che equivale a mettere i figli in **detach**: il loro destino non è più legato al monitoraggio del padre.
- `signal()` è l'alternativa storica e più semplice: associa direttamente la funzione al segnale.

## Server multithreaded

In alternativa ai processi si può servire ogni connessione con un thread.

```c
connect_sd = accept(listen_sd, NULL, NULL);
int *thread_sd = malloc(sizeof(int));
*thread_sd = connect_sd;
pthread_create(&tid, NULL, gestisci, thread_sd);
pthread_detach(tid);
```

- I thread **condividono lo spazio di indirizzamento**: due `accept()` consecutive scriverebbero il nuovo descrittore nella stessa variabile, sovrascrivendolo.
- `malloc()` fornisce un contenitore separato per il descrittore di ogni thread, evitando race condition nel passaggio degli argomenti.
- La **gestione della distruzione** della memoria allocata può essere centralizzata o delegata al thread in uscita, che sa cosa distruggere.
- `pthread_detach()` fa sì che le risorse del thread siano rilasciate automaticamente alla sua terminazione, senza chiamare `pthread_join()`.

## Più connessioni sulla stessa porta

Un server TCP può gestire più connessioni simultanee su una sola porta in ascolto.

- Il server crea il socket in ascolto con `socket()`, `bind()` e `listen()`.
- Ogni `accept()` restituisce un nuovo socket connesso, mentre il socket in ascolto resta disponibile per le nuove connessioni.
- Tutti i socket connessi condividono la stessa porta locale del server.
- Ogni connessione TCP è identificata univocamente da una **4-tupla**: (Source IP, Source Port, Destination IP, Destination Port).
- Il numero di connessioni è limitato dalle risorse di sistema, non dalla sola porta in ascolto.
- Il binding implicito assegna ai client porte diverse, distinguendo anche client con lo stesso IP.

Le principali limitazioni al numero di connessioni simultanee sono le seguenti.

- **File descriptor**: ogni socket connesso richiede un file descriptor; il sistema operativo impone limiti per processo e di sistema.
- **Memoria del kernel**: ogni connessione TCP richiede strutture di controllo e buffer di invio/ricezione.
- **Risorse CPU**: gestire molte connessioni attive aumenta l'overhead di elaborazione.
- **Porte sorgente dei client**: le connessioni dallo stesso IP client verso lo stesso endpoint server richiedono porte sorgente distinte (numeri a 16 bit).
- **Architettura del server**: un modello a processo o thread per client consuma più risorse di un I/O multiplexing event-driven.

Info

**Esaurimento delle porte sorgente**

L'esaurimento delle porte sorgente limita le connessioni **da un particolare IP client** verso uno specifico endpoint server, non il numero totale di connessioni che un server può accettare.

## Multiplexing I/O con `select()`

Definition

**select()**

**Funzione di I/O multiplexing** che attende eventi di I/O su più socket simultaneamente e permette all'applicazione di identificare quali socket sono pronti per la lettura o la scrittura.

- Evita di bloccarsi su un socket mentre altri hanno dati disponibili.
- Elimina la necessità di un thread o processo separato per ogni client, evitandone la proliferazione al crescere dei client (al prezzo di overhead e maggiore complessità del codice).

- È utile per gestire più client in un server **single-threaded**.

```c
#include <sys/select.h>
int select(int nfds, fd_set *readfds, fd_set *writefds,
           fd_set *exceptfds, struct timeval *timeout);
```

- `select()` blocca finché uno di questi eventi si verifica: un descrittore diventa pronto in lettura o scrittura; occorre una condizione eccezionale su un descrittore monitorato; scade il timeout; una signal interrompe la chiamata.
- Valore di ritorno: `-1` in caso di errore (`errno` impostato), `0` se il timeout è scaduto, valore positivo pari al numero totale di descrittori pronti negli insiemi restituiti.

## Insiemi di descrittori

I tre insiemi di descrittori da monitorare sono puntatori a variabili `fd_set`, tipicamente implementate come bitmask.

- **`readfds`**: descrittori da monitorare in lettura; ad esempio un listening socket è pronto quando si può accettare una connessione con `accept()`.
- **`writefds`**: descrittori da monitorare in scrittura.
- **`exceptfds`**: descrittori da monitorare per condizioni eccezionali (spesso `NULL`).
- **`nfds`**: specifica l'intervallo di descrittori esaminati; se `maxfd` è il descrittore più alto monitorato, allora `nfds = maxfd + 1`.
- `FD_SETSIZE` definisce la capacità di un `fd_set` (tipicamente 1024 su Linux).

Gli insiemi `fd_set` si manipolano con quattro macro.

| Macro | Effetto |
|---|---|
| `FD_SET(fd, set)` | aggiunge `fd` all'insieme |
| `FD_CLR(fd, set)` | rimuove `fd` dall'insieme |
| `FD_ISSET(fd, set)` | restituisce vero se `fd` è nell'insieme |
| `FD_ZERO(set)` | svuota l'insieme |

- Dopo il ritorno di `select()`, `FD_ISSET()` verifica se un descrittore è pronto (0 = non pronto, non-zero = pronto).

Warning

**Gli insiemi sono modificati**

`select()` **modifica gli insiemi** passati, mantenendo solo i descrittori che si sono resi pronti: vanno ricostruiti prima di ogni chiamata partendo dall'insieme completo.

- Poiché la `select()` si sblocca solo quando c'è qualcosa da fare, l'operazione successiva sul descrittore pronto **non si blocca**: si è certi che ci sia un dato da ricevere (o spazio per scrivere).

## Condizioni di prontezza

`select()` identifica i descrittori pronti per operazioni di I/O senza bloccarsi.

Un socket è pronto in lettura (`readfds`) quando si verifica una di queste condizioni.

- Ci sono dati nel buffer di ricezione (`SO_RCVLOWAT`, default: 1 byte).
- Il peer ha chiuso il suo lato di invio (EOF).
- C'è un errore pendente sul socket.
- Un listening socket ha connessioni in attesa di `accept()`.

Un socket è pronto in scrittura (`writefds`) quando si verifica una di queste condizioni.

- C'è spazio sufficiente nel buffer di invio per un'operazione di scrittura (anche solo in parte).
- Un tentativo di connessione non bloccante è completato (con successo o con errore).
- C'è un errore pendente sul socket.

Warning

**Prontezza in scrittura**

La prontezza in scrittura **non garantisce** che una `send()` di grandi dimensioni completi senza bloccare: lo sblocco attesta solo che c'è un po' di spazio, quindi i byte effettivamente scritti vanno comunque controllati.

L'insieme `exceptfds` monitora condizioni eccezionali: per TCP è usato principalmente per rilevare dati **Out-of-Band (OOB)**.

- TCP segnala dati urgenti con il flag `URG`.
- Su Linux i dati urgenti si ricevono con `recv()` e il flag `MSG_OOB`.

## Timeout

Il parametro `timeout` specifica il tempo massimo per cui `select()` attende che un descrittore diventi pronto.

```c
struct timeval {
    time_t tv_sec;        // secondi
    suseconds_t tv_usec;  // microsecondi
};
```

- **`timeout == NULL`**: `select()` blocca indefinitamente finché un descrittore non diventa pronto o la chiamata è interrotta.
- **`tv_sec == 0 && tv_usec == 0`**: `select()` restituisce immediatamente senza bloccare (polling).
- **Timeout non nullo**: `select()` attende al massimo il tempo specificato, ma ritorna prima se uno o più descrittori diventano pronti.

## Server e client multiplexing

Un server multiplexing gestisce più client con un solo processo o thread, monitorando sia il socket di ascolto sia i socket dei client connessi.

- Si inizializzano le strutture `fd_set` per la lettura (e, se serve, la scrittura) e vi si aggiungono il listening socket e i socket dei client connessi.
- Quando `select()` ritorna: se il listening socket è pronto si accetta una nuova connessione e il nuovo descrittore entra nell'insieme; se un socket client è pronto si leggono o scrivono dati su di esso.
- Il contatore dei descrittori pronti va decrementato a ogni gestione e il ciclo su di essi si interrompe quando raggiunge zero.
- Alla chiusura di un client (`recv()` restituisce 0 o errore): `close()`, `FD_CLR()` e rimozione del descrittore dalla lista dei client.
- Un server single-threaded può così gestire più client senza creare thread o processi per ogni connessione.

Un client multiplexing monitora in lettura due descrittori.

- **Standard input** (`stdin`): per leggere i messaggi digitati dall'utente.
- **Socket connesso** (`sockfd`): per ricevere i messaggi dal server.
- Con `select()` il client risponde al descrittore che diventa pronto, senza bloccarsi sull'altro.

Capitolo 12

# Lezione 12 Broadcast, Multicast e Risoluzione dei Nomi (DNS)

Broadcast e multicast sono meccanismi a livello IP, usati comunemente con socket UDP (non standard TCP).

- **Broadcast**: supportato da IPv4.
- **Multicast**: supportato da IPv4 e IPv6.
- IPv6 non ha il broadcast: si usa il multicast al suo posto.

## Broadcast

Definition

**Broadcast**

**Invio** di dati a tutti gli host di una rete locale, senza richiedere l'adesione a un gruppo.

- A livello rete IPv4 permette l'invio di un pacchetto a tutti gli host di una rete; nella programmazione di socket si usa tipicamente con UDP, perché UDP supporta la comunicazione connectionless, mentre TCP non supporta il broadcast.
- **Directed broadcast**: `192.168.1.255`, dipende dalla subnet mask.
- **Limited broadcast**: `255.255.255.255`, non inoltrato dai router.
- Mittente: crea un socket con `socket(AF_INET, SOCK_DGRAM, 0)`, abilita il broadcast con `setsockopt()` e `SO_BROADCAST`, e invia il datagram all'indirizzo di broadcast e alla porta di destinazione con `sendto()`.
- Ricevente: crea un socket UDP, fa `bind()` su `INADDR_ANY` e sulla porta di destinazione, e riceve i datagram con `recvfrom()`.

## Multicast

Definition

**Multicast**

**Invio** di datagram UDP a un gruppo di riceventi identificati da un indirizzo IP multicast (es. `239.255.0.1`).

- Per ricevere datagram multicast un socket deve **aderire al gruppo** con l'opzione `IP_ADD_MEMBERSHIP`.
- Il mittente imposta il TTL dei pacchetti multicast in uscita con `IP_MULTICAST_TTL`; con TTL pari a 1 il pacchetto resta nella LAN locale, e invia all'indirizzo del gruppo **senza conoscere i singoli riceventi**.
- Il ricevente imposta `SO_REUSEADDR` (con `SO_REUSEPORT` per permettere a più client di fare `bind()` sullo stesso indirizzo), fa `bind()` su `INADDR_ANY` e sulla porta, aderisce al gruppo e riceve in un ciclo.

```c
struct ip_mreq mreq;
inet_pton(AF_INET, "239.255.0.1", &mreq.imr_multiaddr);
mreq.imr_interface.s_addr = htonl(INADDR_ANY); // interfaccia scelta dall'OS
setsockopt(sock, IPPROTO_IP, IP_ADD_MEMBERSHIP, &mreq, sizeof(mreq));
```

- La struttura `ip_mreq` specifica l'indirizzo del gruppo multicast e l'interfaccia di rete locale da usare.

Quando un'applicazione aderisce a un gruppo multicast, sistema operativo e rete cooperano alla gestione della comunicazione.

- **Group membership**: il ricevente richiede l'adesione con `IP_ADD_MEMBERSHIP`.
- **Sistema operativo**: gestisce la richiesta e comunica le informazioni necessarie alla rete locale.
- **Rete**: i router multicast-aware usano le informazioni di appartenenza per inoltrare i pacchetti verso le reti con riceventi interessati.
- **Consegna dei dati**: il mittente invia i datagram all'indirizzo IP multicast senza conoscere i singoli riceventi.

Info

**Socket option e rete**

Le socket option permettono alle applicazioni di richiedere comportamenti a livello di rete, mentre la rete sottostante gestisce la consegna dei pacchetti.

# Risoluzione dei nomi DNS

I socket Berkeley funzionano solo con indirizzi IP, ma su Internet gli host sono tipicamente identificati da hostname.

- Gli hostname appartengono al livello applicativo, mentre i socket operano al livello di trasporto: per connettersi tramite hostname serve la traduzione DNS per ottenere l'indirizzo IP da passare al socket.
- C/C++ offre funzioni e strutture che contattano i server DNS e recuperano gli indirizzi.

### `gethostbyname()`

**Definition**

**gethostbyname()**

**Funzione legacy** che risolve un hostname in indirizzi IPv4 tramite il resolver di sistema, restituendo una struttura `hostent` (`NULL` in caso di errore).

- La struttura non contiene solo l'indirizzo IP, ma informazioni sull'host: nome canonico, eventuali alias, uno o più indirizzi.
- Un DNS può restituire una lista di indirizzi, in ordine anche casuale se la query arriva a server round-robin.
- Il primo elemento della lista si ottiene con `h_addr_list[0]` (o `h_addr`); l'indirizzo risolto può inizializzare una `sockaddr_in`.

```c
struct hostent {
    char  *h_name;      // nome originale dell'host (canonico)
    char **h_aliases;   // array di alias, terminato da NULL
    int    h_addrtype;  // famiglia dell'indirizzo (tipicamente AF_INET)
    int    h_length;    // lunghezza dell'indirizzo (tipicamente 4 byte)
    char **h_addr_list; // array di indirizzi, terminato da NULL
};
```

- Esiste anche `gethostbyaddr()`, che restituisce una struttura `hostent` per un indirizzo IP specificato.

### `getaddrinfo()`

**Definition**

**getaddrinfo()**

**Funzione moderna** che risolve un hostname in uno o più indirizzi IP (IPv4/IPv6) con un'interfaccia unificata, restituendo le informazioni necessarie per creare e connettere un socket.

- Restituisce una lista concatenata di strutture `addrinfo` contenenti le informazioni pronte per `socket()` e `connect()`.
- La struttura `hints` specifica i criteri con cui `getaddrinfo()` seleziona gli indirizzi adatti; `res` punta alla lista collegata degli indirizzi corrispondenti.

```c
struct addrinfo hints, *res;
memset(&hints, 0, sizeof(hints));
hints.ai_family = AF_UNSPEC;      // IPv4 + IPv6
hints.ai_socktype = SOCK_STREAM; // TCP
err = getaddrinfo("www.example.com", "80", &hints, &res);
```

- **`ai_family`**: famiglia di indirizzi (`AF_INET`, `AF_INET6`, `AF_UNSPEC`).
- **`ai_socktype`**: tipo di socket (`SOCK_STREAM`, `SOCK_DGRAM`).
- **`ai_protocol`**: protocollo di trasporto specifico (`0` = non specificato).
- **`ai_flags`**: opzioni aggiuntive che controllano la risoluzione degli indirizzi.

Gli indirizzi restituiti si usano in sequenza finché uno non si connette.

```c
for (p = res; p != NULL; p = p->ai_next) {
    sock = socket(p->ai_family, p->ai_socktype, p->ai_protocol);
```

```
    if (connect(sock, p->ai_addr, p->ai_addrlen) == 0)
        break;  // connesso
    close(sock);
}
freeaddrinfo(res);
```

- `p->ai_addr` si può usare direttamente con `connect()`; per accedere all'IP e stamparlo va fatto il cast a `sockaddr_in` (IPv4) o `sockaddr_in6` (IPv6).
- `freeaddrinfo()` libera la memoria allocata dinamicamente da `getaddrinfo()`; gli errori si descrivono con `gai_strerror()`.

## Richieste HTTP su socket TCP

Una richiesta HTTP su un socket TCP specifica tre elementi: hostname del server web, numero di porta e path della risorsa da recuperare.

- La stringa di richiesta contiene metodo, path e versione del protocollo, e si invia con `send()`; la risposta si riceve con `recv()` in un ciclo fino alla chiusura della connessione da parte del server.
- Con `Connection: close` la connessione è **non persistente**: basta quando si vuole controllare una sola pagina.

# Parte III

# Transport Layer

Capitolo 13

# Lezione 13 Il livello di trasporto

## Comunicazione logica tra processi

Il livello di trasporto fornisce **comunicazione logica** tra processi applicativi in esecuzione su host diversi: dal punto di vista dell'applicazione, i processi appaiono connessi direttamente come se fossero sulla stessa macchina, qualunque cosa si frapponga tra loro (router, collegamenti).

- Al **sender**, il livello di trasporto incapsula i dati dell'applicazione: **TCP** organizza il flusso di byte in **segmenti**, mentre **UDP** incapsula i singoli messaggi in **datagram**, preservandone i confini.
- Ogni segmento/datagram riceve un'intestazione di trasporto e viene passato al livello di rete, che lo incapsula in un datagramma IP e tenta una consegna *best-effort*.
- Al **receiver**, il livello di trasporto fa il demultiplexing dei segmenti/datagram in arrivo verso la socket appropriata: TCP consegna un flusso di byte ordinato, UDP consegna i singoli datagram.
- La segmentazione è resa **trasparente all'applicazione**: è il livello di trasporto che spezza lo stream, e la rete provvede eventualmente a una frammentazione ulteriore.
- I protocolli di trasporto sono implementati **solo nei sistemi finali**, mentre il routing segue la topologia e deve girare sui nodi: il trasporto connette processi, la rete connette host.

Info

**Segmentazione e frammentazione**

La **segmentazione TCP** e la **frammentazione IP** sono meccanismi diversi che operano a livelli diversi. A livello di trasporto lo stream viene diviso in segmenti; a livello di rete i segmenti vengono spezzati in frammenti più o meno piccoli secondo le caratteristiche della linea e i vincoli di rappresentazione, ad esempio l'**MTU**. Proprio per questi vincoli strutturali, qualche informazione dal livello sottostante può emergere verso il trasporto: la separazione dei livelli non è completamente impermeabile.

## Responsabilità del livello di trasporto

I protocolli di trasporto (UDP e TCP) hanno quattro responsabilità principali.

- **Consegna processo-processo**: i messaggi raggiungono i processi destinatari indipendentemente da dove si trovano, a differenza della consegna host-to-host tra due macchine, che è responsabilità del protocollo IP.
- **Controllo di integrità**: campi di rilevamento errori inclusi nelle intestazioni dei segmenti.
- **Trasferimento affidabile dei dati**: i dati arrivano dal processo mittente al processo destinatario corretti e in ordine.
- **Controllo di congestione e di flusso**: evita di sommergere i dispositivi di rete (collegamenti o router) con un traffico eccessivo, con beneficio alle prestazioni e all'intera rete.

UDP, più veloce e semplice, fornisce solo i primi due servizi; TCP fornisce tutti e quattro.

## Multiplexing e demultiplexing

A livello di applicazione più processi accedono alla rete contemporaneamente, e un processo può essere connesso a più processi nello stesso momento (proxy, mail server): non esiste una corrispondenza uno-a-uno tra processi e connessioni. Le **socket** forniscono gli endpoint della comunicazione processo-processo, e non c'è una socket per forza per ogni processo: un server può gestire più connessioni concorrenti.

Definition

**Multiplexing e demultiplexing**

**Operazioni complementari** del livello di trasporto: il **multiplexing** (lato sender) raccoglie i dati da socket diverse, aggiunge le intestazioni di trasporto con i numeri di porta e passa i segmenti/datagram risultanti al livello di rete; il **demultiplexing** (lato receiver) esamina le informazioni nei pacchetti ricevuti e ne consegna i dati alla socket appropriata.

L'apertura di una socket espone un **buffer di ricezione a livello kernel**: è il demultiplexing, gestito dallo stack di rete del kernel, a indirizzare lì i dati in arrivo, e l'applicazione li preleva con `recvfrom()`.

### Demultiplexing senza connessione in UDP

Nel client UDP la socket si crea con `socket(AF_INET, SOCK_DGRAM, 0)` e tipicamente non viene legata a una porta sorgente: è il sistema operativo a sceglierne una libera (es. 46428), benché sia sempre possibile il `bind()` esplicito; la destinazione si imposta nella struttura `sockaddr_in` passata a `sendto()`.

- Nel demultiplexing **connectionless** di UDP, il datagram in arrivo viene consegnato alla socket in base a due soli campi: l'**indirizzo IP di destinazione** e la **porta di destinazione**.
- Per questo una socket UDP è identificata da due numeri, mentre una socket TCP da quattro: è la ragione per cui TCP necessita di `accept()` e UDP no.
- La `recvfrom()` riempie la struttura `cliaddr` con indirizzo e porta del mittente: la stessa socket può così inviare la risposta (`sendto()`) a più porte e indirizzi sorgente, chiunque sia il client.

### Porta di destinazione non raggiungibile

Se un datagram UDP arriva su una porta senza socket in ascolto si verifica la seguente sequenza.

- Il livello UDP della macchina ricevente tenta il demultiplexing con le informazioni di destinazione, non trova socket corrispondenti e **scarta** il datagram.
- L'host genera normalmente un messaggio **ICMP Destination Unreachable (Port Unreachable)** verso il mittente, la cui consegna però non è garantita.
- Il sistema operativo può riportare l'errore all'applicazione (es. `ECONNREFUSED` su Linux); una `recvfrom()` bloccante può comunque attendere indefinitamente se non si configura un timeout.

UDP non garantisce la consegna né offre meccanismi di instaurazione della connessione: le applicazioni devono gestire da sole le risposte mancanti e i timeout.

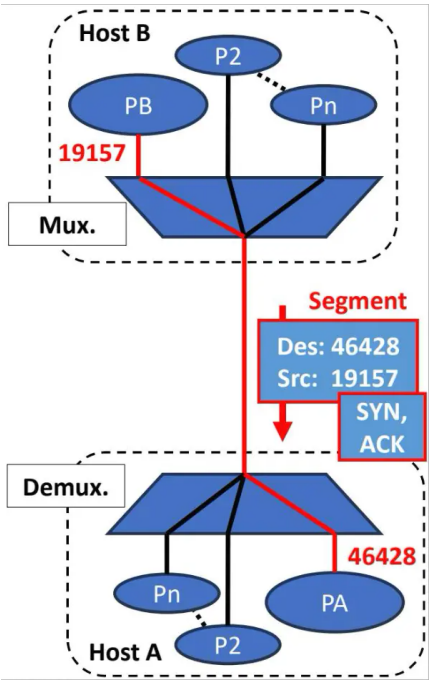
### Demultiplexing in TCP

Nel TCP la comunicazione parte da una **welcoming socket** del server, in ascolto su una porta nota (es. 19157): il client crea la socket con `socket(AF_INET, SOCK_STREAM, 0)` e con `connect()` invia un **segmento di richiesta di connessione**, cioè un segmento TCP con porta di destinazione 19157 e il bit **SYN** a 1 nell'intestazione.

La connessione si stabilisce con il **three-way handshake**.

1. **SYN**: il client invia un segmento con SYN = 1 per richiedere la connessione.
2. **SYN-ACK**: il server risponde con un segmento con SYN = 1 e ACK = 1.
3. **ACK**: il client riconosce il SYN del server con un segmento con ACK = 1.

I tre segmenti portano le porte sorgente e destinazione che identificano gli endpoint; completato l'handshake, `accept()` sulla socket in ascolto restituisce una **nuova socket connessa** e riempie `cliaddr` con indirizzo e porta del client. Il transport layer registra i quattro valori letti nel segmento in arrivo: ogni segmento successivo con gli stessi quattro valori viene consegnato alla nuova socket, e la socket in ascolto può continuare ad ascoltare.

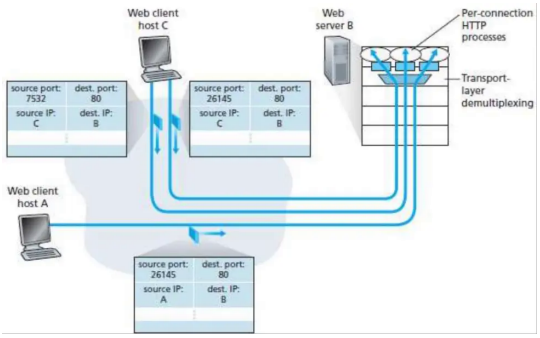



**Definition**

**4-tuple**

**Insieme di quattro valori** che identifica una connessione TCP stabilita: indirizzo IP sorgente, porta sorgente, indirizzo IP di destinazione, porta di destinazione. Più connessioni possono condividere la stessa porta del server e restare distinte per le loro 4-tuple.

Host diversi possono riusare lo stesso numero di porta sorgente: nell'esempio del libro l'host C apre due sessioni HTTP verso il server B assegnando alle connessioni le porte sorgente 26145 e 7532, e l'host A può usare la porta 26145 per la propria sessione senza saperlo di C, perché le due connessioni differiscono per l'indirizzo IP sorgente. Il server apre tipicamente un processo (o thread) per ogni connessione in arrivo, assegnandogli la nuova socket; troppi processi o thread possono però comprometterne le prestazioni.




# Porte

I protocolli di trasporto usano i numeri di porta per distinguere gli endpoint di comunicazione e supportare multiplexing e demultiplexing; porta sorgente e destinazione compaiono nelle intestazioni TCP e UDP.

**Definition**

**Porta**

**Numero intero a 16 bit** (da 0 a 65535) che identifica un endpoint di comunicazione a livello di trasporto; la classificazione è curata dalla IANA (*Internet Assigned Numbers Authority*).

Le porte si distinguono in tre categorie.

- **Well-known** (0–1023): universalmente note e assegnate ai servizi storici.
- **Registered** (1024–49151): assegnate dalla IANA su registrazione, ad es. MySQL su `3306/TCP`; sono una convenzione, non un obbligo, e un servizio può essere configurato su un'altra porta.
- **Dynamic/private** (49152–65535): libere, ad esempio per le porte sorgente scelte dai client.

Le porte ben note assegnate ai servizi principali sono le seguenti.

| Porta | Servizio |
|---|---|
| `20` | FTP data transfer |
| `21` | FTP command control |
| `22` | SSH |
| `23` | Telnet, remote login con messaggi di testo non cifrati |
| `25` | SMTP, instradamento e-mail |
| `53` | DNS |
| `80` | HTTP |
| `110` | POP3, recupero e-mail dal server |
| `119` | NNTP |
| `123` | NTP |
| `143` | IMAP |
| `161` | SNMP |
| `443` | HTTPS (HTTP su TLS/SSL) |

### Ispezione delle porte con nmap e ss

`nmap` (*Network Mapper*) opera dalla prospettiva di rete: manda sonde a un host remoto, per nome o indirizzo, per scoprire host e servizi e individuare le porte aperte, cioè quelle su cui un'applicazione è in ascolto.

- `nmap scanme.nmap.org` scansiona le 1000 porte TCP più comuni; `--top-ports N` limita alle N più usate; `-p 22,80,443` specifica le singole porte.

- `-sV` attiva il *service version detection*: interroga le porte aperte per identificare il servizio effettivo e la versione del software; senza `-sV` i nomi dei servizi sono dedotti dalle assegnazioni convenzionali.

Ogni porta può risultare in uno di tre stati.

- **open**: qualcosa è legato a quella porta ed è in ascolto.
- **closed**: l'host ha risposto, ma nulla è in ascolto su quella porta.
- **filtered**: nessuna risposta è tornata: un firewall ha ingoiato la sonda e lo stato resta sconosciuto.

`ss` opera invece dalla prospettiva locale del sistema operativo, ispezionando le socket direttamente dal kernel: `ss -tulnp` mostra le socket locali con `-t` (TCP), `-u` (UDP), `-l` (in ascolto), `-n` (indirizzi e porte numerici), `-p` (processi associati). La differenza chiave: `nmap` scopre i servizi accessibili via rete su host remoti, `ss` mostra le socket aperte sulla macchina locale, a un livello più basso.

# UDP

Definition

**UDP**

**Protocollo di trasporto connectionless**, senza handshake tra le entità di trasporto prima dell'invio, che fornisce **consegna processo-processo** (porte, multiplexing e demultiplexing) e **controllo di errori** (checksum), e nient'altro: nessuna garanzia di consegna, nessun controllo di congestione o di flusso. È un protocollo minimalista che aggiunge a IP solo "l'essenziale".

### Perché UDP invece di TCP

Le ragioni per preferire UDP sono le seguenti.

- **Controllo a livello applicativo**: UDP è più diretto e lascia all'applicazione più controllo sulla trasmissione, utile nelle applicazioni real-time dove ritmo di invio e ritardi sono cruciali mentre la perdita di dati è tollerabile.
- **Instaurazione rapida della connessione**: niente handshake, quindi meno ritardo nel stabilire la comunicazione, ad es. per DNS.
- **Nessuno stato di connessione**: niente parametri di controllo della congestione, né numeri di sequenza o di riconoscimento; un server regge tipicamente molti più client attivi su UDP che su TCP.
- **Overhead minimo sui pacchetti**: il segmento TCP aggiunge 20 byte di intestazione, il datagram UDP solo 8.

### Uso di UDP

Le applicazioni che richiedono trasferimento affidabile (e-mail, terminale remoto, Web, file transfer) girano su TCP o su protocolli affidabili basati su UDP come **QUIC**; SNMP usa UDP perché la gestione di rete deve funzionare proprio quando la rete è sotto stress, cioè quando il trasferimento affidabile è difficile; le applicazioni multimediali (telefonia Internet, videoconferenza, streaming) usano spesso sia UDP sia TCP, perché si può tollerare una piccola perdita di pacchetti, mentre le applicazioni real-time reagiscono molto male al controllo di congestione di TCP.

| Servizio | Protocollo applicativo | Trasporto |
|---|---|---|
| E-mail | SMTP | TCP |
| Remote terminal | Telnet | TCP |
| Web | HTTP | TCP o UDP |
| File transfer | FTP | TCP |
| Name translation | DNS | UDP |
| Gestione dispositivi di rete | SNMP | UDP |
| Streaming multimediale | Proprietary | UDP o TCP |
| Telefonia Internet | Proprietary | UDP o TCP |

Warning

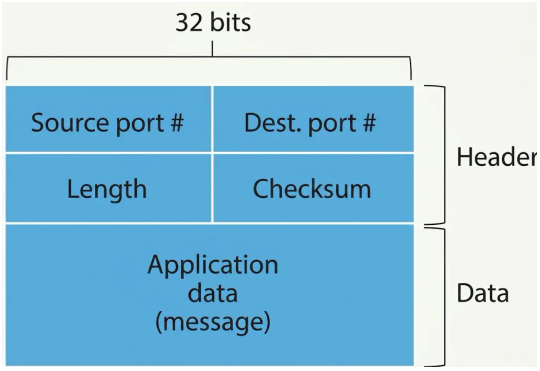
**Multimedia su UDP**

Fare streaming ad alto bitrate su UDP senza controllo di congestione è controverso: se tutti iniziassero a trasmettere selfish senza controllo, i dispositivi di rete andrebbero in overflow causando la perdita di più pacchetti UDP; i tassi di perdita indotti dai sender UDP fuori controllo farebbero abbassare drasticamente i tassi dei sender TCP.

### Formato del datagram

Il datagram UDP è composto da un'intestazione di 8 byte e dai dati dell'applicazione (N byte).

- L'intestazione contiene quattro campi da 16 bit ciascuno: **source port** (porta del processo mittente), **destination port** (porta del processo destinatario), **length** e **checksum**.
- **Length**: lunghezza dell'intero datagram in byte, intestazione + dati (N + 8 byte).
- **Checksum**: usato dal ricevente per verificare che il messaggio sia intatto.




Il campo Length a 16 bit rappresenta valori da 0 a $2^{16} - 1 = 65535$, quindi il payload teorico massimo è $65535 - 8 = 65527$ byte. Il livello IP impone però vincoli ulteriori: in IPv4 il payload UDP massimo è 65507 byte (con intestazione IP di 20 byte); anche i limiti di rappresentazione del livello di rete vincolano le dimensioni, perché un datagram troppo grande non risulta neanche spezzettabile dal livello sottostante.

### Checksum

Info

**Calcolo del checksum UDP**

Il sender calcola il **complemento a 1** della somma di tutte le parole da 16 bit del datagram (più la *pseudo-header* IP); l'eventuale bit di overflow viene ri-sommato alla cifra meno significativa (*end-around carry*): il bit che rappresentava $2^{17}$ torna a valere 1. Il risultato va nel campo checksum. Con le parole `0110011001100000`, `0101010101010101`, `1000111100001100`: la somma con end-around carry dà `0100101011000010`, il cui complemento a 1 è il checksum `1011010100111101`. Il receiver somma tutte le parole del messaggio, checksum incluso: se il risultato è `1111111111111111` (16 uni) il check passa; se anche un solo bit è 0, sono stati introdotti errori nel pacchetto.

Il checksum permette di **rilevare** gli errori, non di correggerli: un datagram con checksum invalido viene scartato silenziosamente, senza alcuna ritrasmissione.

### Limiti del checksum

Il checksum non può rilevare tutti gli errori di trasmissione, perché contenuti diversi possono produrre lo stesso checksum.

- Un checksum a N bit rileva con certezza l'errore su un **singolo bit**, ma non garantisce la rilevazione di due o più errori.
- **Errori che si compensano**: due bit flip con variazioni opposte si annullano lasciando la somma invariata (es. una parola che passa da `0001` a `0000` e un'altra da `0000` a `0001`).
- **Riordino**: scambiare due parole da 16 bit non cambia la loro somma, quindi l'errore non viene rilevato.
- È computazionalmente economico e cattura molti errori comuni, ma non garantisce l'integrità; meccanismi più robusti come il **CRC** (*Cyclic Redundancy Check*) sono usati al livello data link.

### Perché un controllo end-to-end

Info

**Errore end-to-end**

I controlli del livello link non bastano: non tutti i collegamenti offrono gli stessi controlli di integrità, e i dati possono corrompersi dentro i dispositivi intermedi (es. in un router) anche quando i controlli link-by-link sui frame riescono. Per il **principio end-to-end**, i controlli di integrità che devono valere tra gli endpoint non possono affidarsi esclusivamente ai componenti intermedi; i controlli link e quello di trasporto proteggono il dato in punti diversi del percorso e sono complementari, nessuno dei due sostituisce l'altro.

### Consegna, perdita e ordinamento

UDP fornisce un servizio connectionless *best-effort*.

- **Nessuna garanzia di consegna**: i datagram possono perdersi durante la trasmissione.
- **Nessuna ritrasmissione**: i datagram persi o corrotti non vengono rinviati automaticamente.
- **Nessun ordine**: i datagram possono arrivare in un ordine diverso da quello di invio.
- **Nessuna soppressione dei duplicati**: non è garantito che un datagram sia consegnato una sola volta.

Le applicazioni che richiedono affidabilità devono implementare meccanismi aggiuntivi o usare un protocollo di trasporto come TCP.

### Buffer delle socket e perdita dei datagram

I datagram in arrivo vengono smistati dal demultiplexing nel **buffer di ricezione della socket** (spazio kernel) e l'applicazione li preleva uno alla volta con `recvfrom()`.

- Se i datagram arrivano più velocemente di quanto l'applicazione li elabora, il buffer di ricezione si può riempire e i datagram in arrivo possono essere scartati.
- Se il buffer dell'applicazione è troppo piccolo, il datagram viene troncato e i byte rimanenti scartati; il buffer kernel deve invece accoglierlo completo, perché UDP preserva i confini del messaggio.
- Una `sendto()` di successo non garantisce che il datagram abbia raggiunto l'applicazione destinataria; UDP non ha controllo di flusso, quindi il sender non viene rallentato quando il receiver non tiene il passo.

### Percorso di ricezione in Linux

Il percorso di un datagram nello stack di rete del kernel segue questi passi.

1. La scheda di rete riceve il frame e lo memorizza in memoria kernel.
2. Il livello IP elabora il pacchetto e riassembla i frammenti se c'è stata frammentazione.
3. Il livello UDP controlla intestazione e checksum, poi fa il demultiplexing del datagram verso la socket appropriata.
4. Il datagram entra nella coda di ricezione della socket.
5. L'applicazione legge il datagram con `recvfrom()`, che copia i dati nel buffer applicativo.

### Affidabilità a livello applicazione

UDP non fornisce riconoscimenti, ritrasmissioni né consegna in ordine, ma le applicazioni possono implementare questi meccanismi quando servono.

- **Sequence number** per identificare i messaggi e rilevare dati mancanti o duplicati.
- **Acknowledgment** per confermare la ricezione avvenuta.
- **Timeout e ritrasmissioni** per recuperare le risposte mancanti.
- **Controllo della congestione** per regolare il traffico immesso nella rete.

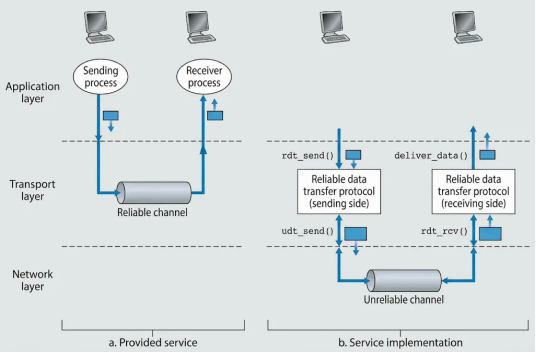
Il client DNS può ritentare le query senza risposta; **QUIC** implementa affidabilità e controllo della congestione sopra UDP e supporta HTTP/3. UDP fornisce il servizio di trasporto minimo, lasciando le funzionalità aggiuntive ai protocolli di livello superiore.

# Lezione 14 Trasferimento affidabile dei dati

## Il problema del trasferimento affidabile

Il problema generale è come ottenere un protocollo affidabile sopra un canale che non lo è, indipendentemente dallo strato considerato: le tecniche valgono anche per livelli diversi dal trasporto e servono da base per parlare di TCP.

- **Obiettivo**: garantire che i dati arrivino corretti, completi e nell'ordine originale.
- **Sfida**: il canale di comunicazione sottostante può corrompere bit o perdere pacchetti.
- **Astrazione**: fornire ai livelli superiori un servizio di comunicazione affidabile anche quando il canale sottostante è inaffidabile.
- **Ambito**: un problema fondamentale ai livelli link, trasporto e applicazione, non specifico di TCP; l'assunzione realistica è che il livello di rete sottostante sia inaffidabile, come IP sotto TCP.




Nel caso ideale di un canale perfettamente affidabile (`rdt1.0`) non serve alcun meccanismo: il sender trasmette i pacchetti e il receiver li consegna immediatamente ai livelli superiori, senza complicazioni aggiuntive. Il controllo d'errore (es. con checksum) permette al receiver di capire se il messaggio è corrotto, ma non basta: la comunicazione affidabile richiede tre garanzie.

- Nessun bit trasmesso è **corrotto** (cambiato da 0 a 1 o viceversa).
- Nessun bit è **perso o ripetuto**.
- Tutti i bit sono consegnati nell'**ordine esatto** di invio.

Si considera il **trasferimento dati unidirezionale**, dal sender al receiver; i pacchetti di controllo (ACK/NACK) devono però viaggiare anche in direzione opposta: flusso unidirezionale $\neq$ comunicazione unidirezionale. Entrambe le parti usano servizi di trasmissione inaffidabile per inviare pacchetti sul canale inaffidabile.

La progressione da canale perfetto a canale reale riassume ciò che il protocollo deve aggiungere a ogni passo.

| Cosa fa il canale | Cosa deve aggiungere il protocollo |
|---|---|
| niente va storto | niente: si crea il pacchetto e lo si invia |
| i bit possono corrompersi | checksum, riconoscimenti, ritrasmissione |
| anche il riconoscimento può corrompersi | un numero di sequenza, per riconoscere una copia |
| i pacchetti possono perdersi | un timer al sender e la ritrasmissione che innesca |

L'ultima riga è il **protocollo alternating bit**: checksum, numero di sequenza a un bit, riconoscimenti numerati, un timer; è corretto su ogni canale descritto, e resta da misurare quanto sia lento.

## Stop-and-wait e ARQ

Primo caso: canale in cui i pacchetti possono solo **corrompersi**, non perdersi. La corruzione è un problema tipico, dovuto ai componenti fisici della rete: un pacchetto viene trasmesso, propagato o bufferizzato più volte durante la comunicazione.

Definition

**Stop-and-wait**

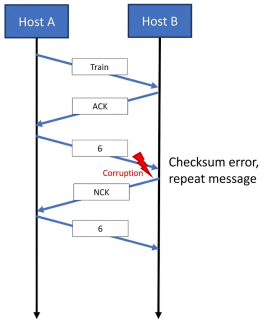
**Protocollo** in cui ogni messaggio deve essere riconosciuto prima di inviarne uno nuovo: si manda un messaggio alla volta e si attende il riscontro, **ACK** (messaggio ricevuto intatto) oppure **NACK** (errore occorso, il messaggio va ripetuto).

Definition

**ARQ**

**Famiglia di protocolli** basati sulla ritrasmissione, detti *Automatic Repeat reQuest*: se arriva un NACK, il messaggio viene rinviato. Per funzionare richiedono tre meccanismi, nessuno dei quali è opzionale, e i tre insieme non bastano ancora.

- **Error detection**: bit aggiuntivi (il checksum) perché il receiver noti che il messaggio è cambiato.
- **Receiver feedback**: il sender non vede la destinazione, può sapere solo se gli viene detto; in principio il feedback è un bit, un valore per sì e uno per no.
- **Retransmission**: ciò che è arrivato con errore viene rinviato; non esiste altra riparazione.




Lo schema ha però due problemi: cosa succede se si corrompe l'ACK/NACK stesso, e come fa il receiver a capire che un messaggio è la ripetizione di uno precedente e non uno nuovo? La risposta è aggiungere un **sequence number** all'header dei pacchetti.

- Nello stop-and-wait c'è sempre **un solo messaggio in transito**: non serve l'intera sequenza, basta distinguere il pacchetto corrente dal precedente.
- Si aggiunge un campo `S = 0/1` che **alterna**: un valore ripetuto segnala un messaggio ripetuto.
- Il riconoscimento riporta **lo stesso bit**, così indica a quale messaggio risponde; se un NACK arriva corrotto, il sender ritrasmette e il receiver riconosce dal numero di sequenza se la ritrasmissione riguarda il pacchetto vecchio o uno nuovo.

### ACK numerati

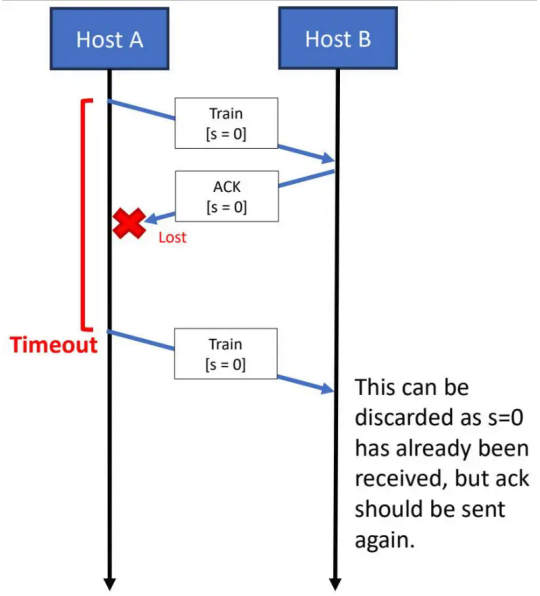
I riconoscimenti numerati rendono i NACK superflui.

- Un **ACK numerato** identifica l'ultimo pacchetto ricevuto con successo.
- Se il sender riceve un ACK per il pacchetto precedente, il pacchetto corrente non è stato riconosciuto.
- Il receiver risponde a pacchetti corrotti e duplicati **ripetendo l'ultimo ACK**: così l'ACK fornisce sia feedback positivo che negativo, eliminando i NACK espliciti.

## Perdita di pacchetti e timeout

Secondo caso: canale in cui i pacchetti possono anche **perdersi**, cosa plausibile perché i dispositivi di rete possono andare in buffer overflow sotto traffico intenso. Lo stop-and-wait ha un chiar problema di **loop**: perso un messaggio oppure un acknowledgment, il sender non riceve mai lo stimolo per inviare il successivo e la comunicazione resta ferma per sempre; restare in attesa all'infinito non ha via d'uscita.

- **Timeout**: soluzione semplice ed efficace, un timer lato sender che, scaduto, permette di ritentare; il protocollo fa ritrasmettere lo stesso pacchetto, con lo stesso numero di sequenza.
- Il duplicato è riconosciuto dal numero di sequenza, scartato, e riconosciuto di nuovo; il receiver sa di averlo già ricevuto e si limita a rimandare l'ACK.
- Il meccanismo funziona sia nel caso di perdita del pacchetto sia dell'acknowledgment; una query DNS su UDP fa esattamente questo: nessuna risposta in tempo, si chiede di nuovo.




La sfida è stimare un timeout adeguato, che dipende dall'**RTT** (*Round-Trip Time*), il tempo che un pacchetto impiega per andare e tornare.

- Un timeout **troppo lungo** rallenta la comunicazione: si aspetta inutilmente, con più sicurezza ma tempi peggiori.
- Un timeout **troppo breve** fa sovrapporre i pacchetti: si rinvia mentre l'ACK stava già tornando.
- Un timeout ragionevole deve quindi essere in qualche modo più lungo dell'RTT; in questi protocolli teorici è fisso: sono strumenti concettuali, che gli algoritmi reali poi usano in modo più mirato, e il protocollo deve restare robusto tenendo conto di tutto ciò che può avvenire.

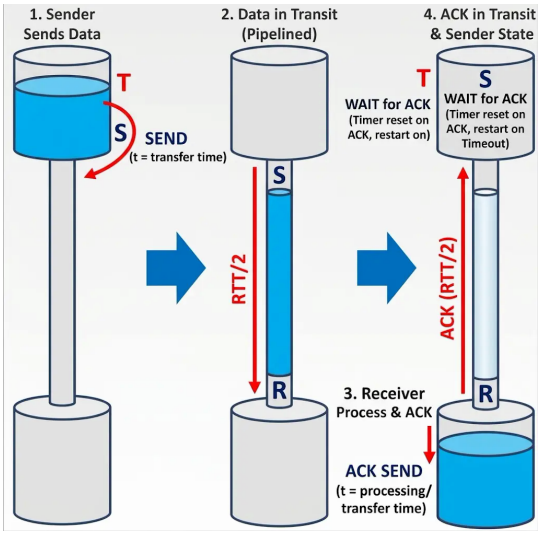
Warning

**Il paradosso del timeout breve**

Un timer più corto dell'andata e ritorno non lascia scampo: l'acknowledgment non può arrivare prima che il timer scatti, quindi ogni messaggio viene ritrasmesso su un canale che non sta perdendo nulla, come il timer da 50 ms su un percorso da 80 ms, dove tutto va due volte e il numero di sequenza al receiver è l'unica ragione per cui il programma funziona. L'effetto paradosso è che un timeout più breve può **ritardare** la sequenza: ogni ritrasmissione rimette la finestra d'attesa e, se l'ACK arriva sistematicamente in ritardo, il ciclo si ripete fin quasi ad allungare la faccenda all'infinito.

## Prestazioni dello stop-and-wait

Il tempo totale di consegna si ragiona con l'analogia dell'acqua: c'è il tempo `t` per versare l'acqua dal serbatoio nel tubo e il tempo `RTT/2` perché il fluido attraversi il tubo; il tempo di versata si conta **una sola volta**, perché emissione e ricezione si sovrappongono (simmetria tra i due lati): l'ultima goccia arriva al serbatoio di destinazione a `RTT/2 + t`.



Il caso di scuola: due host sulle coste opposte degli Stati Uniti, con RTT alla velocità della luce di circa 30 ms, un canale con tasso di trasmissione R = 1 Gbps e un pacchetto di L = 1000 byte (8000 bit).

- Il tempo per trasmettere il pacchetto sul canale è $t = L/R = 8000/10^9 = 8$ µs.
- L'ultimo bit entra nel canale a `t0 + 0,000008` s e il pacchetto raggiunge il receiver a `RTT/2 + L/R = 0,015008` s.
- Con ACK di dimensione trascurabile (tempo di invio ignorato), l'ACK riemerge al sender a `RTT + L/R = 0,030008` s.
- In 0,030008 s di tempo totale, il sender ha trasmesso per soli 8 µs: è rimasto ad attendere il **99,973%** del tempo, senza sfruttare minimamente la banda disponibile.

**Formula**: l'utilizzazione è la frazione del tempo totale in cui il sender trasmette, `U = (L/R) / (RTT + L/R)`; su un percorso da 100 ms di RTT con un messaggio da 1 KB e un link a 100 Mbps, `L/R` = 80 µs, quindi `U = 0,0008` e la chat si muove intorno a 80 kbps. Su loopback e in rete locale l'RTT è una frazione di millisecondo e il problema non si vede; dieci messaggi al secondo vanno bene per una chat, ma non è un modo per muovere un file, e non è ciò che fa TCP.

# Pipelining

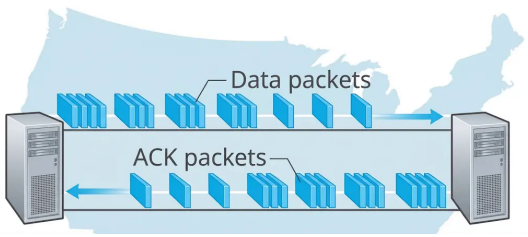
Definition

**Pipelining**

**Tecnica** in cui, invece dello stop-and-wait, al sender è permesso inviare **più pacchetti** senza attendere i riconoscimenti.

Il pipelining impone due cambiamenti rispetto allo stop-and-wait.

- Il **range dei numeri di sequenza deve aumentare**: ogni pacchetto in transito (escluse le ritrasmissioni) deve avere un numero di sequenza univoco, e possono esserci più pacchetti in transito non riconosciuti; i pacchetti sparati a raffica possono prendere strade diverse, quindi gli ACK devono riferirsi al pacchetto giusto.
- Servono **buffer** per memorizzare i pacchetti in arrivo: non si sa se sono corretti né se ci sono "buchi" nella trasmissione, e si vuole che dopo il primo arrivi il secondo, poi il terzo e così via.




Già mandarne tre alla volta invece di uno dà un piccolo miglioramento: il sender inizia a `t0`, l'ultimo bit dell'ultimo pacchetto entra nel canale a `t0 + 3L/R = 0,000024` s e, con ACK trascurabile, l'ultimo riconoscimento torna al sender a `RTT + L/R = 0,030024` s; l'attesa scende dal 99,973% al 99,920%. Il guadagno è minimo, ma cambia il paradigma: ci si muove verso il pipelining. I due protocolli base che lo usano sono **Go-Back-N** e **Selective Repeat**, nell'ordine di complessità; nelle forme viste non sono implementati così in pratica, ma forniscono gli strumenti concettuali a cui i protocolli reali si rifanno.

# Go-Back-N

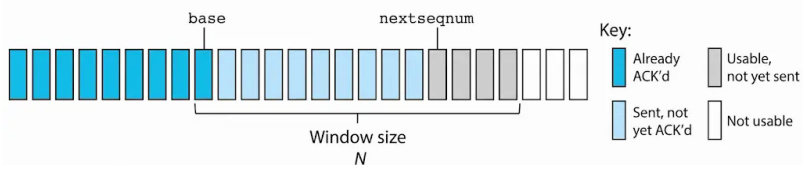
Definition

**Go-Back-N**

**Protocollo sliding window** (detto anche *sliding-window protocol*): il sender può trasmettere più pacchetti quando disponibili senza attendere l'acknowledgment, ma con il vincolo di avere **al massimo N pacchetti non riconosciuti**. La gestione della pipeline è tutta sul sender, mentre al receiver resta pochissimo.

Il sender mantiene una finestra di pacchetti descritta da due indici: la **base**, il numero di sequenza del pacchetto non riconosciuto più vecchio, e **nextseqnum**, il più piccolo numero di sequenza non ancora usato (prossimo da inviare).

- I pacchetti in `[0, base-1]` sono trasmessi e riconosciuti: storia passata, fuori dalla finestra.
- I pacchetti in `[base, nextseqnum-1]` sono inviati ma non ancora riconosciuti.
- I pacchetti in `[nextseqnum, base+N-1]` possono essere inviati.
- I pacchetti da `base+N` in poi non sono utilizzabili finché non arriva un nuovo acknowledgment: a finestra piena il sender si ferma in attesa.

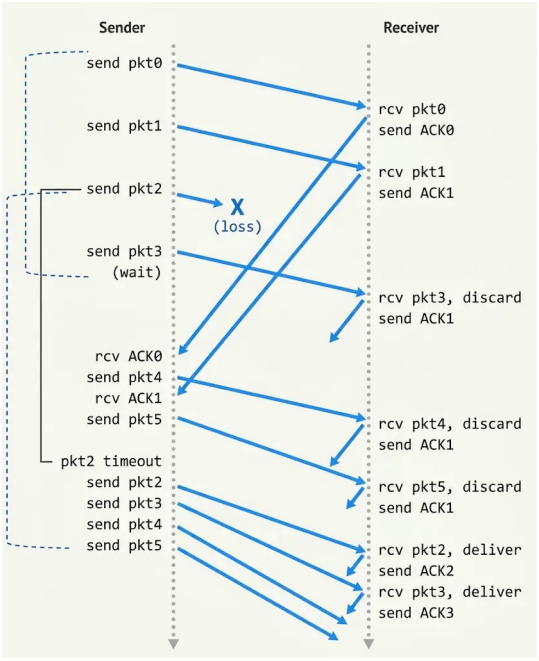



Lato sender valgono tre meccanismi.

- **ACK cumulativi**: l'acknowledgment per il pacchetto `n` significa che tutto fino a `n` incluso è arrivato correttamente; un acknowledgment perso si ripara da solo, perché l'ACK 5 rende superflui gli ACK 3 e 4 e la finestra salta avanti.
- **Timer unico**: appartiene al pacchetto non riconosciuto più vecchio, non a ogni singolo pacchetto; alla scadenza il sender rinvia **tutti** i pacchetti in attesa (da qui il nome Go-Back-N).
- **Finestra scorrevole**: quando arriva un acknowledgment la finestra avanza e il timer viene riavviato.

Il receiver mantiene un solo numero: **expectedseqnum**, il numero di sequenza del prossimo pacchetto che è disposto a consegnare ai livelli superiori.

- Pacchetto **corretto e in ordine**: consegnato all'applicazione, riconosciuto, e si avanza di uno.
- **Qualsiasi altro pacchetto**: scartato, anche se perfettamente valido e arrivato in anticipo, buttato esattamente come se fosse danneggiato, e ri-riconosciuto l'ultimo pacchetto in ordine ricevuto.
- **Nessun buffer**: la complessità è deliberatamente tutta da un lato, il sender tiene una finestra, il receiver tiene un intero.




Con una finestra di quattro i pacchetti 0–3 escono in raffica, poi il sender si ferma perché la finestra è piena. `pkt2` si perde: i pacchetti 3, 4 e 5 arrivano corretti ma vengono scartati, perché il 2 non è stato consegnato; il receiver continua a rispondere ACK1, l'ultimo effettivamente consegnato all'applicazione. Scaduto il timer di `pkt2`, il più vecchio in attesa, i quattro pacchetti in coda vengono ritrasmessi, tornando all'ultimo ricevuto con successo.

I limiti del protocollo sono pesanti.

- **Una perdita, N ritrasmissioni**: i pacchetti corretti buttati via li pagano sender, receiver e canale.
- **Reazione a catena**: gli out-of-order scartati forzano ritrasmissioni successive, e più è larga la finestra più costa una singola perdita; il comportamento è peggiore proprio dove il pipelining vale di più, cioè finestra grande su un percorso veloce e lungo.
- La **ritrasmissione stessa** può perdersi o corrompersi, richiedendo a sua volta altre ritrasmissioni; le ritrasmissioni continue finiscono inoltre per intasare la rete.

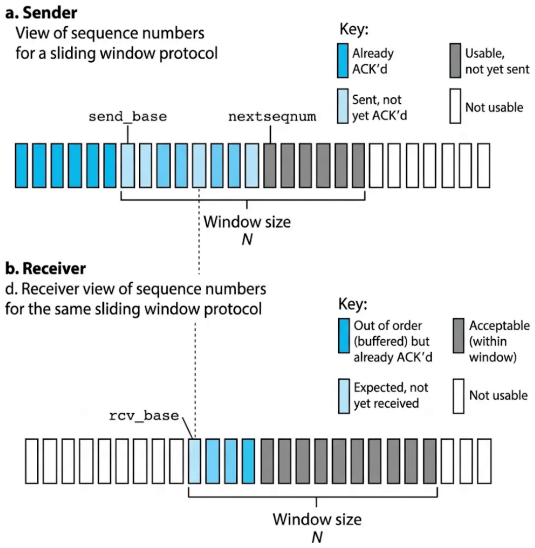
## Selective Repeat

**Definition**

**Selective Repeat**

**Protocollo sliding window** in cui il sender ritrasmette **solo i pacchetti probabilmente ricevuti con errore** (persi o corrotti); a differenza di Go-Back-N, il receiver **bufferizza e riconosce** i pacchetti fuori ordine invece di scartarli.

Si usano **due finestre di dimensione N** per limitare i pacchetti non riconosciuti, ciascuna con la propria base: **send_base**, il bordo inferiore della finestra del sender (posizione più vecchia non ancora riconosciuta), e **rcv_base**, il bordo inferiore della finestra del receiver (prossimo pacchetto necessario per la consegna in ordine).




Lato sender valgono tre regole.

- **Invio**: se il numero di sequenza cade nella finestra, il pacchetto viene creato e inviato, fino a finestra piena.
- **Un timer per pacchetto**: alla scadenza si rinvia quel solo pacchetto, e solo quello.
- **ACK `n`**: marca `n` come ricevuto; se `n` era la base della finestra, questa scorre fino al prossimo numero non riconosciuto, saltando in avanti di tutti i pacchetti consecutivi già riconosciuti.

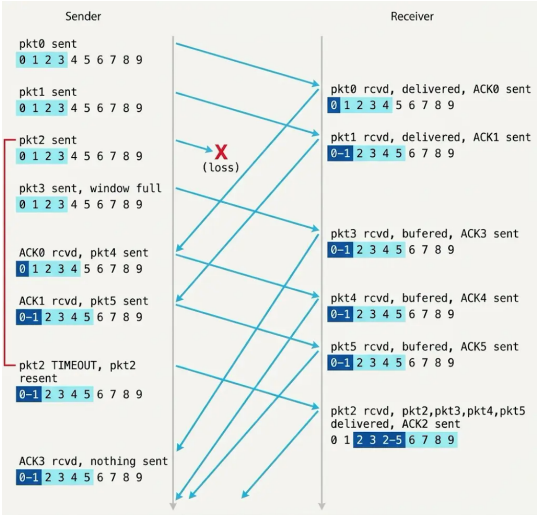
Lato receiver valgono tre regole complementari.

- Un pacchetto **dentro la finestra** viene riconosciuto e, se nuovo, bufferizzato.
- Se è la **base** della finestra, viene consegnato all'applicazione insieme a tutti i pacchetti consecutivi già bufferizzati, e la finestra scorre fino a dove la contiguità è garantita.
- Un pacchetto **poco sotto la finestra** viene riconosciuto di nuovo e scartato.

Info

### Perché riconoscere ciò che è già sotto la finestra

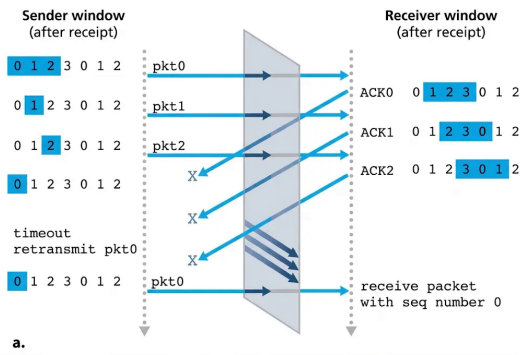
Il receiver ha già quel pacchetto, consegnato tempo fa: perché rispondere? Perché il sender non sa che è arrivato. Se l'acknowledgment del pacchetto più vecchio non riconosciuto si è perso, il sender continuerebbe a ritrasmetterlo e la sua finestra non avanzerebbe mai. Le due parti non condividono la vista di ciò che è arrivato e, in Selective Repeat, questo scarto è permanente, non temporaneo: il silenzio non è mai una risposta accettabile a una ritrasmissione, perché è indistinguibile da una seconda perdita. Quindi il receiver ri-riconosce e scarta.




In presenza di pacchetti persi il comportamento è il seguente: il receiver bufferizza `pkt3`, `pkt4` e `pkt5` mentre attende il `pkt2` perduto; solo il `pkt2` viene ritrasmesso, sul proprio timer, e nient'altro esce due volte; quando arriva, quattro pacchetti salgono all'applicazione in un colpo solo, in ordine, evitando di rinviare 3, 4 e 5 e quindi ulteriori perdite o errori di trasmissione.

### Dimensione della finestra e ambiguità

Un problema del Selective Repeat è che la finestra è vincolata allo spazio dei numeri di sequenza, e questo spazio è **finito**.




Scenario con quattro numeri di sequenza (massimo 3) e finestra di dimensione 3: i pacchetti da 0 a 2 arrivano correttamente e i relativi ACK vengono inviati ma non ancora ricevuti; la finestra del receiver avanza comunque fino al sesto pacchetto, cioè su `[3, 0, 1]`.

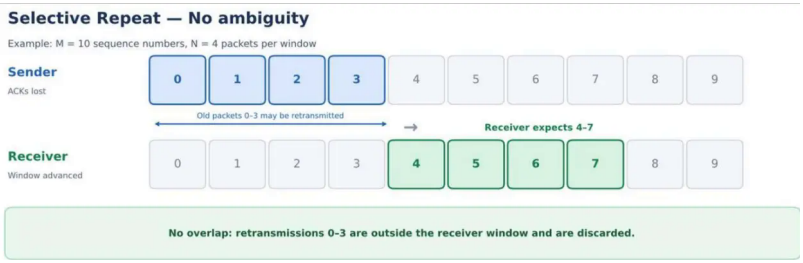
- **Caso a**: gli ACK dei primi tre pacchetti si perdono e il sender li ritrasmette, il pacchetto 0 è nuovo o vecchio?
- **Caso b**: gli ACK arrivano, il sender invia 3, 0, 1 e il pacchetto 3 si perde; il receiver vede un pacchetto numerato 0 dentro la sua finestra e questa volta è dato nuovo, il pacchetto 0 è nuovo o vecchio?

I due casi producono la stessa sequenza di arrivi e sono indistinguibili: il receiver non può "vedere" il sender, quindi non può decidere se acquisire o scartare il pacchetto. Il protocollo non deve avere ambiguità di questo tipo.

Il vincolo che risolve il problema è il **massimo disallineamento delle finestre**.

- Il sender può trasmettere al più N pacchetti non riconosciuti.
- Se tutti gli N pacchetti arrivano ma i loro ACK si perdono, il receiver avanza di N posizioni mentre il sender resta bloccato sulla sua base.
- Il receiver non può avanzare oltre: senza ACK il sender non trasmette nuovi pacchetti, e il blocco del sender non alimenta lo spostamento del receiver.
- Quindi la massima distanza tra le due finestre è di **N posizioni**.

Per evitare la sovrapposizione tra finestre vecchie e nuove, lo spazio dei numeri di sequenza M deve coprire due finestre: **`M ≥ 2N`**. Con finestre di dimensione 4 servono così numeri da 0 a 7: solo quando lo 0 è uscito dalla finestra può ricomparirne uno davanti, e i due casi diventano distinguibili.



Capitolo 15

# Lezione 15 Transport Layer TCP

## TCP rispetto a UDP

**Definition**

**TCP**

**Transmission Control Protocol**: protocollo di trasporto **orientato alla connessione**, **affidabile** e **consapevole di congestione e flusso**.

- **Orientato alla connessione**: prima della trasmissione c'è una fase di **handshake** che assicura che i processi mittente e ricevente siano "connessi".
- **Affidabile**: implementa rilevamento errori, ritrasmissioni, acknowledgment, timer e numeri di sequenza.
- **Consapevole di congestione e flusso**: regola il ritmo di trasmissione in base alle prestazioni del ricevente (flusso) e della rete (congestione).

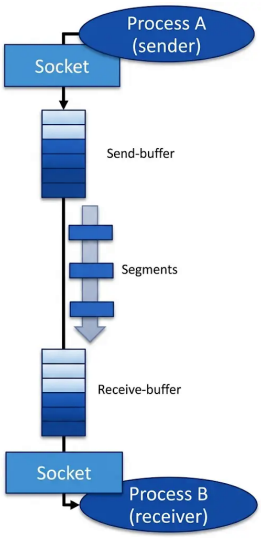
L'orientamento alla connessione rende TCP **full-duplex** e **point-to-point**:

- **Full-duplex**: se A è connesso con B, allora B è connesso con A.
- **Point-to-point**: un solo mittente e un solo destinatario; non è possibile trasmettere da un mittente a più destinatari (UDP invece permette il multicasting).

TCP è orientato allo scambio di **flussi di dati** (*data stream*): più segmenti TCP possono essere parti di un flusso più grande (sequenza ordinata di dati), mentre i singoli datagram UDP sono per lo più scollegati.

## Buffer e Maximum Segment Size

Le operazioni di invio e ricezione in TCP si basano fortemente su **buffer**, che disaccoppiano parzialmente i tempi di trasmissione da: >[!aside] > - Ritardi dell'applicazione. > - Ritardi del SO nel multiplexing/demultiplexing dei pacchetti. > - Ritardi di rete (oscillazioni delle prestazioni). > > Esiste anche il limite del ritmo di trasmissione: messaggi lunghi possono essere spezzati in segmenti più piccoli, raccolti nei buffer prima di essere **disassemblati** (lato mittente) e **riassemblati** (lato ricevente). > > — >




>

**Definition**

**MSS**

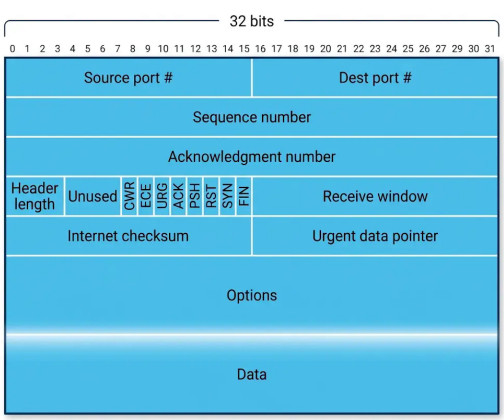
**Maximum Segment Size**: quantità massima di dati dentro un segmento, legata all'**MTU** (*Maximum Transmission Unit*, lunghezza massima del frame accettabile dal livello link, es. 1500 byte in Ethernet) dalla relazione MSS + 40 = MTU, dove 40 byte sono le dimensioni combinate delle header TCP e IP (tipicamente 20+20).

- All'invio i segmenti sono creati dai dati applicativi e passati al livello rete, dove sono incapsulati separatamente in datagram IP.
- Alla ricezione i dati sono messi nel **receive buffer**, da cui l'applicazione legge quando è pronta.

## Il segmento TCP

Il segmento TCP consiste di campi di header e di un campo dati (al massimo di MSS byte). Campi principali:

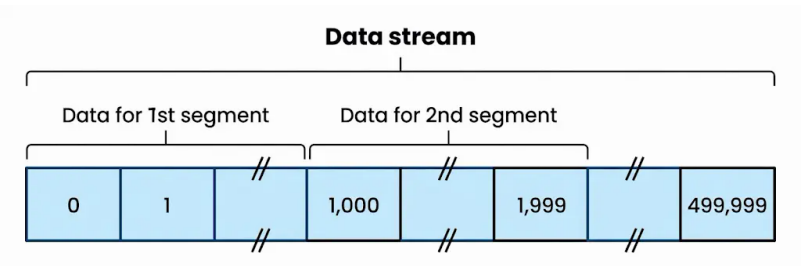
- **Sequence number** (32 bit): per il trasferimento affidabile.
- **Acknowledgment number** (32 bit): per il trasferimento affidabile.
- **Receive window** (16 bit): numero di byte che il ricevente è disposto ad accettare (controllo di flusso).
- **Header length** (4 bit): numero di parole da 32 bit nella header; la header TCP è **variabile** per via del campo opzioni.
- **Options** (K bit): processi opzionali, come la negoziazione dell'MSS o i timestamp.
- **Flags** (6/12 bit):
  - **ACK**: il segmento è un ACK (il campo acknowledgment number è in uso).
  - **RST**, **SYN**, **FIN**: apertura e chiusura della connessione.
  - **CWR**, **ECE**: notifica esplicita di congestione (opzionale).
  - **PSH**: dice al ricevente di passare subito i dati all'applicazione (raro).
  - **URG**: il segmento contiene dati "urgenti" (raro), con l'**Urgent data pointer** (16 bit) che indica la posizione dell'ultimo byte urgente.
- **Checksum** (16 bit): controllo di integrità.




## Numeri di sequenza e acknowledgment

I numeri di sequenza servono anche a gestire la **segmentazione**: un grande flusso di dati va spezzato in "pezzi" di dimensione dipendente dall'MSS, e ogni pezzo entra in un segmento.

- Il **numero di sequenza** di un segmento è convenzionalmente la **posizione che il primo byte del suo dato occupa nel flusso**.
- Il **numero di acknowledgment** è convenzionalmente il **prossimo pezzo del flusso atteso** (calcolato dal numero di sequenza del segmento precedente).
- TCP non sa nulla dei dati trasmessi: dal suo punto di vista i dati sono solo un **flusso ordinato di byte** (è un protocollo di trasporto general-purpose).




**Example**

Per un flusso da 500.000 byte (~500 KB) con MSS di 1000 byte, TCP costruisce 500 segmenti: il primo ha numero di sequenza 0, il secondo 1000, il terzo 2000, e così via.

Trasferimento A → B (partendo dal numero di sequenza 0):

- Il ricevente riceve il primo segmento da 1000 byte (byte 0–999) e conferma con **acknowledgment number 1000** (prossimo byte atteso nel flusso).
- Poi riceve il segmento 1000–1999, e così via.

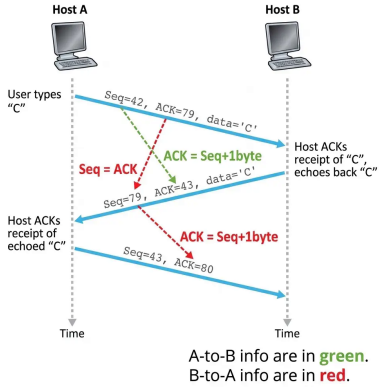
**Full-duplex**: se A e B comunicano, esistono **due flussi** da considerare per l'affidabilità (A→B e B→A), e i numeri si usano in coppia:

- I segmenti di A hanno numeri di sequenza relativi al flusso A→B e acknowledgment relativi al flusso B→A.
- I segmenti di B hanno numeri di sequenza relativi al flusso B→A e acknowledgment relativi al flusso A→B.

Example

**Echo di un byte:**

**A** trasmette il carattere `c` (byte 42 del proprio flusso) con acknowledgment 79 (prossimo atteso dal flusso B→A); **B** risponde echoando `c` con numero di sequenza 79 e acknowledgment 43 (42 + 1); **A** invia infine un **pure ACK** con sequenza 43 e acknowledgment 80.
I segmenti che portano dati e ACK insieme hanno il flag `ACK = 1`.




## Stima dell'RTT e timeout di ritrasmissione

Il meccanismo di affidabilità di TCP fa largo uso di **timeout**: nell'approccio a pipelining si ritrasmette un segmento solo se l'ACK relativo non arriva prima della scadenza.

- La stima del timeout è **critica**: un timeout troppo lungo rallenta la comunicazione, uno troppo corto fa sovrapporre i pacchetti (e ritrasmette pacchetti inutili, generando traffico inutile).
- Il timeout deve essere **maggiore dell'RTT**, altrimenti si generano ritrasmissioni non necessarie.

TCP stima l'RTT **campionando** i segmenti confermati con successo e **non ritrasmessi** (one-shot). Il tempo campionato (*SampleRTT*) oscilla (congestione, carico del ricevente), quindi la stima è una **media mobile esponenziale** (EWMA):

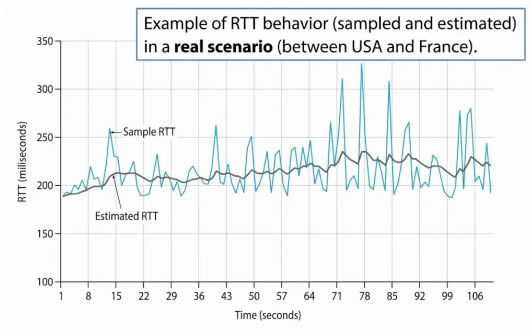
$$\text{EstimatedRTT} = (1-\alpha)\,\text{EstimatedRTT} + \alpha\,\text{SampleRTT}$$

con $\alpha = 0.125$ (cioè 1/8).

Anche la **variabilità** dell'RTT entra nel calcolo:

$$\text{DevRTT} = (1-\beta)\,\text{DevRTT} + \beta\,|\text{SampleRTT} - \text{EstimatedRTT}|$$

con $\beta = 0.25$ (cioè 1/4).




L'**intervallo di timeout** è:

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$$

La deviazione fornisce un margine **ragionevole e adattivo** rispetto all'RTT stimato, esso cresce quando l'RTT oscilla, quindi la finestra di timeout è più larga quando si è incerti. Il valore iniziale di default dell'RTT a $t = 0$: **1 secondo**.

## Fast retransmit

La ritrasmissione a timeout è efficace, ma ha un limite fondamentale che ha motivato l'introduzione di un meccanismo di segnalazione più rapido.

- Il periodo di timeout è **relativamente lungo**: in attesa della scadenza, la connessione resta **stallata**.
- **Fast retransmit**: è il **ricevente** a segnalare al mittente una probabile perdita, senza aspettare il timeout, tramite **ACK duplicati**.

### Come funziona

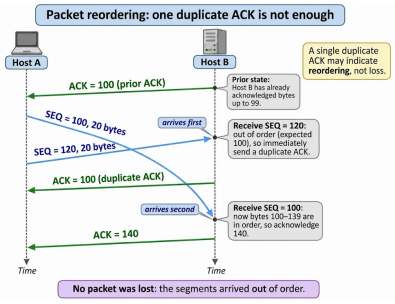
Il meccanismo si svolge in tre passi.

1. Il ricevente TCP riceve un segmento con numero di sequenza **maggiore di quello atteso** → un segmento intermedio è presumibilmente andato perso.
2. Il ricevente **reinvia lo stesso ACK di prima** (duplicato), che riporta il numero del **prossimo byte atteso**.
3. Quando il mittente riceve **N ACK duplicati** (tipicamente **3**, soglia definita in RFC 5681), assume che il segmento successivo all'ultimo ACK non duplicato sia perso e lo **ritrasmette subito**, ben prima della scadenza del timeout.

Warning

**Perché aspettare 3 duplicati**

Gli ACK duplicati possono nascere anche **senza perdite**: se due segmenti arrivano **scambiati di ordine** (riordino nella rete), il ricevente emette 1 ACK duplicato prima di accorgersi dello scambio. Reagire al **primo** duplicato causerebbe ritrasmissioni inutili o sbagliate. La soglia di 3 fa da **filtro** contro il riordino occasionale.




### Dopo la ritrasmissione

Completata la ritrasmissione, la ripresa della trasmissione dipende dalla politica di ritrasmissione adottata.

- All'arrivo del segmento ritrasmesso, il ricevente invia l'ACK del **prossimo segmento atteso**, secondo la propria politica (go-back-N o selective repeat).

### TCP: Go-Back-N o selective repeat?

A prima vista TCP assomiglia a entrambi, ma il confronto va fatto punto per punto.

- **Come Go-Back-N**: ACK **cumulativi**, nessuna risposta individuale per ciò che arriva in anticipo, **un solo timer** e due variabili.
- **A differenza di Go-Back-N**: al timeout si ritrasmette **un solo segmento**, non tutto ciò che è in attesa; e nemmeno quello, se l'ACK successivo lo copre già.
- **Come selective repeat**: il ricevente **buffera** ciò che arriva in anticipo invece di scartarlo.
- **Conclusione**: TCP non è né l'uno né l'altro; ACK cumulativi su un ricevente che **conserva** i dati, cioè **la metà utile di ciascuno**.

Info

**Selective Acknowledgement (SACK)**

Un'opzione, il **selective acknowledgement**, permette al ricevente di **nominare i blocchi** che sta trattenendo; e porta TCP **fino in fondo** verso il selective repeat.